AN INTUITIVE INTRODUCTION TO MACHINE LEARNING



SÎNICĂ ALBOAIE

© 2026 Sînică Alboaie. This first iteration was donated to ScriptaHub (scriptahub.com) by Sînică Alboaie. Feedback and public contributions are warmly invited. Educational, non-commercial use is permitted.

An Intuitive Introduction to Machine Learning

With examples from everyday life and from an engineer's desk

Oct 14, 2026 · Sînică Alboaie

Contents

No.	Title
	Preface
1	**The Shift in Thinking**
1.1	The Recipe and the Apprentice
1.2	The Answers on the Back of the Page
1.3	Sorting the Closet in the Dark
1.4	The Exam You Are Not Allowed to See
2	**The Classic Toolbox**
2.1	A Ruler, Some Springs and a Dimmer Switch
2.2	Tell Me Who Your Neighbors Are
2.3	Twenty Questions
2.4	The Wisdom of Crowds, and of Editors
2.5	The Aisles We Walked Past
3	**Deep Learning**
3.1	The Neuron That Was Promised Too Much
3.2	Down the Mountain in the Fog
3.3	What the Cat’s Brain Revealed
3.4	The Telephone Game
4	**What a Model Is Made Of**
4.1	It Is Not a Brain
4.2	Turning the World into Numbers
4.3	Everything Is a Table
4.4	What Matters, and Who Decides
5	**How Models Are Built and Run**
5.1	The Box of Lego
5.2	The Two Lives of a Model
5.3	A Different Kind of Memory
5.4	How Much Is Enough
6	**The Other Machine**
6.1	A Thousand Models of a Coffee Cup

No.	Title
6.2	The Prediction Machine
7	**Language and Meaning**
7.1	A Map Where Meaning Has an Address
7.2	The Detectives' Meeting
7.3	The Game of Guessing the Next Word
7.4	The Open-Book Exam
7.5	Living on a Whiteboard
7.6	Why Guessing the Next Word Goes So Far
8	**Making Pictures, Building Systems**
8.1	Learning to Draw
8.2	Blueprints
8.3	Data With a Shape of Its Own
8.4	What the Plumbing Hides
9	**Ways of Learning**
9.1	A Dog, a Maze and a Boat That Would Not Finish the Race
9.2	The Many Ways to Teach
10	**How to Fail Successfully**
10.1	The Crammer and the Slacker
10.2	The Horse Who Could Count
10.3	The 99.9% Detector and the Smoke Alarm
10.4	The Funhouse Mirror
10.5	How Sure Is It?
10.6	Opening the Box
11	**Models in the World**
11.1	The Rooster and the Sunrise
11.2	Grading the Graders
11.3	Stickers, Poison and Letters With Instructions
11.4	What the Model Remembers, and Who Owns It
11.5	The Long Life After Launch
11.6	The Edge of the Map

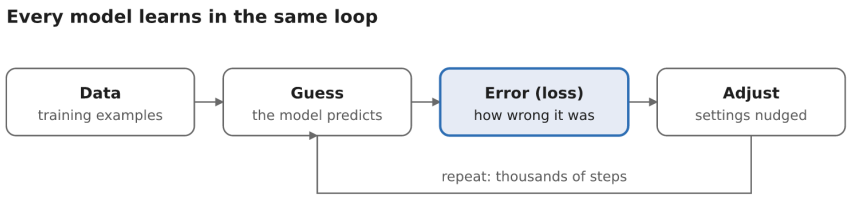
No.	Title
A	**The Math Behind Each Section**
B	**Dictionary of Metaphors**
C	**Where to Go Next**

Preface

Picture an engineer who spends an entire week tuning the cache of a busy service. She picks a time-to-live, waits for the dashboard to settle, looks at the hit rate, nudges the number, and waits again. By Friday the hit rate is excellent and she is exhausted. Over coffee, someone asks whether she has ever done any machine learning. She laughs: no, never, that is for people with PhDs.

She is wrong, and the rest of this book is, in a sense, an explanation of why.

What she does all week is guess, measure how wrong the guess was, adjust, and guess again. That loop is the whole engine. Nearly every model in this book, from a straight line drawn through a scatter plot to the language models that write code alongside you, runs the same four steps. What changes from one section to the next is mostly what is being adjusted, how the wrongness is measured, and how cleverly the adjustment is chosen. The few methods that skip the loop, such as the neighbor search of Chapter 2, are interesting precisely for what they leave out.



The difference between her week and a machine learning system is scale and patience. She turns one knob, by intuition, a few dozen times. A model turns anywhere from a handful to billions of knobs at once, often thousands or millions of times, and decides which way to turn each one by arithmetic rather than by feel. It does not try the knobs one at a time, which would take forever with a billion of them: a single backward sweep through the arithmetic tells every knob at once which way is downhill and how steeply, as Chapter 3 explains. That difference in scale is enormous, and it produces behavior that looks like magic. But it is not a difference in kind. If you understand her week, you can understand the machine.

Who this book is for

This book is written for anyone who is curious to really understand how machine learning works, not just to know its vocabulary. If you enjoy finding out how things work on the inside and have the patience of a good student, you have everything you need. Students who want to enter the field will find it useful, and so will engineers and programmers who already use these systems or are about to build them, researchers from other disciplines moving toward machine learning, and product managers, team leads and investors who need to weigh what a team, a vendor or a startup is promising.

You do not need to know anything about machine learning to start. You do not need more mathematics than you met in school: there is almost no formula in the main text. Where a formula would help, it waits in Appendix A, explained in words, and by the time you reach it, it should read like a compact summary of something you already understand. Many examples come from software engineering, because that is where most readers work, but every idea is first introduced through something from ordinary life, a kitchen, an exam, a dark closet, a mountain in fog. If you have never written code, you can follow the book by skipping the occasional programming aside; nothing important hides there alone.

How the book is built

There are eleven chapters, each a small arc of related sections. **Chapter 1** sets up the new way of thinking: learning from examples instead of writing rules, and how to measure that learning honestly. **Chapter 2** is the classic toolbox, the methods that still do most of the work on business data. **Chapter 3** is deep learning: what a neural network is and how it learns. **Chapters 4 and 5** open the hood. Chapter 4 looks at what a model physically is and how data is turned into something it can use; Chapter 5 at how its pieces fit together, what it costs to train and to run, and one road not taken. These two are the most demanding chapters in the book; take them a little slower. **Chapter 6** is an interlude about the other learning machine, the brain, and what some leading theories in neuroscience say it does. **Chapters 7 and 8** are the modern world: embeddings, attention, language models, image generators and the architectures built from them. **Chapter 9** is about the many ways a model is taught, and **Chapter 10** about the many ways a well-taught model still fails. **Chapter 11** follows models into the world, where they meet experiments, benchmarks, attackers, laws and time, and ends at the edge of the map, with the open questions about where the field is heading.

Chapters 2, 4, 5, 7, 8 and 10 each end with a deeper section or two, for the questions curious readers ask most often once the main idea is in place: which classic methods the tour of the toolbox walked past, what a model relies on and who decides, how much data, size and computing it needs, how much it can hold in mind at once, why guessing the next word goes so far, which kinds of data do not come as a table, what routines hide inside the training of large models, how sure a model is, and what can be read from inside it. Several of them also own up to what the main sections simplified, wherever an honest answer was too long to give on the spot. They go a little further than the rest of their chapter; on a first, hurried reading you can skim them and come back later.

Each section builds on the ones before it, and most end with a question that the next one answers. When a question raised in a chapter is taken up again later in the book, the chapter's last paragraph, a short one called "Going further", says where; you never need to jump ahead to follow the argument.

How to read it

The first time through, read it in order; each section takes ten to twenty minutes. If you have less time, a few shorter paths work well. To understand today's AI tools quickly: Chapter 1, sections 3.1 and 3.2, then Chapters 7 and 10. If you are a manager or an investor: Chapters 1, 7 and 10; then, for cost and for how models are taught, sections 5.2, 5.4 and 9.2; and from Chapter 11, the sections on reading benchmarks, on privacy, ownership and the law, and on where the field may be heading. If you are an engineer about to build something: all of it, with special attention to Chapters 4, 5, 10 and 11, and to sections 8.4 and 10.5. If you are a researcher arriving from another field: Chapters 3 to 6, Chapter 9, and sections 7.6 and 10.6.

Appendix B is a dictionary that pairs every picture in the book with the technical term it stands for. Keep it nearby when you start reading documentation and papers; it is the bridge from this book's language to the field's. Appendix C suggests where to go after the last page: courses, books, tools to try and a first set of projects, in a sensible order.

One warning before we begin. Every picture in this book lies a little. A model is not an apprentice, a crowd at a country fair or a room full of detectives, and wherever the analogy stops being true, I will tell you. Those are often the most interesting moments, because the place where the picture breaks is usually the place where the machine surprises its builders.

Chapter 1 — The Shift in Thinking

1.1 The Recipe and the Apprentice

Here is a small programming exercise. Write a function that takes a photo and returns `true` if there is a cat in it.

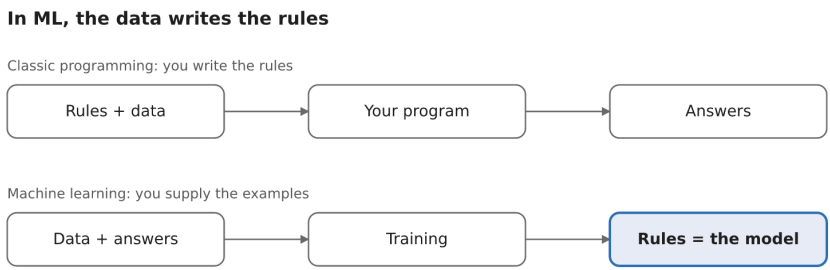
You start confidently. Cats have pointed ears, so you look for two triangles above a roundish shape. Then you remember cats photographed from behind, cats curled into a ball, cats under a duvet with only a tail sticking out. You add a rule for whiskers, and discover that a hairbrush has whiskers, at least as far as your rules can tell. You add a rule for fur, and a fluffy slipper starts passing as a cat. By the end of the day you have hundreds of lines of conditions and a function that is still wrong far too often.

Meanwhile, any three-year-old can do this perfectly. Ask her how, and she cannot tell you. The knowledge is real; it simply does not exist in the form of rules. And that is the wall that classic programming runs into.

Classic programming is the art of writing recipes. Grandma's card says two hundred grams of flour, three eggs, forty minutes at 180 degrees, and if you follow it, you get the cake. Every program you have ever written is a recipe of this kind: precise steps, decided in advance by someone who understood the problem. It works beautifully as long as the world looks the way the recipe expects. It breaks down most badly in one situation, and a very common one: when you cannot put into words what you want.

So imagine a different kind of kitchen. There is no recipe card, only an apprentice. You let the apprentice taste a hundred cakes that turned out well and a hundred that failed, each one labeled. At first the apprentice's judgments are random. But after every cake you tell them whether they were right, and slowly they build an internal sense of what good proportions feel like. Ask them to write it down and they can't. Hand them cake number 201, which they have never tasted, and they judge it correctly anyway.

That apprentice is machine learning. The arrows have been reversed: instead of giving the computer rules and data and getting answers back, you give it data together with the answers, and it hands you the rules.



The history of the spam filter shows why this reversal mattered so much. The first filters were recipe cards: block emails containing “FREE MONEY”, block senders on a blacklist, block anything with five exclamation marks. Spammers adapted within days. “FR3E M0NEY” sailed through, new domains appeared every morning, and every rule the defenders wrote started another round of an arms race they could not win. In 2002, the programmer Paul Graham published an essay called *A Plan for Spam* that helped popularize a different idea: stop writing rules, and let the filter learn from the mail each user had already marked as spam or not. The filter would discover for itself which words, links and patterns gave spam away. When spammers changed tactics, nobody rewrote anything; the filter simply learned from the new examples. Small filters of that kind could update their word counts the moment a user clicked “spam”, because their training was little more than counting; larger models are usually retrained on a schedule, with the newest examples added. Either way, people change the data, not the code. The arms race did not end, but the defenders finally had a weapon that kept up.

You may be sitting on a similar situation right now. Picture a support team that receives hundreds of tickets a day and must route each one to Billing, Infrastructure or Mobile. Someone writes keyword rules, and “invoice” goes to Billing. It handles the obvious cases. Then a customer writes “I was charged twice after the app crashed”, which mentions neither invoices nor phones, and the rules shrug. Yet in the ticketing system lie years of tickets that humans have already routed correctly. Every one of them is a cake with a label. The apprentice is waiting for them.

What is this apprentice made of? Underneath, a model is still a function, just like the ones you write. The difference is that it has blanks in it: numbers called **parameters** or **weights**, whose values nobody types in. **Training** is the search for the values that make the function agree with the examples. When training ends, the “rule” the apprentice learned is not an `if` statement you could read in a code review. It is a long list of numbers: a few dozen for a small model, billions for a large one. People call models black boxes. The box is not magic; it is written in a language of numbers rather than a language of conditions, a language people find very hard to read.

None of this means the recipe is dead. Validating an IBAN, calculating VAT, checking that a date lies in the past: here the rules are known, exact and often written into law. An apprentice would be slower, harder to audit and occasionally, confidently wrong. If you can write the rule in twenty lines and it rarely changes, write the rule. Machine learning earns its considerable cost, in data, in monitoring, in living with uncertainty, only when the rules are too many, too subtle or too quick to change.

There is one more thing about the apprentice, and it will haunt the rest of this book. A human apprentice understands *why* sugar makes a cake sweet. A model like this one has no such understanding; it only notices what goes together. Suppose every good cake in your photos happened to sit on a white plate and every failure on a blue one. The model will learn “white plate means good cake”, score perfectly on your examples, and be completely happy with itself. No score on examples that share the same accident will reveal it. You find out by testing on cakes where plate and quality disagree, a good cake on a blue plate, or by asking the model which inputs it leans on most.

The apprentice learns exactly what the examples teach, nothing more and nothing less. Which means that the most important question in machine learning is not which algorithm to use. It is where the examples come from, and who wrote the answers on them.

1.2 The Answers on the Back of the Page

Somewhere in your company there is a column of data that nobody thinks about, and it may be the most valuable thing you own.

It is not in a fancy data warehouse. It is the `resolved_by_team` field on old support tickets. It is the `duration_seconds` on every CI (continuous integration) build since the project started. It is the little checkbox where an engineer, after investigating a failed test, marked it as "flaky" or "real bug". Each of these is an answer, written by someone who knew, attached to a question. Collect enough of them and you have what the apprentice needs: cakes with labels.

Learning from questions paired with their answers is called **supervised learning**, and its everyday version is something you did at school. You solve a problem in the workbook and flip to the back to check. The answer was B; you said A. Nobody explains why. You only get the verdict, so you adjust your thinking a little and move on. After a few thousand problems, something has settled in your head that you could not fully put into words, yet it gets the next problem right.

Anyone who has learned German with flashcards knows how strange and wonderful this feels. You drill hundreds of nouns with their articles on the back, and one day you notice that you just *know* a word ending in *-ung* takes *die*. No teacher stated the rule. It condensed out of the examples, the way frost forms on a window. That is the entire promise of supervised learning in one small moment: the rule was never given, only the examples and the verdicts.

Now go back to that `duration_seconds` column. Every build in your CI history comes with its circumstances: how many files changed, which modules they touched, how many tests ran, whether the cache was warm. In the vocabulary of the field, those circumstances are **features**, the measurable properties of each build that the model gets to look at. The true duration is the **label**, the answer on the back of the page. Give a model a few thousand past builds and it learns to estimate the next one before it starts, which is genuinely useful when you are scheduling runners or warning a developer that their change will keep CI busy for forty minutes. Other features might be a client's age, the color of a pixel or whether a word appears in a text. Laying features out as a table, one row per example and one column per feature, is just one convenient arrangement, natural for business records like these builds; for photos, text and sound, whose raw material is pixels, words and samples, the story is different, and a later chapter tells it.

The flaky-test checkbox works the same way, except that the answer is a category rather than a number. The model sees the test's history, the error message, the runner and the time of day, and learns to suggest "probably flaky" before anyone looks. Predicting a number is called **regression**; predicting a category is called **classification**. Between them, they cover most of what supervised learning is used for.

There is one more ingredient, and it is easy to overlook because it sounds so mundane. After every guess, something has to say how wrong it was, as a single number. That number is the **loss**, and the way you define it quietly shapes everything the model learns. Say you are predicting delivery times. If you simply add up the minutes of error, being an hour late once counts the same as being six minutes late ten times. Customers do not feel it that way: one delivery an hour late generates an angry call, while several slightly late ones go unnoticed. So the usual choice squares the errors, and a miss of sixty minutes then weighs far more than ten misses of six. The loss is chosen by people, not by the algorithm, and it is a statement of what you care about. Squares are popular for that reason and because they make a smooth valley that is easy to walk down; if every minute of lateness really costs the same, you can add up plain minutes instead, and the model learns to aim at the typical delivery rather than be pulled toward the rare disasters.

For categories, the usual loss has a personality too. It is gentle with a model that was unsure and wrong, and merciless with one that was confident and wrong. You know this from weather forecasts. A forecaster who said “50% chance of rain” on a rainy day has not embarrassed herself. One who said “99% sun” on that same day has lost your trust for a month. The usual loss looks only at the probability the model gave to what actually happened and takes its logarithm: giving the truth 50% costs a little, giving it 1% costs a lot, and giving it 0% costs infinitely much, which is why a well-trained classifier almost never says “impossible”.

So far this sounds like a tidy story: collect answers, define a loss, let the model learn. Here is where it turns.

Picture a team under pressure. A test fails on Friday afternoon, the release is blocked, and someone needs to merge before the weekend. The test gets marked “flaky”, the build is rerun, everyone goes home. It happens again the next Friday, and the one after. Months later, someone trains a model on the flaky-test checkbox. It learns, faithfully and correctly, that tests failing on Friday afternoons are flaky. The model has not made a mistake. It has learned precisely what was written on the back of the page.

This is the uncomfortable truth at the heart of supervised learning: a model can only be as good as its answers, and real answers are written by tired people, buggy scripts and old processes nobody has questioned. Workbook answers are checked by an editor; yours usually are not. This is why, in real projects, collecting and cleaning labels often costs more than building the model itself, and why the best machine learning engineers spend a surprising amount of their time reading individual examples rather than tuning algorithms. The model cannot notice wrong labels by itself, since the labels are its only notion of truth. There is a useful trick, though: train a model, then look at the examples where it confidently disagrees with its label; many turn out to be the Friday-afternoon checkboxes, and that short list is a far better place for a human to start reading than the whole pile.

Which raises a natural question. What if you have no answers at all? No labels, no checkboxes, just a mountain of data that nobody has ever sorted. Can a machine learn anything from that?

1.3 Sorting the Closet in the Dark

The power goes out on a winter evening, just as you have dumped a basket of clean laundry on the bed. You have no flashlight and no patience to wait, so you start sorting by touch. Thick, long-sleeved things go in one pile. Thin, short things in another. Stiff things with buttons in a third, small soft things in a fourth. When the lights come back, the piles make perfect sense: sweaters, T-shirts, shirts, socks. Nobody told you those categories existed. You found them because the clothes themselves were different, and similar things ended up together.

That is the answer to the question at the end of the last section. Yes, a machine can learn without answers, in roughly the way you sorted the laundry. Learning without labels is called **unsupervised learning**, and its most common form, finding groups of similar things, is called **clustering**.

It is three in the morning when you will appreciate it most. An alert has fired, and the logging system shows forty thousand stack traces from the last hour. Nobody can read forty thousand stack traces. But most of them are the same few problems repeated, decorated with different timestamps, user IDs and request numbers. Cluster the traces by similarity and the wall of text collapses into a short list: this error, twelve thousand times; that one, nine thousand; a strange new one, seventeen times, starting at 02:41. You read the list, not the wall, and the strange new one is probably your incident. To get there, the noise is stripped out first: timestamps, user IDs and request numbers are replaced by placeholders, so that two traces of the same bug become nearly identical text. Then each trace is turned into numbers, for instance by counting which words and function names it contains, and similar traces are those whose lists of numbers are close.

How does a computer decide what goes together? The classic method, **K-Means**, works the way a pizza chain might choose where to put its delivery stations in a new city. You decide you can afford, say, five stations, and drop them on the map at random. Every house is assigned to its nearest station. Then each station moves to the middle of the houses it serves. Now some houses are closer to a different station, so they switch, and the stations move again. Repeat, and after a few rounds nothing changes: the stations sit in the centers of natural neighborhoods. Notice who chose the number five. You did, not the algorithm. To choose it well, you usually try several and watch how much tighter the groups get with each extra station: the first few help a lot, then the gains flatten out, and the bend in that curve is a reasonable choice. The final judgment is still yours, made by opening the groups and asking whether they mean anything; other methods find the number of groups by themselves.

The same idea turns into an early-warning system. Every server in a fleet reports CPU (central processing unit) load, memory, latency, error rate and traffic. Healthy servers end up in a few tight groups, like houses around their pizza stations. A server that sits far from every group is behaving unlike anything else, even if no single metric has crossed an alert threshold yet. Finding such outliers is **anomaly detection**, and it is one of the most practical things unsupervised

learning does. Where “far” begins is yours to decide, usually just beyond where almost all healthy servers have sat over the past month: draw the line too close and you get pager fatigue, too far and the drifting server is caught late.

Groups are not the only hidden structure worth finding. Hold a coffee mug in front of a flashlight and look at its shadow on the wall. From the side, you see the handle and would recognize it as a mug anywhere. From above, it is a plain circle that could be a glass, a can or a coin. If you were allowed to keep only one shadow, you would rotate the mug until the shadow told you the most.

Spec sheets have the same property. A laptop comes with thirty numbers: cores, clock speed, RAM, graphics memory, price, weight, screen size, battery life and more. But those numbers are not independent. Machines with more cores tend to have more RAM, a better GPU (graphics processing unit, the chip on a graphics card) and a higher price; lighter machines tend to have smaller screens and longer battery life. The thirty numbers are mostly telling two stories, “how powerful” and “how portable”. **Principal component analysis**, or PCA, finds such summary directions automatically, and lets you describe every laptop with two numbers instead of thirty while losing surprisingly little. It is the mathematical version of choosing the most informative shadow. It looks for the direction along which the laptops differ most, then for the next most varied direction at right angles to the first, so that it does not repeat what the first already says, and so on. Keeping two directions means discarding the smaller differences along all the others, which is fine if they were mostly noise and painful if a rare but important property lived there. Engineers use it to squash a dashboard of two hundred metrics per service into a single two-dimensional plot, where the one service drifting away from its siblings jumps out at a glance.

All of this is useful. Now for the part that should make you a little suspicious.

In 1976, NASA’s Viking orbiter photographed a hill on Mars that looked unmistakably like a human face staring up from the desert. For years it fed theories about ancient civilizations. Later, sharper images showed an ordinary eroded mesa. The face had never been there; the human brain is simply built to find faces, in clouds, in electrical sockets, in the front of a car. Clustering algorithms have the same affliction. Give K-Means pure random noise and ask for five clusters, and it will hand you five clusters, each with a neat center, every time. It never answers “there is nothing here”.

There is a quieter trap too. When you sorted laundry, you chose to sort by thickness. An algorithm sorts by whatever its notion of distance happens to measure. Cluster employees by salary in euros and years of experience, and a difference of 1,000 euros swamps a difference of ten years, purely because the numbers are bigger. The “natural groups” you discover are then just salary bands. This is why data is usually rescaled before clustering, so that every feature speaks at a similar volume.

So unsupervised learning gives you hypotheses, not conclusions. It says: these things seem to belong together, this one seems odd. Whether that means anything is your call, and the only reliable way to make it is to open each group and look at real examples.

Which brings us to the question that every method in this book must eventually face, supervised or not. How do you know, honestly know, that a model works? Teams have celebrated too early more often than you would think.

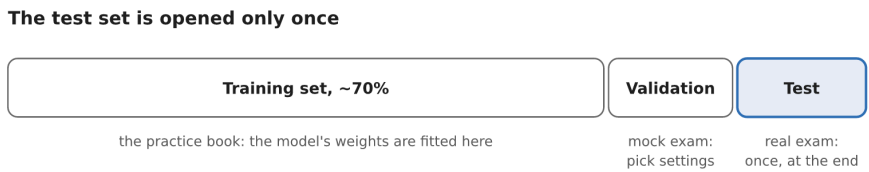
1.4 The Exam You Are Not Allowed to See

Picture a team that has spent two months building a ticket router. The demo goes beautifully: they feed it a thousand tickets and it sends 98% of them to the right team. There is cake. A month after launch, a support lead quietly forwards a spreadsheet. In production, the router is right about seven times out of ten. Nobody can find a bug. The code is clean, the model loads correctly, the inputs look normal.

What went wrong is almost embarrassingly simple, and it happens all the time. The thousand tickets in the demo were tickets the model had learned from. The team had measured its memory, not its skill. It is like judging a job candidate by asking the exact interview questions they were handed to study the night before.

Think about how a driving test works, because it solves this problem rather elegantly. You learn from a practice book, which you can read as often as you like, answers included. Along the way you take mock exams, which tell you whether your way of studying works and which topics need another pass. Then comes the real test, once, at the end. If someone leaked you the real questions in advance, your score would be real, and it would say nothing about whether you can drive.

Machine learning copies this structure exactly. The **training set** is the practice book: the model's weights are fitted to it. The **validation set** is the mock exam: you use it to compare approaches and choose settings. The **test set** is the real exam, and it only means something if it stays sealed until the end.



The sealed envelope is harder to keep sealed than it sounds. Suppose you evaluate on the test set, notice the score is a bit low, tweak something, evaluate again, tweak again, twenty times, and finally report the best number. Each peek leaked a little information, and after twenty peeks you have tuned the model to that particular exam. Every engineer knows the CI version of this: rerunning a flaky pipeline until it goes green and then declaring the build healthy. The green run did happen. It just tells you much less than it seems.

It helps to keep two kinds of numbers apart in your head. Parameters, the blanks in the function that training fills in, are what the model learns by itself from the practice book, the knowledge in the student's head. **Hyperparameters** are choices about how the learning happens: how big the model is, how fast it learns, how long it trains. In the driving test picture, they are how many hours a day you study and which practice book you buy. The model sets its parameters; you set the hyperparameters, and you set them by watching the mock exams. Watching them leaks information too, in exactly the same way, only it is allowed: after many rounds of tuning, the validation score is a little flattering, which is precisely why the test set exists, the one number nobody was allowed to tune against.

When data is scarce, holding back a whole mock exam feels wasteful, and there is a neat trick for that. Split the data into five parts and train five times, each time using a different part as the mock exam and the other four as the practice book. Every example gets used for practice four times and for the mock exam once, and the average of the five scores is a steadier verdict than any single one. This is **cross-validation**.

There is a second lesson hiding in the ticket router story, and it is about what the student is given to study at all. A model sees only the features you hand it. Give it a raw timestamp like 2026-03-14T08:47:12Z and most models can do very little with it; it is just a big, ever-increasing number. Hand it the same moment as "hour of day: 8", "day of week: Saturday" and "public holiday: no", and suddenly it can learn that Saturday mornings are quiet and Monday mornings are chaos. Turning raw data into inputs that expose what matters is called **feature engineering**, and on ordinary business data it often matters more than the choice of algorithm.

Now for the twist that no driving test prepares you for. The road exam is drawn from the same rulebook year after year. Your data is not. Customers change their habits, prices move, a competitor launches, slang shifts, a pandemic rearranges everything in a fortnight. A model that scored brilliantly on last year's data can quietly lose its edge this year without a single line of code changing. The slow shift in the data behind this decay is called **data drift**, and it is the reason models in production need monitoring like any other service, with alerts when their inputs or their accuracy start to wander. When the true answers arrive only weeks later, you watch the inputs, which arrive at once: if this week's tickets are longer, come from new countries or mention words the model rarely saw, the world has moved before any accuracy number can say so. The usual remedy is unglamorous, retraining on recent data on a schedule or whenever the alarms fire.

It also changes how you should seal the envelope. For anything that unfolds over time, never pick the test set at random. Train on the past and test on the most recent period, the way the model will actually be used. A random shuffle lets the model learn from next Tuesday while being tested on last Monday, and a model that has seen the future will always look like a genius.

That is enough about how to judge a student. It is time to meet some actual students, beginning with the humblest one of all: a straight ruler, a handful of nails and some springs.

Going further. If you wondered how a model like the one with the white plate gets caught cheating, section 10.2 takes that up, and if you wondered how many labeled examples are enough, section 5.4 does. Whether anyone can read what a trained model has learned from its long list of numbers is the question of section 10.6. And what it takes to keep a model useful once the world drifts away from its data is the story of section 11.5.

Chapter 2 — The Classic Toolbox

2.1 A Ruler, Some Springs and a Dimmer Switch

Take a wooden board and hammer in a nail for every apartment sold in your neighborhood last year: left to right is the size, bottom to top is the price. You now have a cloud of nails that drifts upward, roughly, as you move right. Tie a short spring to each nail, pointing straight up or down, and clip all the springs to a long, rigid ruler. Then let go.

Where does the ruler come to rest?

It wobbles, the springs pull from above and below, and it settles in the one position where the total energy stored in the springs is as small as it can be. That position is the line of **linear regression**, the oldest and still one of the most useful models in this book. You did not have to calculate anything. The springs did it. A computer has no springs, and for a straight line it does not need them: a formula gives the resting position directly, in one calculation. For models with no such formula, the computer imitates the wobbling ruler step by step, which is the walk down the mountain of the next chapter.

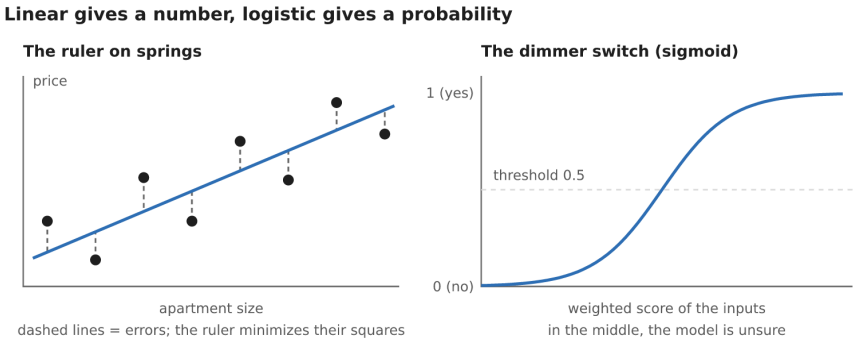
Why springs, and not pieces of string? Because the energy in a stretched spring grows with the *square* of the stretch. Pull a spring twice as far and it stores four times the energy. That is precisely what linear regression minimizes: the sum of the squared vertical distances between the points and the line. And it explains something you may have seen go wrong in practice. Add one penthouse that sold for ten times the price of anything else on the street, and its spring is stretched so far that it drags the whole ruler toward itself. The other predictions get worse, on the whole, to accommodate one extreme point, an **outlier**. A single nail can bend the line for everyone.

This little physical model turns up in engineering constantly. Plot an API (application programming interface) endpoint's latency against the size of the request payload and you usually get a cloud drifting upward. Let the springs settle and the ruler gives you a sentence a product manager can use: every extra megabyte costs roughly so many milliseconds, on top of a fixed overhead. The slope and the starting height are the only two numbers the model learned, and unlike almost every other model in this book, you can read them and argue with them. With many inputs, reading the weights as a ranking of importance takes more care than it seems, as a later chapter shows.

But some questions cannot be answered by a ruler, because the answer is not a number. Will this deployment fail? Is this payment fraud? Will this customer cancel? Ask a straight line, and it will cheerfully predict 1.7, or minus 0.3, which mean nothing as answers to a yes-or-no question.

The fix is charming. First compute the same kind of weighted score as before, adding up the evidence. Then pass that score through a dimmer switch. When the score is very low, the light is off: zero, no. When it is very high, the light is fully on: one, yes. In between, the light brightens smoothly, and you read its brightness as a probability. A reading of 0.83 means "fairly likely

yes”. The S-shaped curve of the dimmer is called the **sigmoid**, and the whole arrangement, score plus dimmer, is **logistic regression**, despite the name a classification method. One caution about the picture: a dimmer’s brightness is simply where the knob is set, but the model’s 0.83 is an honest probability only if training made it one, and many models turn out to be more confident than their track record justifies; checking this is called calibration.



Picture it applied to your deployment history. Each past deployment comes with a few facts: how many lines changed, how many services were touched, whether the database schema moved, what hour it went out. Each is labeled with whether a rollback followed. The model learns how much each fact should push the score, and every new deployment gets a risk between 0 and 1. You might require a second reviewer above 0.3, and refuse Friday evening deployments above 0.6. Notice that those thresholds are not the model’s decision; they are yours, and they say how much risk your team can stomach. That theme will come back with some force.

Under both models sits the same structure: each input is multiplied by a **weight** that says how much it matters, the results are added, and a constant called the **bias** shifts everything up or down. Linear regression stops there. Logistic regression adds the dimmer. And the place where the dimmer reads exactly 0.5, the line between yes and no, is called the **decision boundary**. Hold on to that idea, because in the next chapter you will discover that logistic regression is, quietly, a single artificial neuron.

So far, so tidy. Now the ruler shows its limits, and the first limit is in its name: it is straight.

Every engineer who has load-tested a service has seen a relationship that is not. At 20% CPU, latency is fine. At 40%, still fine. At 60%, fine. At 85%, requests start queuing and latency shoots up like a hockey stick. A straight ruler through those points does something terrible: it underestimates latency precisely in the region you care about most, near saturation. The remedy is often not a fancier algorithm but better inputs. Hand the model the load squared, or the logarithm of a quantity, and a straight line in the new inputs becomes a curve in the old ones. Which extra inputs to offer comes mostly from plotting the data and knowing the domain: queues blow up near saturation, and sizes and waiting times tend to spread over many orders of magnitude. Then you try the candidates on the mock exam; the next chapter is the story of models that learn such bends for themselves.

The second limit is subtler, and it is not really about the ruler at all. Ice cream sales and drownings rise and fall together, beautifully, on a straight line. Ice cream does not cause drowning; summer causes both. In engineering data the same illusion is everywhere. The services with the most tests often have the most bug reports, not because tests create bugs but because those are the biggest, busiest, most critical services. A large weight means two things move together. It never, on its own, means one causes the other: a model fitted to historical data can only report what went together, and telling a cause from a fellow traveler takes an experiment, such as the A/B tests that split traffic at random, which the last chapter takes up.

The ruler makes a strong assumption about the world: that it is, more or less, a straight line. What if you refused to assume anything at all, and simply asked the neighbors?

2.2 Tell Me Who Your Neighbors Are

You have just moved to a new city and found an apartment you like. The asking price seems high, but you have no idea what is normal here. You do not build a pricing model. You look up the five apartments most like this one that sold recently, same street, similar size, similar floor, and average their prices. If the asking price is far above that average, you negotiate.

Without noticing, you have run **k-nearest neighbors**, with k equal to five. It may be the most human algorithm in this book, because it is how we reason about almost everything new. A restaurant is probably good if the three friends whose taste is closest to yours loved it. A film is probably worth watching if people who liked the same films as you rated it highly. Many “customers like you also bought” boxes on shopping sites are this idea running at industrial scale. Real recommender systems add a good deal to it.

It is also quietly at work in engineering tools. Picture a bug tracker receiving a new report: “Login button does nothing on Safari”. Before anyone triages it, the tool looks up its nearest neighbors among existing issues and finds “Sign-in click unresponsive on macOS browser”, filed last week, already assigned, already half fixed. The words barely overlap, so the tool cannot simply count shared words; it measures how close the meanings are, with a trick a later chapter explains, and the duplicate is caught before two engineers spend an afternoon on the same problem. Or picture an incident tool that, the moment an alert fires, shows the five past incidents most similar to this one, with links to how each was resolved. Often the fix is sitting in one of those postmortems.

What makes k-NN unusual is that it does no learning in advance at all. There is no training phase in which weights are fitted. The model *is* the data, stored as it is. All the work happens when a question arrives: measure the distance from the new case to every stored case, take the k closest, and let them vote on a category or average their numbers. It is the laziest algorithm imaginable, and that laziness comes with a gift. Every prediction can be explained by pointing at real examples: we think this apartment is worth this much because of these five sales. The price comes at scale: measuring the distance to a hundred million stored cases for every question is too slow, so real systems build an index in advance, a little like the sections of a library, that jumps to

the right neighborhood and checks only a few thousand candidates. They occasionally return the second-nearest neighbor instead of the nearest, a trade almost everyone accepts; these are the vector databases that reappear later in the book.

The one setting you must choose is k, and it behaves like a dial between nervousness and blur. With k equal to one, a single odd neighbor, a mislabeled ticket or a sale between relatives, decides everything. With k equal to five hundred, you are averaging over half the city, and every apartment looks the same. Somewhere in between lies a sensible value, and you find it the usual way: by trying a few on the mock exam.

So far, the neighbors have been wise. Now something strange happens, and it is one of the genuinely counterintuitive facts of this field.

On a street map, “near” is obvious. Two dimensions, and your eye does the job. But suppose you describe each person not by two numbers but by five hundred: height, age, income, the number of books they own, how many hours they sleep, their favorite color encoded as a number, and on and on. Pick any two people. On some of those five hundred traits they are bound to differ a lot. Add up all those differences, and everyone ends up roughly equally far from everyone else. Your “nearest” neighbor is barely nearer than your farthest. In a high-dimensional space, everyone is a stranger.

This is the **curse of dimensionality**, and it is why k-NN works best with a modest number of meaningful features, or after something has compressed many raw features into a few informative ones. And like clustering, it falls into the scaling trap: unless the features are rescaled, the one with the biggest numbers dominates every distance.

Asking the neighbors works beautifully when you know what makes two things similar. When you don’t, when there are dozens of features and only a few of them matter, you need a different strategy. Instead of comparing everything at once, you would want to ask the right questions, in the right order. Which is exactly how the game of twenty questions is won.

2.3 Twenty Questions

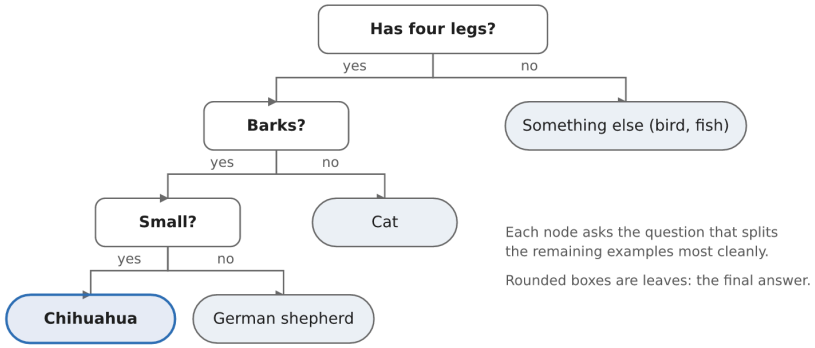
The release goes out in two hours, and something in the last two hundred commits broke the checkout page. Nobody knows which commit. Testing them one by one would take all night.

Then someone types `git bisect`, and the mood in the room changes. Bisect checks out the commit in the middle and asks one question: is the bug here? Whatever the answer, half the suspects are gone. Another question, another half. After eight questions, two hundred commits have shrunk to one, and the culprit is on the screen before the coffee is cold.

What makes bisect so satisfying is not the speed of the computer. It is the quality of the questions. Each one is chosen to remove as much uncertainty as possible, whatever the answer turns out to be.

You have played this game since childhood. In twenty questions, a beginner opens with “Is it a Chihuahua?” and learns almost nothing from the inevitable “no”. A good player opens with “Does it have four legs?”, because either answer cuts the world in half. Then “Does it bark?”, then “Is it small?”. Three questions in, the candidates have collapsed to a handful.

Three yes/no questions lead to Chihuahua



A **decision tree** is this game, played by a machine that has studied thousands of past rounds. You have seen the hand-drawn version of it many times: the triage checklist in an emergency room, the mechanic’s diagnostic flowchart, the “have you tried restarting it?” script at a help desk. Someone experienced wrote those flowcharts from years of cases. A decision tree writes them automatically, from data.

To do that, it needs a way to tell a good question from a bad one, and the answer comes from an unlikely place: your sock drawer. A drawer containing only black socks is perfectly tidy; reach in blindly and you know what you will get. A drawer with half black and half white socks is as messy as it gets; every grab is a coin toss. **Entropy** and the **Gini index** are two ways of putting a number on that messiness. A good question is one that turns one messy drawer into two tidier ones. The amount of tidying is called **information gain**, and at every step the tree simply tries every feature and every possible cut-off (“more than 400 lines changed?”, “size above 54 square meters?”) and keeps the question with the largest gain. Then it repeats the process inside each new drawer. Add up how much tidying each feature was responsible for across all the questions in the tree, and you have a ranking of the features: a useful first look at which ones mattered, though not an impartial one, for reasons a later chapter explains.

Picture what this produces on your own history. Feed a tree every pull request merged in the last two years, each labeled with whether it was reverted within a week. It might come back with a runbook nobody ever wrote down: does it touch database migrations? If yes, is it larger than a few hundred lines? If yes, was it reviewed by only one person? Then the revert rate is high. Each path from the top of the tree to a leaf reads like a rule a senior engineer might state, which is exactly why trees are loved in places where decisions must be explained, such as credit approvals and compliance checks. They also have pleasant manners: they handle numbers and categories

alike, and they do not care that one feature is in euros and another in years. A tree only ever asks whether a value is above or below a cut-off, and rescaling a feature moves the cut-off along with the values without changing which examples fall on which side, so the scaling trap that caught the neighbors never touches it.

It all sounds ideal. But the tree has two character flaws, and they are serious.

The first is impatience. A skilled player thinks a few questions ahead; sometimes a slightly worse first question sets up a brilliant second one. A tree never does this. At each step it grabs the best question available right now and never reconsiders. It is **greedy**, in the technical sense.

The second flaw is worse: a tree does not know when to stop asking. Left alone, it keeps splitting until every drawer is perfectly tidy, which on real data often means only one or a few examples per leaf. It ends up asking things like “Is it 54.3 square meters, on the third floor, with a green front door?”. That question separates one apartment from all others in the training data, and it is useless for any apartment the tree has not seen. The tree has stopped learning patterns and started memorizing. The usual cure is to set limits in advance: no more than, say, six questions deep, no drawer with fewer than twenty examples, or no question that tidies less than some minimum. These are hyperparameters, chosen by watching the mock exam.

Worse still, the memorizing is unstable. Change a handful of rows in the training data and the first question might change, and with it everything below. Two analysts training trees on almost the same data can get trees that look nothing alike. A single tree, in other words, is a brilliant but erratic expert: readable, fast, and far too impressed by the particular cases it happened to see.

There is a fix, and it comes from an observation made at a country fair more than a century ago. What if, instead of one erratic expert, you asked a whole crowd of them?

2.4 The Wisdom of Crowds, and of Editors

In the autumn of 1906, the statistician Francis Galton, then in his eighties, wandered through a livestock fair in Plymouth and came across a contest. For a small fee, visitors could guess the weight of an ox once it had been slaughtered and dressed. About eight hundred people entered: butchers and farmers who knew cattle, and plenty of clerks and families who did not. Galton, who had a low opinion of the average person’s judgment, collected the tickets afterwards, curious to see how well the crowd had done.

The individual guesses were all over the place. But when he took the middle value of all of them, it landed within about one percent of the true weight. The crowd, as a crowd, had been almost exactly right. Galton published the result the following year in the journal Nature, under the title *Vox Populi*.

Why did it work? Because each person’s error pointed in a different direction. One guessed too high because the ox looked big from where he stood; another too low because she was thinking of a smaller breed. Independent mistakes, added together, largely cancel. What survives is the part everyone saw correctly.

Now return to the erratic decision tree from the last section, the brilliant expert who memorizes every quirk of the cases it saw. A **random forest** builds a Galton crowd out of such experts, and the clever part is how it makes them independent. Each tree receives a slightly different copy of the training data, drawn at random so that some rows appear more than once and others are missing; this is called a **bootstrap sample**. And at every question it asks, each tree may only consider a few features chosen at random. Every expert is half-blind in a different way, so each one makes different mistakes. When the forest votes, or averages, the individual quirks cancel, just like the guesses at the fair. Training many models on bootstrap samples and combining them is called **bagging**; adding the random choice of features is what makes it a forest.

But the fair has a darker version, and it explains the design. Imagine that before anyone wrote a ticket, one loud farmer announced his guess to the whole crowd. Half the people would anchor on it. Their errors would no longer be independent; they would all lean the same way, and averaging would not cancel anything. The crowd would become a choir singing one man's wrong note.

Trees face exactly this danger. If one feature in your data is very strong, every tree will want to ask about it first, and they will all make the same mistakes together. Hiding a random subset of features from each tree at each step is the forest's way of keeping the loud farmer out of the room.

You already know this principle from code review. A second reviewer with exactly your background, your habits and your blind spots catches little you would not have caught yourself. A reviewer who thinks about security while you think about performance catches far more, precisely because your blind spots differ. Diversity is not a nice extra. It is the heart of the mechanism.

There is a second way to turn weak experts into a strong one, and it looks less like a crowd and more like a publishing house.

An author writes a rough first draft. An editor reads it and fixes only the structural problems; she does not rewrite the book. A copy editor then fixes only the clumsy sentences the editor left behind. A proofreader catches the remaining typos. Nobody redoes the whole job. Each person specializes in what the previous ones got wrong, and the book that comes out is far better than any of them could have produced alone.

Gradient boosting works like this. The first small tree makes a rough guess, say of how long a delivery will take. The second tree is not asked to predict delivery times at all. It is asked to predict the *errors* of the first tree: where it was too optimistic, where too pessimistic. The third tree predicts what is still wrong after the first two, and so on, for hundreds of rounds. Each tree on its own is crude, little more than a few questions deep, yet the chain of corrections becomes remarkably precise.

If your company scores payments for fraud, predicts which customers will cancel, or estimates delivery times from a table of a few dozen columns, there is a good chance a boosted model is doing the work, from one of the well-known libraries such as XGBoost, LightGBM or CatBoost.

On that kind of tabular data, these models are usually the ones to beat, and they frequently beat neural networks. The reason is that a table usually arrives with its features already made: a few dozen of them, each meaningful on its own, where sharp thresholds such as “more than three failed logins” carry the signal. Networks earn their keep by building features out of raw pixels and words; on a table there is little left for them to build, while trees handle mixed units and sharp cut-offs naturally.

The two methods fix opposite weaknesses, and the field has names for both. A random forest takes detailed experts who overreact to the particular data they saw, and calms them down by averaging; it reduces **variance**. Boosting takes simple experts who are systematically too crude, and sharpens them by correction; it reduces **bias**, here meaning an error that always leans the same way (not the constant added by the ruler; a later chapter sorts out the word’s several meanings). In practice, a forest is a robust default that forgives sloppy settings, while boosting usually wins on accuracy but needs supervision, mainly knowing when to stop adding trees before the editors start correcting noise. For a forest, more trees do not cause overfitting, the memorizing that spoiled the single tree: the crowd’s average simply stops changing after a few hundred, and the only cost is time. For boosting, each new tree chases what is still wrong, and after a while that is mostly noise, so you stop when the mock-exam score stops improving.

And one limit applies to both. If the data is skewed for everyone, if the ox was weighed on a broken scale, no crowd and no chain of editors will straighten it.

The ruler, the neighbors, the trees and the crowds are not the whole toolbox, though, and a few of the other tools deserve at least a quick look before we move on.

2.5 The Aisles We Walked Past

Walk into a large hardware store for a box of screws and you go straight down the one aisle you need, past forty others full of tools you have never used. This chapter’s tour of the classic toolbox was a walk of that kind. It went down three aisles, the ruler, the neighbors and the trees, plus the crowds built out of trees, because those ideas carry most of the weight on business data. But open the documentation of any machine learning library and the list of methods runs to pages. A few of those names carry ideas the earlier sections had no room for.

Start with the border. The dimmer’s decision boundary was simply wherever training happened to leave it. Picture instead two villages on either side of a wide plain, and a commission asked to build a road between them that belongs to neither. Many roads would separate the houses. The commission picks the one that leaves the broadest strip of empty land on both sides, on the theory that a road hugging one village’s fences will be wrong about the first new house built at its edge. That is the **support vector machine**, or SVM: of all the boundaries that separate two classes, it chooses the one with the widest **margin**. Only the houses nearest the road decide where it goes; they are the support vectors that give the method its name, and every house further back could move without changing anything. Real data is rarely cleanly separable, so the practical version lets a few houses stand on the wrong side of the road, at a price.

The second idea is the clever one. Some villages cannot be separated by any straight road, because one is a ring around the other. The SVM's answer, the **kernel trick**, is to judge the land in a richer space where a straight road does work. Add each house's distance from the center as an extra coordinate, and the inner village becomes a valley and the outer one a hill, easily split by a flat cut. The trick is that the SVM never builds that richer space. All it needs is how similar each pair of houses is, and a **kernel** (unrelated to the kernels of the image networks in the next chapter) is a formula that computes the similarity the pair would have in the richer space, even an infinitely large one, without ever going there. Through the late 1990s and the 2000s, SVMs were among the strongest general-purpose classifiers, and they remain a sensible choice for small datasets with many features, such as a few hundred patients described by thousands of measurements. The picture breaks in two places. The widest road lives in a space the kernel invented, which nobody can draw or walk along. And comparing every pair of houses becomes expensive once there are millions of them, which is part of why boosted trees and networks took over on large data.

The second aisle is about belief. A doctor sees a patient with a cough. Before any test, she already has a sense of how common each illness is this month: colds are everywhere, pneumonia is rarer. That starting suspicion is the **prior**. Then evidence arrives, a fever, a crackle in the lungs, and for each one she asks how likely that symptom would be under each illness and shifts her suspicion accordingly. What she believes at the end is the **posterior**. The rule that says exactly how to do the shifting is **Bayes' rule**, from an essay by the minister Thomas Bayes published in 1763, after his death, and it is the honest way to combine what you believed with what you have just seen. Its most useful lesson for engineers is the one people most often get wrong. A test that is 99% accurate, for a disease that one person in ten thousand has, will still produce mostly false alarms, because the prior was so small. The same arithmetic returns whenever a model hunts for something rare, and it is why accuracy alone says so little about such models.

Now make the doctor lazy. Suppose she pretends that every symptom is unrelated to every other, so that a fever tells her nothing about whether to expect a cough. Then each symptom becomes a separate multiplication of her odds, and the arithmetic becomes trivial. That shortcut is **naive Bayes**, and "naive" is the honest word, because the assumption is false almost everywhere: "free" and "money" are not independent in an email. Yet methods of this kind powered many of the learning spam filters that replaced the hand-written rules at the start of this book, and naive Bayes is still a surprisingly strong first model for text. It ranks emails well even though the probabilities it computes are badly exaggerated, because counting related evidence twice inflates its confidence without usually changing which way it leans. Here the picture breaks in a useful direction. A doctor who ignored how symptoms travel together would be a bad doctor, and the method that does exactly that works anyway. A wrong assumption can be harmless for ranking and harmful for confidence, which is a lesson about every simplification, including this book's.

A relative from the same aisle deserves a sentence, because it will come back later in the book. A **Gaussian process** predicts a number together with how unsure it is. Think of a flexible wire pinned at every known point: tight near the pins, swinging freely in the long gaps between them, so that its uncertainty grows wherever data is thin. It becomes slow on large datasets but is

excellent on small, expensive ones, which is why it is used to choose hyperparameters; it also belongs among the tools for saying how sure a model is. The wire picture hides the one real choice: how stiff the wire is, how far one pin's influence reaches, is set by the analyst or tuned from the data, and a badly chosen stiffness produces confident nonsense between the pins.

The third aisle settles an old debt. K-Means needed you to choose the number of groups, and it would find five groups in pure noise. Picture instead a music festival seen from a drone. Nobody tells you how many groups of friends there are. A group is simply wherever people stand close together, it extends as far as the crowd stays dense, and the few people wandering between stages belong to no group at all. **Density-based clustering**, of which DBSCAN is the classic example, works this way: it finds as many groups as the density suggests, of whatever shape, and labels the stragglers as **noise**. That answers the face on Mars halfway. Given random scatter, a density method will often report one big blob or mostly noise rather than five tidy clusters, though you still choose what "close" means, and a careless choice can conjure groups like any other method. A second family, **hierarchical clustering**, builds a family tree instead. It merges the two most similar items, then the next two, until everything is one group, and you decide afterwards where to cut the tree, into three branches or thirty.

The same aisle sells the pictures that appear on every slide about embeddings, the long lists of numbers by which later chapters place words and images on a map of meaning: a cloud of colored dots in two dimensions, with neat islands of similar items. They are usually made by methods called **t-SNE** and **UMAP**, which squash hundreds of dimensions onto a page while trying to keep each point's nearest neighbors near it. Think of the flat world map on a classroom wall, on which Greenland looks about as big as Africa. Every flat map of a round planet distorts something, and these maps keep neighborhoods at the cost of almost everything else. The size of an island and the distance between two islands often mean very little, and different settings can split one real group into several or glue two together. Treat the islands as hints worth opening, never as findings.

The last aisle sits where labeled and unlabeled data meet. Often you have a few hundred labeled tickets and a hundred thousand unlabeled ones. **Semi-supervised learning** uses both: train on the few labels, let the model label the rest where it is very confident, retrain on the larger pile, and repeat, like a new employee who learns from the cases her mentor marked and then from her own surest judgments. The risk is plain, since confident mistakes are fed back as truth, and the method works best when the classes form well-separated groups. **Weak supervision** comes from the other side. Instead of labeling examples one by one, experts write quick, imperfect rules, the old keyword rules of spam filtering among them, and a model learns from where the rules agree and disagree. The rules were too crude to be the product; they are good enough to be the teacher. Together with active learning and self-supervision, which come up later in the book, these fill the space between a full answer key and none.

Rare classes, such as the few frauds among millions of honest payments, have standard treatments too. You can **reweight** the loss, so that each missed fraud costs as much as hundreds of false alarms, or **resample** the training data, showing the model the rare cases more often or the

common ones less often. Both make the model pay attention to what is rare. Both also distort its sense of how common the rare thing is, so its scores must be recalibrated, checked and corrected against how often things really happen, before anyone reads them as probabilities.

Every method in this chapter quietly assumed that the data is a table of independent rows that could be shuffled freely. Much valuable data is not. It arrives in time order, as a network, or as a list whose order is the whole point, and such data gets its own section later, once the machinery of deep learning is in place. That machinery is where we go next. Everything so far has depended on someone choosing good features to give the model; the next chapter is about models that learn to build their own. Its story begins with a newspaper headline from 1958, and with a promise that turned out to be wildly premature.

Going further. If you wondered how to choose thresholds like the deployment-risk cut-offs, section 10.3 takes that up, and if you wondered whether a model's 0.83 can really be trusted as a probability, section 10.5 does. How a cause is told apart from something that merely moves along with it is the subject of section 11.1. And if you wondered what to do with data that arrives in time order, as a network or as a ranked list, section 8.3 takes it up.

Chapter 3 — Deep Learning

3.1 The Neuron That Was Promised Too Much

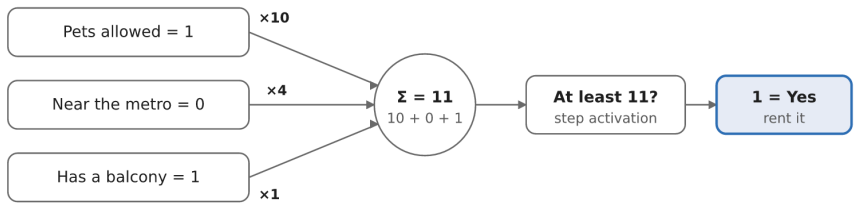
In July 1958, newspapers reported on a demonstration by the United States Navy. A young psychologist named Frank Rosenblatt had built a machine he called the **perceptron**, and after some practice it had learned to tell cards marked on the left from cards marked on the right. The coverage was breathless. The New York Times reported that the Navy expected such machines one day to walk, talk, see, write and even be aware of their own existence.

It is worth knowing what the machine actually did, because the idea behind it is both humble and the seed of everything in modern deep learning. And you have almost certainly reinvented it yourself.

Suppose you are hunting for an apartment and want to decide quickly, yes or no. You check three things: are pets allowed, is it near the metro, does it have a balcony? Each answer is 1 for yes or 0 for no. Not every criterion matters equally. You have a dog, so pets get a weight of 10. The metro gets 4. The balcony, a nice extra, gets 1. And you set a bar: you will rent only if the apartment scores at least 11.

An apartment near the metro with a balcony scores 4 + 1 = 5. Below the bar: no. One that allows pets and has a balcony scores 10 + 1 = 11. It clears the bar: yes.

Pets and balcony reach the threshold of 11 exactly



The threshold is the bias in disguise: subtract 11 from the sum and the neuron fires whenever the result is 0 or more.

That is all a perceptron is: weigh the evidence, add it up, compare with a threshold. If you have ever been on call, you may have written one without knowing its name. Many teams page an engineer with a rule like this: add 5 points if the error rate is above normal, 3 if the slowest requests are slower than usual, 8 if customers have opened tickets in the last ten minutes, and page when the total reaches 10. Rosenblatt's contribution was not the sum itself. It was a procedure that let the machine find the weights and the threshold on its own, from examples, instead of in a meeting.

For a decade the excitement held. Then, in 1969, Marvin Minsky and Seymour Papert published a book called *Perceptrons* that explained, with mathematical precision, what a single perceptron could never do. The most famous example is embarrassingly small.

Imagine a primary database and its replica. You want an alarm when the two *disagree*: exactly one of them reports a successful write. Both succeeding is fine. Both failing is a different alarm entirely. Try to set up the checklist. Any weights that fire for “primary only” and for “replica only” will also fire for “both”, or else fire for “neither”. There is no setting that works, not because you are not clever enough, but because a single weighted sum with a threshold can only split the world with one straight line, and “exactly one of two” cannot be separated by a single straight line. This is the **XOR (exclusive or) problem**, and the disappointment that followed the book helped push neural network research into a long winter.

The frustrating thing is that the way out was visible at the time. Use more than one neuron. Let one learn to detect “primary yes, replica no”. Let another detect “primary no, replica yes”. Then let a third fire if either of the first two fires. Three simple checklists, arranged in two layers, solve what one checklist never could. How many neurons and layers a real problem needs is a hyperparameter, chosen by trying a few sizes on the mock exam. In principle one hidden layer with enough neurons can approximate almost any relationship, but in practice several layers do the same job with far fewer neurons, because later layers build judgments out of the judgments of earlier ones. Stack neurons like this, with an input layer, one or more **hidden layers** and an output layer, and you have a **multi-layer perceptron**, or MLP.

Two things about such a network are easy to misunderstand, and both matter.

First, nobody assigns the hidden neurons their jobs. There is no neuron programmed to judge “the neighborhood” or “the paperwork”. The hidden neurons discover their own sub-criteria during training. Think of an experienced recruiter who, after reading ten thousand CVs, has developed gut feelings she has never named: a certain kind of gap, a certain way of describing a project. Hidden neurons are like that. They invent useful intermediate judgments, and often those judgments have no name in any human language. They do not all end up with the same judgment because they start from different random weights: if every hidden neuron started identical, every one would receive identical blame and stay identical forever, and a little randomness at the start is what lets them drift toward different specialties.

Second, there must be a bend between the layers. Each neuron’s sum passes through an **activation function** before it moves on. Without it, the layers collapse into each other. Think of converting euros to dollars and then dollars to pounds: however many conversions you chain, the result is one fixed exchange rate from euros to pounds. A hundred layers of plain weighted sums are mathematically identical to a single layer. A small bend at every step, such as **ReLU (rectified linear unit)**, which passes positive values through and turns negative ones into zero, is what lets a stack of simple neurons draw complicated boundaries. It manages this the way a curve can be drawn with many short straight segments: each neuron contributes one bend at its own place, and hundreds of bends, added and stacked, can trace almost any shape. Each neuron also has a bias, the constant met with the ruler, and it turns out to be simply the threshold moved

to the other side of the comparison: instead of “score at least 11”, the neuron computes “score minus 11” and asks whether that is positive. A single neuron whose bend is the dimmer switch, the sigmoid, computes exactly what logistic regression computes: it weighs the evidence, adds the bias and passes the sum through the S-curve. The classifier of the last chapter was a one-neuron network all along.

So why did it take until the 1980s for multi-layer networks to take off? Because knowing that a hidden layer *could* solve the problem is not the same as knowing how to find its weights. With one neuron, it is easy to see which weight was wrong. With three layers, when the final answer is wrong, which of the hundreds of weights inside is to blame, and by how much?

That question waited years for a good answer. The answer turned out to involve a mountain, a thick fog and a very careful way of passing the blame backwards.

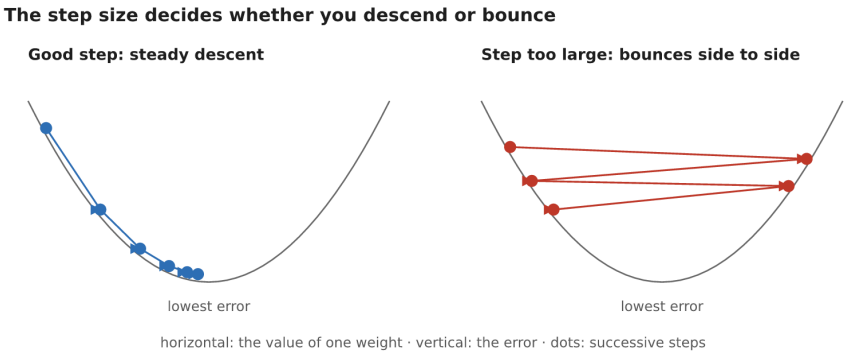
3.2 Down the Mountain in the Fog

You are standing somewhere on a mountain at night, in fog so thick you cannot see your own boots. You need to get down to the valley. You have no map and no light. What do you do?

You feel the ground. Through your soles you can sense which way it slopes, and how steeply. So you take one step in the direction that goes downhill most sharply. Then you stop, feel the ground again, and take another step. Step after step, without ever seeing the valley, you descend.

This is how virtually every modern neural network learns, and once you see it, it is hard to unsee. The mountain is the model’s error. Every position on it corresponds to one particular setting of all the weights, and the height at that position is how wrong the model is with those weights. Training means walking downhill. The walk starts at a random spot: the weights begin as small random numbers. Two runs from different starting points usually end in different valleys of similar depth, which is one reason two training runs of the same network rarely produce identical weights. The slope under your boots, measured in every direction at once, is called the **gradient**, and the method, feel the slope and step downhill, is **gradient descent**.

The length of your stride turns out to matter enormously, and it has a name: the **learning rate**. Take tiny steps and you will still be walking at sunrise. Take huge ones and something odd happens. You leap over the valley floor, land on the opposite slope, feel that it goes downhill back the way you came, leap again, and bounce from side to side without ever settling.



If that bouncing looks familiar, you may have run it in production. Picture an autoscaler configured to react hard to every change in load. Traffic rises a little; it adds ten instances. Load per instance drops sharply; it removes ten. Load spikes; it adds them back. The cluster flaps all night, and the person on call learns the hard way what a learning rate that is too high feels like. The fix, in both worlds, is the same: smaller, calmer steps. The stride is chosen partly by trial on a short training run, and in serious training it rarely stays the same from the first step to the last; how it is varied is one of the routines a later chapter opens up.

That is the walking. Now for the hard part, the question the previous section left hanging. In a network with many layers, how do you know which way the slope goes for each of thousands of weights hidden deep inside?

Picture a furniture workshop with three stations in a row: cutting, assembly and painting. A customer returns a crooked wardrobe. The inspector does not blame everyone equally, and he does not start at the beginning. He starts at the end and walks backwards. The painter received a door that was already crooked, so only a small share of the blame is his. The assembler glued it crooked because the panel arrived two millimeters short, so most of the blame travels further back, to the cutter. By the time the inspector reaches the start of the line, every station knows its own share of the defect, and each adjusts its procedure by exactly that much.

This is **backpropagation**. The error is measured at the output and passed backwards through the network, layer by layer, and at each layer the calculus rule for chained effects, the chain rule, divides the blame among the weights according to how much each one influenced the result. In 1986, David Rumelhart, Geoffrey Hinton and Ronald Williams published a short paper in *Nature* showing how well this worked for multi-layer networks, and the long winter began to thaw.

If you work with microservices, you already have an intuition for it. When a page loads slowly, distributed tracing shows you where the time went: forty milliseconds in the frontend, three hundred in the API, of which two hundred and eighty were spent waiting for one database query. You do not optimize the frontend; you fix the query. Backpropagation is distributed tracing for mistakes instead of milliseconds. It tells every weight, out of millions, how much of the final error passed through it.

Now watch our apartment checklist learn, with all the pieces in place. Its weights start out random: balcony 10, metro 1. A client is sent to a flat with an enormous balcony, nowhere near a metro station, and comes back furious. That bad review is the error. The checklist does not get torn up in a fit of temper. Backpropagation works out how much each weight contributed: the balcony pushed the score up a lot, so its weight drops a little; the metro barely counted, so its weight rises a little. Each correction is the blame multiplied by the learning rate. After tens of thousands of clients, the checklist settles somewhere like balcony 2, metro 9, and nobody ever had to tell it.

In practice, the model does not consult every client before taking a step. Computing the exact slope over millions of examples would make each step unbearably slow. Instead it looks at a small random handful, a **mini-batch** of perhaps a few dozen or a few hundred examples, estimates the slope from those, and steps. It is the logic of an opinion poll: a thousand well-chosen people tell you roughly what millions think. The estimate is noisy, so the descent wobbles, which is why the method is called **stochastic** gradient descent. One full pass through all the training data is called an **epoch**. Modern training usually adds one more refinement: optimizers such as **Adam** remember recent slopes and adapt each weight’s step size, like an experienced hiker who lengthens her stride on steady ground and shortens it on loose rock.

There is one last surprise on this mountain, and it overturns a fear that worried researchers for years. A real mountain has two horizontal directions. The error landscape of a network has one direction for every weight, millions of them. People feared that the walker would get trapped in some small hollow halfway down, a **local minimum**, unable to climb out. In such vast spaces, it turns out, hollows that trap the walker far above the valley floor are rare. Most flat spots are saddles: downhill in some direction you have not tried yet, and the jitter of mini-batches tends to find it. The walker who gets lost in practice is usually the one with the wrong stride. Nor does the walker ever know she has reached the lowest point of the whole mountain, and that matters less than it sounds: in practice you stop when the error on the mock exam stops improving, often before the training error stops falling, and stopping early, as a later chapter shows, is a feature rather than a compromise.

With backpropagation, networks could finally learn. But point one at a photograph and a new problem appears. A modest photo has a million pixels; if every neuron in the first layer needs its own weight for every pixel, the network drowns in weights before it learns anything. The way out came, in part, from an experiment with a cat in the late 1950s.

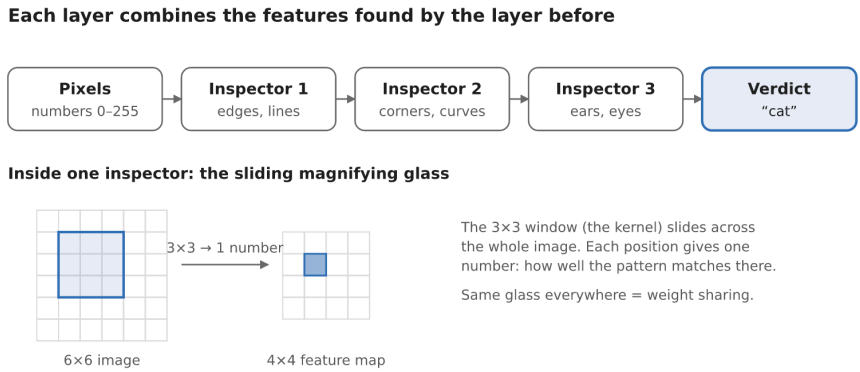
3.3 What the Cat’s Brain Revealed

In 1958, two young neuroscientists at Johns Hopkins, David Hubel and Torsten Wiesel, were recording the activity of single neurons in the visual cortex of an anesthetized cat. Their plan was to project spots of light onto a screen in front of the cat and see which spots made a neuron fire. For hours, nothing much happened. Then, as one of them slid a glass slide into the projector, its faint edge swept a thin line of shadow across the screen, and the neuron erupted, crackling through the loudspeaker like a machine gun.

The neuron did not care about spots. It cared about *edges*, at one particular angle, in one particular patch of the visual field. Over the following years, Hubel and Wiesel mapped a whole hierarchy: simple cells that respond to edges, more complex cells that respond to combinations of edges, and further stages that respond to ever richer patterns. The work earned them a Nobel Prize in 1981, and it planted an idea that computer scientists would eventually turn into one of the most successful ideas in machine learning.

Here is the idea as a story. A team of inspectors examines a photograph, each holding a small magnifying glass. The first inspector has no idea what a cat is. She only looks for sudden changes in brightness and marks where there are lines. The second inspector never sees the photo at all, only the first one's notes, and marks places where two lines meet at an angle. The third, reading those notes, notices that two sharp angles side by side look like a pair of ears. The last one puts ears, eyes and whiskers together and says: cat.

Nobody in this chain ever sees the cat whole. It is assembled from local details that grow more abstract at every level, much as you read a word from letters and letters from strokes.



The magnifying glass is something you have used yourself. The blur, sharpen and edge-detect filters in any photo editor are small grids of numbers, typically three by three, slid across the image. At every position, the grid is multiplied by the pixels beneath it and the results are summed into a single new pixel. An edge-detection grid produces a large number wherever brightness changes abruptly, and almost nothing on smooth areas. Sliding a small grid like this across an image is a **convolution**, the grid itself is a **kernel** or **filter**, and a **convolutional neural network**, or CNN, is a stack of layers that do exactly this, with one decisive twist: nobody designs the grids. Their numbers are learned through backpropagation, and the network invents whatever detectors help it reduce its error. Some end up looking remarkably like the edge detectors Hubel and Wiesel found in the cat. Many have no name at all.

This also dissolves the problem that closed the last section, the million weights per neuron. Think of how a linter works. It does not keep a separate rule for line 1, another for line 2, another for line 3 of your file. It has one rule and applies it to every line. A CNN does the same: one small detector, with its handful of weights, is applied at every position in the image, because an edge is

an edge wherever it appears. This is **weight sharing**, and it cuts the number of weights by orders of magnitude. Shifts therefore come largely for free; rotations and changes of size do not, and the network learns them only by seeing rotated and resized examples, which is why training photos are often deliberately rotated, cropped and zoomed, while the condensing of notes described next adds a little tolerance for small wobbles.

The inspectors higher up see more of the picture because, between layers, the notes get condensed. A common trick keeps only the strongest value in every two-by-two block, which is called **pooling**. Each layer therefore summarizes a larger patch of the original photo than the one before, until the inspectors at the top are effectively looking at the whole image through their notes.

The idea had a long incubation. In the late 1980s and 1990s, Yann LeCun and his colleagues trained convolutional networks to read handwritten digits, and versions of their system were used to read the amounts on bank checks. Then, in 2012, a much deeper CNN trained on graphics cards, later known as AlexNet, entered the ImageNet image-recognition competition, built on a public collection of over a million labeled photos, and won by a margin that startled the field. Within a few years, CNNs were reading medical scans, inspecting circuit boards on production lines, sorting parcels in warehouses and comparing screenshots in visual regression tests.

And then the inspectors began to show a disturbing habit. They latch on to whatever reduces the error, including things nobody intended. In a well-known demonstration, researchers deliberately trained a classifier on photos in which wolves always appeared on snow and huskies never did, then examined what it relied on. It had learned, mostly, to detect snow. Real projects can stumble into the same trap by accident: picture a defect detector that quietly learns the lighting of the one inspection station where most defective parts happened to be photographed. Researchers have also found that CNNs trained on ordinary photo collections lean on texture far more than people do. Show a person a cat-shaped outline filled with elephant skin and they say cat; such a network is often more impressed by the skin. The defense is to break the coincidence in the data: photograph defective parts at every station, find huskies on snow and wolves on grass, and test deliberately on such mismatched cases before trusting the model. The network will usually take the easiest clue that works; your job is to make sure the easy clue is the right one.

The inspectors were built for images, where everything arrives at once in a neat grid. Language is different. Words arrive one after another, sentences have no fixed length, and the meaning of a word at the end can depend on something said at the very beginning. To read a sentence, a network needs some kind of memory. The first attempts at giving it one looked a lot like a children’s party game.

3.4 The Telephone Game

At a children’s birthday party, twelve kids sit in a row. The first one whispers a sentence to the second, who whispers what she heard to the third, and so on down the line. The last child stands up and announces the message. “Grandma bought a red bicycle” has become “Grandpa caught a dead fish”, and the room collapses in giggles.

The first serious attempts to make neural networks read language worked very much like this game, and they inherited its weakness.

The idea is natural. A sentence arrives one word at a time, so let the network read one word at a time, and let it carry a running summary from each word to the next. At every step it combines the new word with the summary so far and produces an updated summary. That summary is called the **hidden state**, and a network built this way is a **recurrent neural network**, or RNN.

If you have ever written a `reduce` or a `fold`, you already know its shape. You walk through a list in order, and at each step you combine the current item with an accumulator to produce the new accumulator. An RNN is exactly that, with two twists: the accumulator is a fixed-size list of numbers, and the function that combines it with each new word is a small neural network whose weights are learned. However long the sentence, the accumulator never grows. That fixed size is both the beauty of the design and its curse. In practice the accumulator typically holds a few hundred to a couple of thousand numbers, chosen once by the designer: a lot of room for one sentence, and very little for a whole novel. The party game misleads about one thing, the cause of the loss: the children mishear by accident, while the network decides, by learned rules, what to keep and what to drop, and the trouble is only that a summary of fixed size must drop something.

To feel the curse, imagine reading a detective novel under a strange rule: you may keep only a single index card of notes, and after each page you must update it. By chapter twenty, the card holds a fine summary of the current scene. But the color of the coat the murderer wore, mentioned once in chapter two, was crossed out long ago to make room for something more recent. When the detective finally reveals that the coat was green, you have no idea why it matters.

There is a second, deeper reason for the forgetting, and it has to do with how the network learns. Training must send blame backwards, and in an RNN that means backwards through every word of the sentence. At each step back, the blame gets multiplied by a number, and if that number is typically a little below one, the signal shrinks like compound interest running in reverse. Multiply by 0.9 fifty times and about half a percent is left. (Multiply by 1.1 instead and the blame grows more than a hundredfold, the opposite trouble, called an exploding gradient, which turns out to be the easier one to tame.) The early words receive almost no correction, so the network barely learns to connect things that are far apart. This is the **vanishing gradient** problem.

In 1997, Sepp Hochreiter and Jürgen Schmidhuber proposed a cleverer accumulator called the **LSTM**, short for long short-term memory. Instead of one index card that gets overwritten every page, imagine giving each reader a small notebook with explicit operations: a switch that decides what to erase (a refinement added a few years later), a switch that decides what to write, and a switch that decides what to read out right now. These learned switches are called **gates**. In programming terms, it is the difference between a single variable overwritten in every iteration and a small struct where one field can be deliberately left untouched for a hundred iterations. The coat’s color can survive until chapter twenty. Nobody tells the switches what to erase or keep: each is a tiny neuron with a dimmer that looks at the current word and the notebook and outputs

a number between zero and one, how much to erase, write or read, and its weights are learned by backpropagation like everything else, so the network discovers for itself which kinds of details tend to be worth keeping.

LSTMs were a triumph. For roughly a decade they and their relatives powered speech recognition, phone keyboard predictions and machine translation; when Google moved its translation service to neural networks in 2016, an LSTM-based system was at its core. Recurrent ideas still appear wherever data truly arrives one tick at a time, such as sensor streams and live telemetry, and a newer family called state-space models (Mamba is a well-known example) revisits the running-summary idea with better mathematics.

But two problems refused to go away. The notebook helped, yet messages still traveled hand to hand, and over very long texts they still faded. And the hand-to-hand structure was slow: word fifty cannot be processed until word forty-nine is done, just as you cannot parallelize a `reduce` whose every step depends on the previous result. Graphics cards are built to do thousands of things at once, and an RNN makes them wait in a queue.

It is worth pausing on what the whispering was for. Each child needed to know the message so far only because they could not hear the first child directly. So in 2017, a group of researchers at Google asked a question that sounds almost naive in hindsight. What if nobody had to whisper? What if every word in a sentence could simply look at every other word, all at once? Their paper had a confident title, *Attention Is All You Need*, and it changed the field. Before we follow it, though, it is time to stop and look at what we have actually been building these last four sections. We have been calling it a neural network, and a surprising number of people, engineers included, believe that this means we have been building a small brain.

Going further. If you wondered what it really means for every word to look at every other word, section 7.2 takes the idea apart. Why stopping early helps rather than hurts is explained in section 10.1, and how researchers find out that a network has been looking at the snow instead of the wolf is the subject of section 10.6. The rest of the hiker’s equipment, from momentum and warmup to carefully chosen starting weights and a cap on exploding gradients, is in section 8.4.

Chapter 4 — What a Model Is Made Of

4.1 It Is Not a Brain

Imagine explaining your work to a friend who is a neurologist. You say your team is training a neural network to spot defective welds. She lights up, and asks whether the network has something like a visual cortex, whether it gets tired, whether it can forget. For a moment you don't know how to answer, because the honest reply is that your network has about as much in common with her patients' brains as a paper airplane has with a pigeon. Both fly. The resemblance stops there.

To see how little they share, watch one second in the life of each.

Start with a single neuron in your friend's patient, somewhere in the part of the cortex that handles touch. It is a cell, with a body about the width of a hair and a tangle of thin branches reaching out to thousands of other cells. Each branch ends in a synapse, a microscopic gap where chemicals, not wires, carry the signal across. In this one second, pulses arrive at those synapses from all directions: a few hundred from this neighbor, a dozen from that one, a burst from a third. Some arrivals push the cell toward firing; some hold it back. Inside, charge builds up like water behind a dam. Three or four times during the second, the charge crosses a threshold and the cell fires: a sharp electrical spike that races down its single output fiber to everyone it is connected to. Then the charge collapses and the dam starts filling again. Nothing about the spikes is a number. What the next cells receive is a rhythm: how many spikes, how close together, in what pattern over time. Meanwhile, quietly, the synapses themselves are changing. One that was used heavily grows a little stronger; the chemistry takes minutes and hours, and some of it will be consolidated tonight, during sleep. The whole cell, doing all of this, consumes about as much power as a dust mote falling.

Now the same second for one neuron in your weld detector. It is a row of numbers in a table. A vector of inputs arrives, perhaps a few hundred values. Each is multiplied by its weight, the products are added, a bias is added, and the total is passed through a bend, ReLU, that turns negative results into zero. One number comes out. That is the whole second, and in truth it took a few billionths of one. No rhythm, no timing, no chemistry; nothing in the cell changed, because weights change only during training, and this is not training. It will do exactly the same thing, with exactly the same numbers, the next time the same input arrives, and the time after that, forever. In principle, at least; one small exception waits near the end of the book. When a chatbot answers the same question differently each time, though, the randomness is mostly added afterwards, on purpose: the network produces practically the same list of probabilities every time, and a separate step rolls dice to pick a word from it.

One idea survives the metaphor: weigh the inputs, then fire or not

	Biological neuron	Artificial neuron
Signal	electrical spikes, timed	one number per step
Inputs	thousands of synapses	a few to a few thousand weights
Learning	chemistry, growth, sleep	nudging numbers by gradient
Speed	at most a few hundred spikes/s	billions of operations/s
Energy	about 20 W for the whole brain	kilowatts for a large model
Count	about 86 billion neurons	thousands to millions of units

Shared: many weighted inputs → one threshold → an output that feeds other units

So why the name? In 1943, a neurophysiologist, Warren McCulloch, and a young logician, Walter Pitts, wrote a paper arguing that a nerve cell could be thought of as a tiny logic unit: it receives signals from many cells, adds them up, and if the total crosses a threshold, it fires to the cells downstream. That picture, many inputs, one threshold, one output, is exactly the apartment checklist of the previous chapter. Rosenblatt borrowed it for the perceptron fifteen years later, and the word “neural” has clung to the field ever since. It is the central idea that truly crossed over: a unit that weighs many inputs, decides whether to pass something on, and is wired to many others like it. Everything else in the second you just watched, the timing, the chemistry, the slow growth, the twenty watts for the whole brain, did not come along.

Even the word “layer” has drifted. Your visual cortex does have layers, and Hubel and Wiesel’s hierarchy was a real inspiration for convolutional networks. But the brain’s wiring is a dense tangle of loops, with signals running backwards and sideways as much as forwards. The clean left-to-right stack of a network looks like a diagram of a brain from a textbook, not like a brain.

Why does any of this matter to someone who just wants to use these tools well? Because the metaphor quietly shapes expectations, and the expectations are wrong in both directions.

In one direction, the brain comparison makes networks sound more capable than they are. People assume a network “understands” images the way they do, or “remembers” a conversation, or “knows” when it is unsure. The weld detector does not understand welds. It has learned a very good function from pixel patterns to a number, and it can be fooled by a change in lighting that no human inspector would even notice. And it often does not know what it does not know, which is one root of the hallucinations we will meet later.

In the other direction, the comparison makes networks sound more mysterious than they are. Engineers who would never hesitate to debug a compiler sometimes treat a trained network as an oracle, something you can only poke at from the outside. That is a mistake. A network is a function made of arithmetic, every step of which you can print, plot and inspect. The next few sections do exactly that.

It helps to replace the brain picture with a humbler one. Think of the sound desk at a concert, the long board with hundreds of sliders, each one shaping a little of what comes out of the speakers. A neural network is that board with a million sliders, or a billion. Training is the long process of finding the slider positions that make the output match the examples. The desk breaks in one important place: on a real desk every slider has a label and does one thing, while in a network no single slider means anything on its own, and what thousands of them do together is still hard to work out, so hard that reading a network has become a research field of its own. That picture is less exciting than a brain in a box, and far more useful. It tells you that the network can only produce what its arithmetic allows, that everything it knows is in the slider positions, and that if you can read the sliders, nothing inside is, in principle, beyond reach.

And yet. The paper airplane does fly. The same humble arithmetic, scaled up and arranged well, recognizes faces, translates languages and writes code, things that twenty years ago people said would require understanding the brain first. Perhaps the most honest way to put it is this: we did not copy the brain. We borrowed one idea from it and then went our own way, and we are still a little surprised at where that way has led.

The sliders, though, raise a practical question that the brain metaphor was hiding. A slider moves a number. So before anything can be fed into this machine, a photo, a sentence, a customer, a song, it has to become numbers. That turns out to be a craft of its own, and many of the quiet failures in machine learning happen there, before any model is even involved.

4.2 Turning the World into Numbers

Picture an engineer on her first week in a new job, handed a folder and a sentence: “Build us something that predicts which support tickets will escalate.” The folder contains three years of tickets, exported from four different systems. She opens the first file expecting a tidy table and finds what every engineer eventually finds: a mess.

The ticket text is free-form English, with typos, pasted stack traces and the occasional angry paragraph in all capitals. Dates are in three formats. The “priority” column says P1 in one system, Critical in another, and 1 in a third. The customer’s country is sometimes a code, sometimes a name, sometimes blank. Some rows are duplicates. Some have the resolution time filled in as −1. And the one thing she needs, a label saying whether the ticket escalated, lives in a different file, joined by an ID that does not always match.

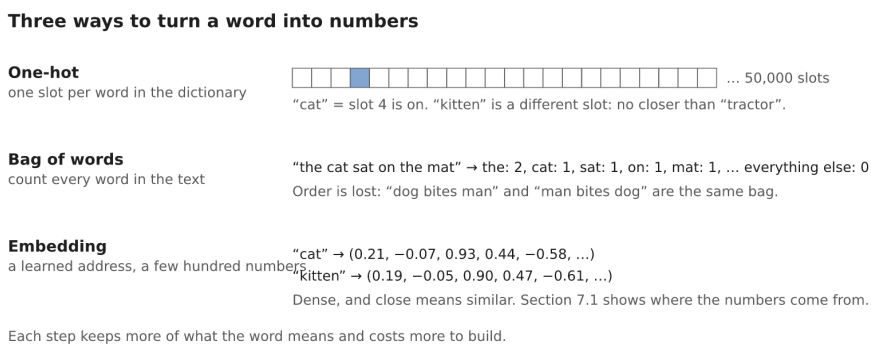
Nothing in the folder is a number a model can use. This section is about the craft of making it so, and it is the part of machine learning that no demo ever shows and most practitioners spend much of their time on. Surveys of data scientists keep finding the same thing: preparing data takes more of their week than modeling does, often far more.

Start with the easiest feature, and watch it turn out not to be easy. Resolution time, in hours. It is already a number, so it can go straight in, right? Look at the values: most tickets resolve in two to fifty hours, a handful take six hundred, and a few say −1. The −1 is someone’s way of writing “unknown”, and if she leaves it, the model will learn that a ticket resolved in negative one hour is

a very special kind of ticket. Missing values have to be found and decided on: dropped, filled with a typical value, or marked with a separate “was missing” flag so the model can learn that the gap itself means something. Then there are the six-hundred-hour tickets. On a straight scale they dwarf everything else, and we have already seen what one far-off nail does to a ruler on springs. A common move is to take the logarithm: two hours becomes about 0.7, fifty becomes about 3.9, six hundred about 6.4. The distances now reflect how people actually experience waiting, where the difference between two hours and ten feels larger than the difference between five hundred and five hundred and eight.

Now a feature that is a word: country. A model cannot multiply “Germany” by a weight. The tempting shortcut is to number the countries, Germany 1, France 2, Japan 3. It is wrong in a subtle way: it tells the model that Japan is three times Germany and that France lies halfway between them. The honest encoding is to give every country its own switch, a new input that is 1 for that country and 0 for all others. This is **one-hot encoding**, and it is exactly what a multiple-choice form does with its row of checkboxes. The cost is width: two hundred countries become two hundred inputs. A country that never appeared in training has no switch of its own, so every switch reads zero and the model treats it as “none of the countries I know”; careful data preparation adds an explicit “other” switch for this case and lumps very rare values into it during training, so the model has learned something sensible to do with it. For a feature with a few thousand possible values, product IDs or user IDs, the row of checkboxes becomes absurd, and that is where learned addresses called embeddings take over: each ID gets a short list of numbers that training adjusts, so that IDs which behave alike end up with similar numbers.

The ticket text is the hard case, and it has a history worth knowing, because the three ways of handling it are three rungs of a ladder the whole field climbed.



The first rung is to treat a document as a **bag of words**: count how many times each word in the dictionary appears, and that list of counts is the document’s numbers. Order is thrown away, “dog bites man” and “man bites dog” become identical, and the word “the” dominates every count. A small refinement called **TF-IDF (term frequency–inverse document frequency)** fixes the second problem by weighting each word by how rare it is across all documents, so “the” counts for almost nothing and “kernel panic” counts for a lot. Crude as it is, the bag of words powered

search engines and spam filters for decades, and it is still a strong first model for tickets. A ticket containing “charged twice”, “refund” and “lawyer” is very likely to escalate, and no understanding of grammar is needed to see it.

The second rung is to keep the order and cut the text into tokens, pieces of words, so that sequence models can read it in order, piece by piece. The third rung is to turn each token into an embedding, a dense address where similar words are close, which is where modern language models begin. Our engineer, in her first week, should probably start on the first rung. It takes an afternoon, it is easy to explain to her manager, and it tells her whether the text carries any signal at all before she spends a month on the third.

There is one more kind of feature, the kind you have to invent. The ticket router already showed the simplest case, a raw timestamp turned into hour of day, day of week and public holiday, and the same thinking goes much further. The customer’s plan tier, pulled from another database. The number of tickets this customer filed in the last month. Whether the ticket text mentions a competitor by name. None of these features exist in the folder; she creates them, by thinking about what an experienced support manager would look at. This is feature engineering again, and on a tabular problem like this one a mediocre model with the right features usually beats a brilliant model with the wrong ones.

That raises a question that divides the field. Does anyone still do this by hand? For images and language, mostly not: deep networks learn their own features, which is the whole point of the convolutional inspectors and of embeddings. Nobody tells a modern vision model to look for edges. For business data, the tables with a few dozen columns that most companies actually have, feature engineering by hand is alive and well, and the engineers who know the domain do it best. The honest summary is that deep learning moved the craft from inventing features to curating examples, without eliminating it.

There is a last step before the table can meet a model, and it is small and easily forgotten. Features come in wildly different units: hours, counts, a 0-or-1 switch, a log of hours. Remember the salaries in euros next to years of experience: the big numbers drown the small ones. So each feature is **scaled** to a common range, typically by subtracting its average and dividing by its spread, so that every feature speaks at roughly the same volume. Tree models do not care; almost everything else does.

And here is the trap that catches even careful people. Where do the averages and spreads for that scaling come from? If she computes them over the whole folder, test set included, then the test set has quietly leaked into training. Every number that preparation learns from the data, the average used for scaling, the typical value used to fill gaps, the dictionary of words, must be learned from the training set alone and then merely applied to the rest. The practical discipline is to put every preparation step into a single pipeline object that is fitted once, on training data, and then applied unchanged everywhere else. Treat it like a build step: deterministic, versioned, and run the same way in the notebook and in production.

Our engineer ends her first week with no model at all. She has a table: one row per ticket, forty-odd columns, every value a number on a sensible scale, a clean label, and a pipeline that can produce the same table from next month's tickets. Her manager is a little disappointed. He should not be. She has done the part that most projects get wrong, and the model that comes next will be good or bad largely because of it.

Which leaves the obvious question: what does the model actually do with that table? The answer, surprisingly, is that it never does anything else *but* tables.

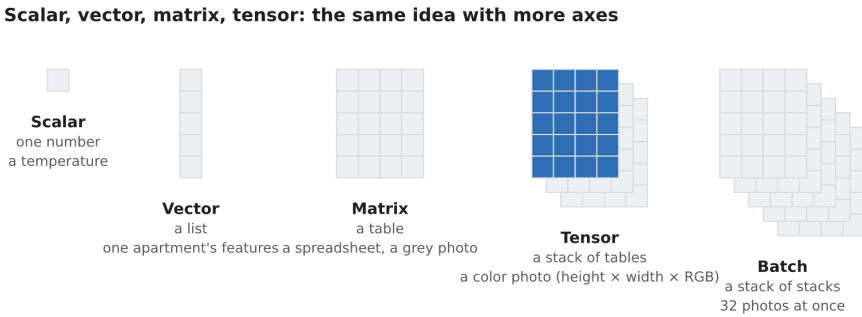
4.3 Everything Is a Table

A colleague sends you a file called `weld_detector.safetensors`. It is four hundred megabytes. There is no code in it, no documentation, no README. "That's the model," she says. "Everything it knows is in there."

It is a strange claim, and worth testing. So open the file. Not with a framework, just with a script that reads it raw, the way you would peek inside any unfamiliar binary.

What you find is almost disappointing. The file is a list of named blocks, and every block is the same kind of thing: a grid of decimal numbers, with a label and a shape. `layer1.weight`, shape (32, 3, 3, 3), 864 numbers. `layer1.bias`, shape (32,), 32 numbers. `layer2.weight`, shape (64, 32, 3, 3), 18,432 numbers. And so on, twenty-odd blocks, down to `classifier.weight` and `classifier.bias`. Add up every number in every block and you get about a hundred million. That is the model: a hundred million decimals, sorted into labeled grids. No rules, no code, no cat-shaped anything. The sliders from the start of this chapter, written down.

The grids have a name, **tensor**, which sounds far grander than what it is. Build it up from things you already know. A single number is a **scalar**: today's temperature. A list of numbers is a **vector**: one ticket's forty features from the last section, in a fixed order. A table of numbers is a **matrix**: the whole spreadsheet, or a black-and-white photo, where every cell holds the brightness of one pixel. Keep adding directions and you have a tensor. A color photo has three: height, width, and the three color channels. A video has four: time, height, width, color. A stack of thirty-two photos handed to the model together also has four: which photo, height, width, color.

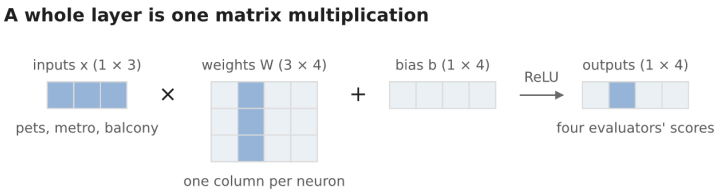


The **shape** is just the list of how long each direction is, and once you can read shapes as sentences, half the mystery of deep learning code evaporates. `layer1.weight` with shape (32, 3, 3, 3) reads: thirty-two filters, each looking at three color channels, each three pixels by three pixels. It is a box of thirty-two of the convolutional magnifying glasses, and now you know they are three by three. Many of the errors you will hit in this field are shape mismatches, the moment you try to feed a (32, 224, 224, 3) into something that expected (32, 3, 224, 224). It is one of the oldest bugs in programming, arguments in the wrong order, in a world where the compiler cannot catch it for you.

So far the file is just storage. The interesting question is how a hundred million numbers at rest become a verdict about a weld. To see it, shrink the problem until you can do it by hand.

Go back to the apartment checklist: three inputs, three weights, add them up, compare with a threshold. Write the three answers as a short row, (1, 0, 1), and the three weights as a short column, (10, 4, 1). Multiply each pair and add: 10 plus 0 plus 1 is 11. That little operation, row against column, multiply and sum, is called a **dot product**, and it is one neuron doing its whole job.

Now suppose four evaluators look at the same apartment, each with their own weights: the beginning of a hidden layer. Stand their four weight columns side by side and you have a grid, three rows by four columns. Multiply the input row by that grid and four numbers come out at once, one per evaluator. That is **matrix multiplication**, and here is the sentence that this section exists to deliver: a whole layer of a network is exactly one of these, followed by adding the biases and bending every result with the activation function. `layer1.weight` is a grid of that kind. The forward pass is the input multiplied through one grid after another, layer by layer, until the last grid produces a single number: defective, or not.



The highlighted column is one neuron: its three weights meet the three inputs, and the sum lands in one output cell. Four neurons side by side are four columns. A batch of 32 apartments turns x into a 32 × 3 table, and nothing else changes.

Every cell of the result is independent of the others, which is exactly what a GPU with thousands of small cores is built for.

The picture pays for itself the moment you have more than one apartment. Say thirty-two of them arrive at once. You do not loop over them. You stack the thirty-two input rows into a table, thirty-two rows by three columns, and multiply it by the very same three-by-four grid. Out comes a thirty-two-by-four table: every apartment, every evaluator, in one operation. This is why deep learning code almost never contains a `for` loop over examples. The loop is folded into the multiplication, and that fold is where the speed comes from.

Which finally explains the graphics cards. Why are the chips designed to draw video games the engines of modern AI? Look at the thirty-two-by-four result. Every cell in it is a dot product that can be computed without knowing anything about any other cell. A CPU is a handful of very clever cores, built to run one complicated thing fast. A GPU is thousands of simple cores, built to do the same simple thing to thousands of pixels at once. Matrix multiplication is thousands of identical dot products that do not depend on each other. It was practically designed for the GPU, decades before anyone intended it. When the 2012 ImageNet result arrived, its authors had trained on two consumer graphics cards in a bedroom, and much of the deep learning decade since has been the story of larger grids on more cards. Today a single data-center graphics card holds tens of gigabytes of memory, sometimes well over a hundred, and performs on the order of a thousand trillion simple multiplications per second at low precision; the largest training runs tie together tens of thousands of such cards for months.

So your colleague was right, and now you can see exactly how. Load the grids into memory. Turn a photo into its tensor of pixels. Multiply it through grid one, bend; grid two, bend; and so on to the end. One number comes out. Nothing else happens. The brain metaphor can go; the model is a spreadsheet that has learned, and running it is arithmetic you could, in principle, do with a pencil, if you had a few years to spare.

One last look at the file, because its size tells a story too. A hundred million numbers, four hundred megabytes: that is four bytes per number, the usual 32-bit floating-point format. Scale up to a language model with seven billion numbers and the same format needs twenty-eight gigabytes, more memory than most graphics cards have. So engineers store the very same numbers with less precision, sixteen bits, eight, sometimes four. This is **quantization**, and it is a large part of the reason a model that needed a server last year can run on a laptop this year. Think of it as JPEG for weights. A little fidelity is lost, a great deal of room is gained, and most of the time the picture still looks fine. The weld detector, quantized to eight bits, would be a hundred

megabytes and, very likely, just as good. This works because most weights are small numbers clustered near zero and each output adds up thousands of products, so the small rounding errors largely cancel, like the guesses at Galton's fair. Push too far, to four bits or fewer without care, and quality does drop, especially on hard questions and in smaller models, which is why quantized models are re-tested before they ship.

You have seen the material. But the file had twenty-odd grids, with different shapes and different names, and somebody decided that there would be exactly these, in exactly this order. The box of pieces they chose from, and the problems that forced each piece into existence, are the subject of the next chapter. First, one more question about the material itself: of everything a model is given, how does it decide what matters?

4.4 What Matters, and Who Decides

A company advertises one engineering job and receives two hundred CVs. Four people help sort them, and each decides what matters in a different way. The junior recruiter works from a scoring sheet handed down by HR: three points per year of experience, five for a relevant degree, two for each programming language on the list, the same sheet for every CV. The second follows a flowchart: has the candidate shipped a production system? If yes, have they worked in a team larger than ten? Each answer sends the CV down one branch or another. The third, a recruiter with twenty years behind her, reads a CV and simply has a feeling about it, built from thousands of small observations she has never named. And the fourth does something the others do not: she lets each CV tell her where to look. On one, a two-year gap catches her eye, and she goes looking for the line that explains it. On another, the gap is irrelevant, because the project list says everything. What she reads closely changes from one CV to the next.

Between them, the four recruiters stand for the main ways a model decides what matters, and they are worth taking apart, because "what matters to the model?" is one of the most frequently asked and least precisely answered questions in the field. The first three stand for models met so far: the ruler, the trees and the deep networks. The fourth stands for a mechanism called attention, at the heart of today's language models, and she will have to wait her turn until the book reaches language.

Start with the word itself. A feature has meant two slightly different things in this book so far, and will soon mean a third. At first it was a property of each example that you hand the model, often written down as a column in a table: lines changed, hour of day, country as a row of switches. Then the hidden neurons invented their own features, intermediate judgments like the experienced recruiter's unnamed hunches. And once the book reaches language, meaning will live in directions on a map, blends of many coordinates, which is usually what researchers mean when they talk about the features inside a large network. A single pixel, by contrast, is only an input: the features of an image, an edge, an ear, a patch of fur, are what the network builds out of many pixels. A word sits in between. It arrives as a mere entry number in a vocabulary and becomes something meaningful only once it has its embedding, its learned address, and has been read together with the words around it.

Now the scoring sheet. A linear model, the ruler from the toolbox, decides what matters once, during training, by settling on one weight per feature, and then applies the same weights to every case. That makes it tempting to read the weights as importance: biggest weight, most important feature. This works only with care. A weight is measured per unit of its input, so a weight per megabyte and a weight per request header cannot be compared until both features have been brought to a common scale. And when two features say nearly the same thing, say lines added and lines changed, the model can split the credit between them almost arbitrarily: a large weight on one and a negative weight on the other can produce much the same predictions as two modest weights. Retrain on slightly different data and the split may change while the predictions barely move. The scoring sheet is readable, but what you are reading is one of many sheets that would have worked about as well.

The flowchart decides what matters by the sock-drawer test: at each step, the feature whose question tidies the drawers most wins. The ranking that adds up each feature's tidying, the one most libraries print by default, is a useful first look, and a biased one. A feature with thousands of distinct values, a user ID or an exact timestamp, offers thousands of possible cut-offs, and by sheer number of tries some of them will tidy the training drawers by accident. Such features tend to look more important than they are.

There is a more honest test, and it works for any model at all. Take the validation set and, for one feature only, shuffle its values between examples, so that each ticket now carries some other ticket's country while everything else stays in place. Then measure the score again. If it drops sharply, the model relied on that feature; if it barely moves, the model can do without it. This is **permutation importance**, introduced by the statistician Leo Breiman in his work on random forests, and its great merit is that it asks the model rather than the analyst. It has a blind spot too, and it is the same pair of twins. Shuffle "lines added" while "lines changed" is still intact, and the model leans on the twin, the score hardly drops, and both features can look unimportant even though together they carry the prediction.

One sentence about all of these measures needs saying plainly, because it is the most common misreading in the field. Importance means that the model used the feature. It never, on its own, means that the feature causes the outcome. Fed the right data, a model could rank ice cream sales as an excellent predictor of drownings.

The experienced recruiter is the multi-layer network. Nobody tells its hidden neurons what to look for; backpropagation shapes each one into whatever intermediate judgment lowers the loss. That also means there is no single place to point at: its verdict on a case runs through hundreds of hunches, each mixing many inputs. You can still ask from the outside which inputs mattered, by shuffling or with tools built for looking inside networks, which come near the end of the book, but the answer is now a summary of something far more tangled.

Which leaves the practical question every team eventually asks: should I just give the model every feature I can find? More features are not free. Each one is another dimension in which nearest neighbors drift apart, another chance to latch on to an accident, the white plate or the snow behind the wolves, and another possible leak. A remedy called regularization, which

charges the model a small price for complexity, softens the cost, and one variant of it pushes the weights of useless features to exactly zero, quietly removing them; plenty of data softens it more. But the sensible default is the one a good recruiter follows: include what you have a reason to believe could matter, check what the model actually relied on, and be suspicious when a feature you thought minor turns out to dominate.

Here the hiring panel stops being a good picture. A recruiter can tell you why she rejected a CV, and her reasons connect to what the job actually requires. A model's notion of mattering is only a statistical association with its loss, on the data it happened to see. Change the data, hire in a different city or a different decade, and what matters to the model changes with it, without the model noticing and without anyone being told.

Going further. If you wondered why a chatbot answers the same question differently each time, section 7.3 explains the dice and the setting called temperature. How words get their learned addresses, the embeddings, is the subject of section 7.1, and how the fourth recruiter reads, letting each case tell her where to look, is the attention of section 7.2, and section 7.5 comes back to her to show what she can and cannot see.

Chapter 5 — How Models Are Built and Run

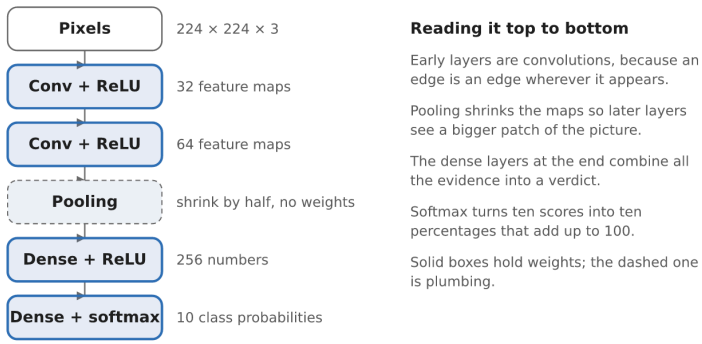
5.1 The Box of Lego

Suppose you decide to build an image classifier from scratch, armed with nothing but the two ideas from the last sections: a layer is a grid of weights, and a stack of layers with bends between them can learn almost anything. Ten thousand photos of handwritten digits, labeled. How hard can it be?

Your first design is the obvious one. Flatten each photo into a long row of pixels, 784 of them for a small 28-by-28 image, and feed that row into a dense layer with a few hundred neurons, then another, then ten outputs for the ten digits. Train it. It works, more or less: about 97% correct. Encouraging. Then you try real photos, 224 by 224 in color, and the design collapses. The first layer alone would need 150,000 inputs times a few hundred neurons, tens of millions of weights, just to look at the pixels. Worse, a “7” shifted two pixels to the right lands on completely different inputs, and your model has to learn it all over again from scratch. It has no idea that a seven is a seven wherever it sits.

You have just discovered why convolutions exist. Replace the first dense layers with the inspectors’ magnifying glasses from the story of the cat’s brain, each a small grid of weights shared by every position, with the pooling step between them that shrinks the maps. The number of weights drops by a thousandfold. The shifted seven is recognized because the glass that found it on the left finds it on the right. Your network now has the shape that LeCun’s digit readers had in the 1990s, and it reads real photos.

Snapping pieces together: a small image classifier



Emboldened, you make it deeper. If four layers are good, surely twenty are better? You stack them and train. The loss does not fall. It sits there. You try thirty layers and it gets *worse* than the four-layer version, on the training data, which should be impossible: a deeper network could at

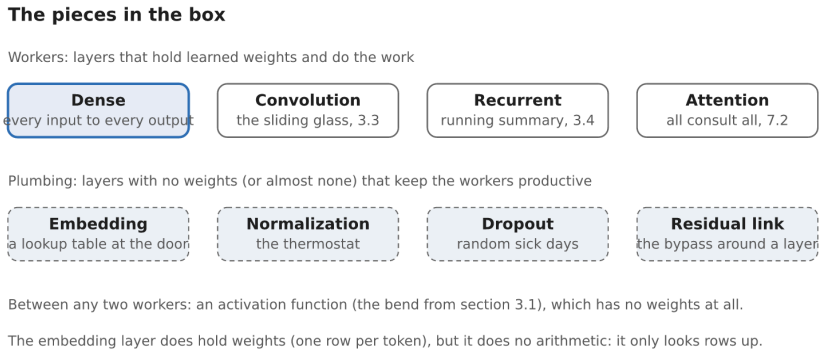
least copy the shallow one and do nothing in the extra layers. Something in the training itself is failing. This is the wall that deep learning hit around 2014, and two inventions knocked it down.

The first came from watching the numbers. As a signal passes through many layers, its scale drifts: after a few layers every value is tiny, after a few more everything is enormous, and gradient descent copes badly with either. The fix was almost embarrassingly practical. After each layer, rescale the numbers back to a standard range, the way a thermostat brings a room back to twenty degrees whatever the weather outside. This is **normalization**, introduced in 2015 under the name batch normalization, and with it, together with careful initialization of the weights, deep networks became far easier to train. Language models use a slightly different thermostat, one per token rather than one per batch.

The second came from asking why a deeper network could not at least copy a shallower one. It could in theory, but learning to pass a signal through a layer *unchanged* is surprisingly hard for a stack of weights that starts out random. So why make it learn that? Give every layer a bypass. Instead of a layer replacing its input with its output, it *adds* its output to the input, and the sum moves on. The layer only has to learn what to change; if it has nothing to add, zero is an easy thing to learn, and the signal flows through untouched. This is the **residual connection**, and the architecture built on it, ResNet, won the 2015 ImageNet competition with 152 layers, a depth far beyond anything that had worked a year earlier. Engineers may recognize the pattern from a different world: a middleware chain where each step can modify a request or pass it along untouched, rather than one where each step must rebuild the whole request from scratch. And there was a bonus. Backpropagation, the blame walking back from the output, now had an express lane: it could flow straight down the bypasses without being diluted by every layer in between.

Your thirty-layer network trains beautifully now. It scores 99.8% on the training photos. On new photos it manages 91%. You have built a crammer: it has memorized the training photos instead of learning to read digits. One remedy for it, in layer form, is odd enough to be memorable. During training, randomly switch off a fraction of the neurons in each layer, different ones on every step. During real use, switch nothing off. The network can no longer rely on any particular neuron always being there, so it learns redundantly, many neurons sharing each job. This is **dropout**, and it is the neural version of a manager who randomly sends people home during rehearsals so the team learns to cover for each other. Your new-photo score climbs to 96%.

Step back and look at what you have built, because the whole box is now on the table.



There are two kinds of pieces. The first kind does the work and holds learned weights, and each one embodies a bet about the data. The **dense layer** bets nothing; it connects everything to everything, which makes it general and expensive. The convolution bets that nearby things belong together and that a pattern means the same thing wherever it appears, the right bet for images, audio and anything on a grid. The **recurrent layer** bets that things arrive in order and the past matters. The **attention layer**, the heart of today's language models, bets that anything in the input might matter to anything else, and lets the input itself decide what. When a layer's bet matches the data, it learns from far fewer examples, because it does not have to discover what it already assumes. When the bet is wrong, no amount of data fully rescues it.

The second kind is plumbing: pieces with few or no weights that exist to keep the first kind trainable. Normalization, the thermostat. The residual connection, the bypass. Dropout, the rehearsal sick days. The activation function, the bend between layers, so thin it hardly counts as a piece and so essential you never leave it out. And at the door, for anything that arrives as a name rather than a number, the **embedding layer**: a lookup table with one row per possible token, product or user, whose rows are learned. It is how a language model begins. Unlike real Lego, though, the pieces are not finished when you snap them together: each one takes its shape during training, alongside its neighbors, so a layer pulled out of one trained model cannot simply be clicked into another without further training.

Now the reveal that makes architecture diagrams readable. From the outside, the zoo of model names, ResNet, U-Net, BERT, GPT, looks endless. From the inside, nearly all of them are these same ten or so pieces, arranged in different orders. And the arrangement follows a pattern you have just rediscovered on your own: specialized, assumption-rich workers near the input, where the data still has its raw structure, and general, assumption-free workers near the output, where the structure has been boiled down to evidence. A speech model starts with convolutions over audio and ends with dense layers. A Transformer starts with an embedding and ends with a dense layer and a **softmax** over the vocabulary, the plumbing piece that turns a row of scores into percentages adding to one hundred. It makes every score positive by raising a fixed number to the power of the score, then divides each result by their total; because of the powers, a score just a few points ahead of the rest ends up with nearly all the percentage, which is why a model often

looks more decisive than its raw scores suggest. The middles differ. The ends are nearly always the same. As for who designs a new middle, mostly nobody does from scratch: teams take a blueprint that has already proved itself, change one piece at a time and keep the change only if the mock-exam score improves; automated searches that try thousands of arrangements exist too, but their expense means most practical work is careful reuse.

If you want a mental model that feels comfortable, think of a data pipeline. Each layer is a stage that accepts a tensor of one shape and emits a tensor of another. Convolutions make the maps smaller and deeper, pooling halves them, dense layers change the length of a vector. The architecture is a type signature, shape in to shape out, and most real design conversations are about which shapes flow where. The first thing to do with any unfamiliar model is to print the shape after every stage.

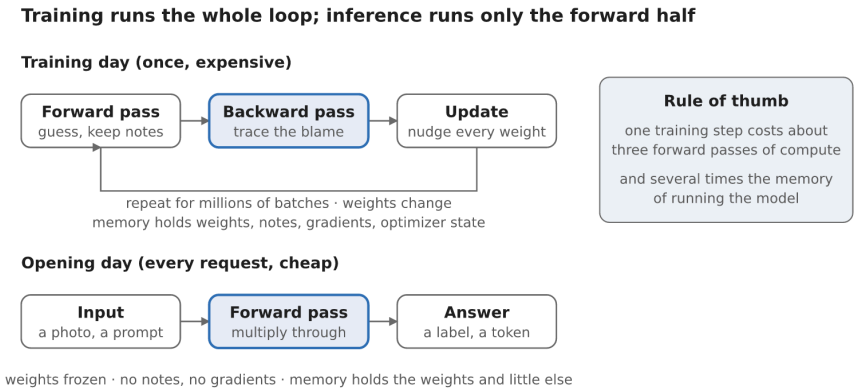
You now have a working network, and a notion of why each part of it is there. What you do not yet have is a feel for what it costs, and in particular why the afternoon you spent training it will be dwarfed, over the following years, by the cost of simply running it.

5.2 The Two Lives of a Model

A restaurant has two completely different days. There is the year before it opens: the chef tasting hundreds of versions of each dish, adjusting the salt, changing the supplier, rewriting the menu three times, burning through money with no customers in sight. And there is opening day, and every day after it: the recipes are fixed, the kitchen executes them, and the only question is how many plates can go out per hour and at what cost.

A model has exactly these two lives, and almost everything confusing about the economics of machine learning comes from mixing them up.

The first life is training. This is the loop from the preface, run in earnest: forward pass, measure the error, backward pass, nudge every weight, next batch. It happens once, or once per version. It is the year before opening. The second life is **inference**: the model is finished, its weights are frozen, and you hand it inputs and collect answers. A photo goes in, “cat” comes out. A prompt goes in, a token comes out. This is opening day, repeated for every request, for as long as the model is in service. Here the restaurant picture breaks in an instructive way: a real kitchen learns from every complaint on opening day, while a model learns nothing from its millionth customer, and whatever it gets wrong it will keep getting wrong until someone trains a new version.



What exactly is different between the two? Walk through one training step and count what the machine has to keep in memory. The forward pass runs the input through every layer, but it must also *keep notes*: the intermediate result at every layer, because the backward pass will need them to trace the blame, exactly as the inspector in the furniture workshop needed to see each station's work. Then the backward pass computes a gradient for every weight, which is a second copy of the model's size. Then the optimizer, if it is Adam, keeps a running memory of past gradients for each weight, which is two more copies. Add it up and training a model needs several times the memory of simply holding its weights, and roughly three times the compute of a forward pass, per step, for millions of steps.

Inference throws nearly all of that away. No notes, because nothing will be traced back. No gradients, because nothing will change. No optimizer state. The weights sit in memory, the input multiplies through them, and the answer comes out. It is the forward pass alone, once. This is why a model that needed a cluster of graphics cards to train can often run on far smaller hardware, sometimes even a phone; and it is why running a model is cheap per request but can still become the dominant cost, because opening day happens a million times.

The two lives also have different bugs, and recognizing which life you are in saves hours. Training problems are about *whether the model learns*: the loss does not fall, or falls and then explodes, or falls on the training set while rising on validation. Those are learning-rate problems, data problems, the crammer that memorizes and the slacker that never learns enough. Inference problems are about *speed, memory and consistency*: the answer takes too long, the model does not fit on the device, or the same input gives slightly different answers in production than it did in the notebook. The last one has a classic cause. Dropout switches signals off during training and must be switched off itself at inference; forget the one line that flips the model into evaluation mode, and your production answers wobble randomly.

Somewhere between the two lives sits a trick that engineers meet constantly: **batching**. At a bakery, the oven costs the same to heat whether you bake one loaf or twenty, so you fill it. A graphics card is the same. Often, the expensive part of inference is moving the weights from memory into the processor, and that cost is paid once per forward pass no matter how many

inputs ride along. So inference servers wait a few milliseconds to collect requests, stack them into one batch tensor, and run them through together. This costs so little because a batch of thirty-two is just a taller table in the same multiplication. The trade-off is latency against throughput, and anyone who has tuned a message queue will recognize the dial.

Language models add one twist to opening day that is worth understanding, because it explains the bills. A classifier runs one forward pass per request. A language model runs one forward pass *per token it generates*, because each new token is fed back in as input for the next. A five-hundred-word answer is several hundred forward passes, one after another, and the model cannot start the second until the first is done. That is why long answers cost more than short ones, why output tokens are usually priced higher than input tokens, and why so much engineering effort goes into caching the intermediate results from earlier tokens so they are not recomputed every time.

To make all of this concrete, follow one request to the bill. Say a company runs a customer-support assistant on a hosted language model that charges, in round numbers, one dollar per million tokens in and four dollars per million tokens out. A customer’s question, plus the instructions and the three retrieved passages the assistant was given, comes to two thousand tokens. The answer is three hundred tokens. The input costs two thousand millionths of a dollar, a fifth of a cent. The output costs twelve hundred millionths, a little over a tenth of a cent. One conversation, about a third of a cent. That sounds like nothing until the assistant handles forty thousand conversations a day, at which point it is roughly a hundred and thirty dollars a day and about fifty thousand a year, and the finance team starts asking why output tokens cost four times as much as input tokens. Much of the answer is the token-by-token loop above: the output tokens are produced one at a time, each with its own forward pass, while the input tokens are all processed together in one batch. It is the difference between serving a dish that is already plated and cooking each spoonful to order. The same arithmetic explains why the engineers on the team spend their time on shorter prompts, caching, and smaller models for the easy questions.

So: training is the expensive year of tasting and adjusting; inference is the cheap, repeated day of serving, which over the life of a widely used model often ends up costing more; and the model file that passes from one life to the other is nothing but the pile of grids from the previous chapter, with every number finally at rest. Between the two lives there is a shorter one, in which a finished model goes back to school for a few weeks on a narrower job. That is fine-tuning, which gets a section of its own later in the book.

Before we leave the engine room, one detour. Every neural network in this book so far has stored knowledge the same way: as weights learned slowly by gradient descent. There is an older and stranger idea for how a machine might remember, one that learns in a single step and treats a thought as a very long, nearly random string of numbers. It has been quietly influencing how people think about embeddings, and it deserves a section of its own.

5.3 A Different Kind of Memory

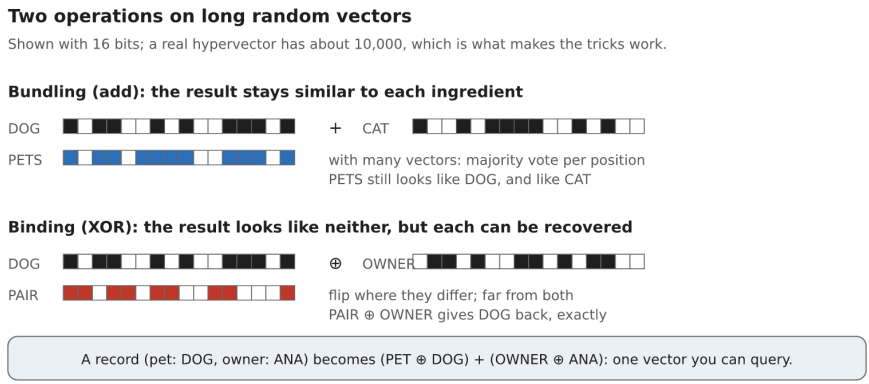
Take two fair coins and flip each one ten thousand times, writing down the results as two long strings of heads and tails. Now compare the strings position by position. How many positions match?

About five thousand, give or take a hundred. That is simply what chance looks like. And here is the thing that makes a whole branch of computing possible: it is practically impossible, in ten thousand flips, to get two strings that match in six thousand positions by accident. The odds are astronomically small. So in the world of ten-thousand-bit strings, “random” means “about half alike”, and anything noticeably more alike than half cannot be a coincidence. It has to mean something.

This is the starting point of **hyperdimensional computing**, also called vector symbolic architectures, or VSA, an idea developed over several decades by researchers such as Tony Plate and Pentti Kanerva. It is an approach that does not learn by gradient descent at all, and seeing it side by side with neural networks sharpens what each one is actually doing.

The idea begins with a bold assignment. Every concept gets a random string of ten thousand bits, drawn once and never changed. DOG is a random string. CAT is another. OWNER, ANA, MONDAY, each gets its own. Because of the coin-flip fact, all these strings are nearly equally far from one another, about half alike, which is the vector way of saying “unrelated”. Nothing has been learned yet. The strings are just names, long enough that no two will ever be confused.

Then come two operations, and together they are the whole grammar.



The first is **bundling**: add vectors together, by taking a majority vote at each position. Bundle DOG and CAT and you get a new string, call it PETS, that is noticeably more than half alike to DOG, and noticeably more than half alike to CAT. The result resembles its ingredients. You can bundle a dozen things into one vector and still ask of it, later, “is DOG in here?”, and get a confident yes by measuring similarity. Bundling is a set, stored as a single string. Each new item

adds a little noise to all the others: with ten thousand bits, a set of a few dozen to a couple of hundred items can still be queried reliably, and beyond that the echoes of the members begin to sink into the noise.

The second is **binding**: combine two vectors with XOR, which writes a 1 wherever their bits differ and a 0 wherever they agree; put another way, it flips the bits of one vector wherever the other has a 1. Bind DOG with OWNER and you get a string that is about half alike to DOG and half alike to OWNER, which is to say it looks like neither. It is a new, unrelated name. And XOR has a lovely property: bind the result with OWNER again, and DOG drops out, exact, because the second XOR flips back exactly the bits the first one flipped. Binding glues two things into a pair, and the glue is reversible.

Put the two together and you can store a whole record in one vector. A database row such as *pet: DOG, owner: ANA, day: MONDAY* becomes (PET bound with DOG) bundled with (OWNER bound with ANA) bundled with (DAY bound with MONDAY). One string of ten thousand bits. To ask “who is the owner?”, bind the record with OWNER; the ANA part falls out clearly, and the other parts turn into noise that the sheer length of the vector washes away. It is a key-value store where the entire table, the keys and the lookup are all the same kind of object: a long string of bits.

If you have ever used a Bloom filter, this will feel familiar. A Bloom filter also stores a set in a fixed-size bit array, also answers membership questions quickly, and also pays for its compactness with a small chance of confusion. Hyperdimensional computing is that idea grown up: not just “is X in the set?” but “what is X paired with?”, with the structure of the data living in the arithmetic.

Now, why should an engineer who works with neural networks care?

Because the comparison is clarifying. In a neural network, knowledge lives in the weights, and getting it there takes millions of gradient steps. In hyperdimensional computing, knowledge lives in the vectors themselves, and getting it there takes one bundle or one bind. There is no training loop. Learning a new fact is one operation, and forgetting it is one operation too. That is close to how a database behaves and very far from how a network behaves, where adding a single new fact means another round of training, and where a model that must not forget an old skill while learning a new one is a known headache, with the slightly dramatic name *catastrophic forgetting*.

The approach also explains something about embeddings, where similar things get nearby vectors. The coin-flip fact tells you why that works at all in hundreds of dimensions. In a high-dimensional space there is an enormous amount of room: you can place millions of concepts so that each is far from all the others, and still have a clear notion of “closer than chance”. Low-dimensional intuition fails here. In three dimensions you cannot fit many mutually distant points. In ten thousand, you can fit more than you will ever need, and that abundance of room is what both embeddings and hypervectors quietly rely on. This is the fact that made everyone a stranger in the story of the nearest neighbors, seen from the other side. Random points in many dimensions are all about equally far apart. For nearest-neighbor search on raw features, where

“near” was supposed to mean something, that is ruinous; for hyperdimensional computing, whose names are meant to be unrelated, it is exactly what is wanted. Learned embeddings then do the extra work of pulling related things close.

There is a price, and it is why hyperdimensional computing has stayed mostly in research labs and specialized hardware rather than in your IDE (integrated development environment, your code editor). The random names know nothing. DOG and CAT start out as unrelated as DOG and SPREADSHEET, and any similarity between them has to be built in by hand or by bundling. A neural network, by contrast, *discovers* that dogs and cats belong together from millions of examples, and discovers structure nobody thought to encode. Hyperdimensional computing is robust to noise, fast, transparent and cheap to run on tiny devices, which is why it attracts interest for sensors and low-power chips, and why some researchers see it as a model of how memory might work in a brain. But it does not learn from raw data the way networks do.

It is tempting to see the two as rivals, and more useful to see them as complementary. Networks are superb at turning messy raw input, pixels, audio, text, into meaningful vectors. Hyperdimensional methods are superb at composing vectors into structures, records, sequences, relations, and querying them without any further training. Several research groups are experimenting with exactly that handoff: a network as the perception layer that produces vectors, and hypervector arithmetic as the memory and reasoning layer that stores and combines them.

And there is one more reason to keep this section in mind for later. The attention mechanism, the fuzzy dictionary lookup at the heart of today’s large language models, is also a way of storing and retrieving key-value pairs as vectors, and of blending many of them into one. It arrived from a different direction and is learned rather than assigned, but when you see queries matched against keys to pull out values, you may find yourself thinking of a record bound with OWNER, and ANA falling out of the noise.

The engine room tour is nearly over. We know what a network is made of, how its pieces snap together, what happens on training day and on opening day, and we have seen one road not taken. One practical question remains, the one every project asks before it starts: how much data, how large a model and how much money is enough?

5.4 How Much Is Enough

Anyone who has taken up the guitar as an adult knows the shape of the first year. In the first month, progress is dramatic: from fumbling a single chord to strumming a whole song. By month six the gains are still real, but smaller, and each one costs more evenings. By the second year, another hundred hours of the same exercises barely moves you, and what would help is something else entirely: a teacher who spots the habit holding you back, or a better instrument that stays in tune. Practice still matters. It has simply stopped being the bottleneck.

Models learn along the same kind of curve, and the questions people ask most about a project, how many examples do we need, would a bigger model help, what will all this cost, are really questions about where on that curve they are standing.

Start with the most practical tool, because it answers the first question for your own data rather than for someone else's. Train the model on 10% of the training data, then on 20%, then 50%, then all of it, and each time measure the error on the same validation set. Plot the four points. This is a **learning curve**, and its shape is a verdict. If the validation error is still falling steeply at 100%, more examples will very likely help, and labeling another batch is money well spent. If it has flattened, more of the same data will buy little; you are the guitarist in year two. Combine it with a simple habit, always reading the training error beside the validation error, and you can usually tell which lever to pull. A large gap between the two that narrows as data grows is the crammer again, starved of examples, and more data will help. Two high errors close together mean a model too simple for the job, which more examples will not cure, though a more expressive model or better features might. Both errors low and flat means you are near what this data can tell you, and the next gain will come from cleaner labels or a sharper question.

People still want a number, and it is fair to give rough orders of magnitude, as long as they are read as starting points for that experiment rather than as promises. A simple classifier on clean tabular data with a handful of informative features is often useful with a few hundred to a few thousand labeled rows per class, and the rarer class usually sets the requirement: a thousand frauds matter more than a million honest payments. An image model built by transfer learning, starting from a network already trained on millions of other photos, can do well with a few hundred to a few thousand photos. Teaching a language model a format or a tone, by fine-tuning it or with LoRA, a light method that trains only a small add-on to the frozen model, often takes hundreds to a few thousand carefully chosen examples, not millions; giving it new knowledge is usually better done with retrieval, letting it look facts up in your documents as it answers. And pretraining a large language model from scratch is another universe, with training text measured in trillions of tokens.

Two warnings travel with every one of those numbers. Quality beats quantity, by a wide margin: the Friday-afternoon "flaky" labels do not improve when there are more of them; they only teach the wrong lesson more firmly. And the examples must cover the cases the model will meet. Here the guitar picture breaks. For a musician, one hour of scales is much like another. For a model, a thousand examples that are near copies of each other teach about as much as one, while a hundred examples from the corners where it fails teach a great deal. That is the whole idea behind **active learning**, in which the model itself points to the examples it would most like a human to label, and behind the unglamorous habit of removing duplicates before training.

Now the larger question, the one behind a decade of headlines: why did models keep getting bigger? Because, for language models at least, bigger turned out to be predictable. In 2020 a team at OpenAI led by Jared Kaplan trained families of language models of very different sizes on different amounts of text, and found that the loss fell along remarkably smooth curves as the number of parameters, the amount of data and the amount of compute grew, across many orders of magnitude. Drawn on logarithmic axes, the curves were close to straight lines. These regularities are called **scaling laws**, and their practical importance was enormous. A lab could train a series of small models cheaply, draw the line through them, and predict roughly how well

a model a hundred times larger would do before spending the money. That predictability, more than any single breakthrough, is what made organizations willing to spend tens of millions on one training run.

Two years later, a team at DeepMind led by Jordan Hoffmann corrected the recipe. Their paper, usually known by the name of its model, Chinchilla, asked what is really a bakery question. Suppose you have a fixed budget for an expansion. Ovens without flour sit idle; flour without ovens goes stale in the storeroom. Somewhere there is a balance. For language models the ovens are parameters and the flour is training text, and the 2022 study concluded that the large models of the day were nearly all ovens: for a given compute budget, it was better to train a smaller model on more text, growing the two roughly in step. A rule of thumb that came out of it, about twenty training tokens per parameter, is still widely quoted. Chinchilla itself, with 70 billion parameters trained on about 1.4 trillion tokens, outperformed DeepMind's own Gopher, four times its size, trained with roughly the same compute.

The bakery picture breaks in two instructive places. First, the Chinchilla balance minimizes the cost of training alone. A model that will answer billions of questions is paid for mostly on opening day, so many recent models are deliberately smaller than "optimal" and trained on far more than twenty tokens per parameter, because a small model is cheaper to serve for years. Second, a bakery's returns are understood: double the ovens and the flour and you bake about twice the bread. Scaling laws are empirical regularities with no agreed theory behind them. They predict the average loss on the next token, not whether a particular skill will appear; whether skills really switch on suddenly at some size is still debated. And nothing guarantees that the lines stay straight forever.

The flour, it turns out, is finite. In 2024 the research group Epoch AI estimated the stock of reasonably good public human-written text at a few hundred trillion tokens, with wide uncertainty, and projected that, at the pace of the time, the largest training runs could use it up sometime between 2026 and 2032. The industry has three responses. Curate better: filter, deduplicate and weight the text, because a well-chosen trillion tokens beats a careless three. Repeat carefully: a few passes over the same text still help, though less each time. And make more: **synthetic data**, text written by models for training models. It works best where the output can be checked, such as math problems with verifiable answers or code that must pass its tests, so that wrong examples can be caught and thrown away. Where it cannot be checked, there is a known risk: researchers have shown that models trained, generation after generation, mainly on the output of earlier models tend to lose the rare and unusual parts of the original data, like a photocopy of a photocopy, a narrowing called **model collapse**.

Finally, cost, which is the arithmetic of running a model, multiplied. Every training step carries the memory and the threefold compute counted for the two lives of a model, and a frontier model takes an enormous number of steps. In practice that means thousands to tens of thousands of graphics cards running for weeks to months. Companies rarely publish the bills, but independent estimates for the largest runs are in the tens of millions of dollars and climbing toward hundreds of millions, while adapting a modest model with LoRA can cost less than a dinner. Energy follows the same curve. One widely cited estimate, by Google researchers in 2021, put the

training of GPT-3 at about 1,300 megawatt-hours, roughly what a hundred and twenty American homes use in a year, and frontier runs since then have been much larger. A single answer is small by comparison: in 2025 Google reported that a median text prompt to its Gemini assistant used about a quarter of a watt-hour, comparable to about nine seconds of television, though such figures depend heavily on what is counted and are hard to check from outside. Multiplied by billions of requests, plus the water that cools the data centers, the total becomes a matter for power grids and town councils rather than for one engineering team.

So “how much is enough?” has an honest answer, though not a single number: enough is where your learning curve flattens, measured on your own data, with duplicates removed and the hard cases covered. Past that point you need a better teacher, not more practice. With that, the engine room tour is over. Before we turn to language and meaning, though, one question has been waiting patiently since the start of the previous chapter. If a network is not a brain, then what is a brain, the one that runs on twenty watts and learns as it goes, actually doing? Neuroscience has some bold answers, and the boldest of them begins with a coffee cup.

Going further. If you wondered what actually happens inside a language model as it writes one token after another, section 7.3 walks through it frame by frame. How a finished model goes back to school for a narrower job, from transfer learning to LoRA, is the subject of section 9.2. The crammer you built, its opposite the slacker, and the remaining cures for both have section 10.1 to themselves. And how sure a model really is, when it answers every question with the same fluent assurance, is the question of section 10.5.

Chapter 6 — The Other Machine

6.1 A Thousand Models of a Coffee Cup

Close your eyes and have someone put an object in your hand. Touch it with one fingertip. You feel something smooth and curved. That could be many things. Slide the finger up and you meet an edge, a rim. Slide it sideways and down, and you find a loop sticking out. Coffee cup. Three touches, perhaps four, and you are certain, even though no single touch told you much.

Now do it again, but grab the object with your whole hand. You know instantly. No sliding, no sequence of guesses.

This small experiment sits at the heart of one of the boldest recent attempts to explain how the brain works, and its author began his career not as a neuroscientist but as an engineer. Jeff Hawkins designed the PalmPilot, one of the first successful handheld computers, then spent the following decades on what he considered the more interesting problem: how the neocortex, the wrinkled outer layer of the brain where perception, language and planning live, actually computes. He laid out his early ideas in *On Intelligence* in 2004 and a fuller theory in *A Thousand Brains* in 2021. What follows is that theory as he and his colleagues tell it. It is influential, it is debated, and it is a wonderful way to think about how a brain might do what no network in this book yet does well.

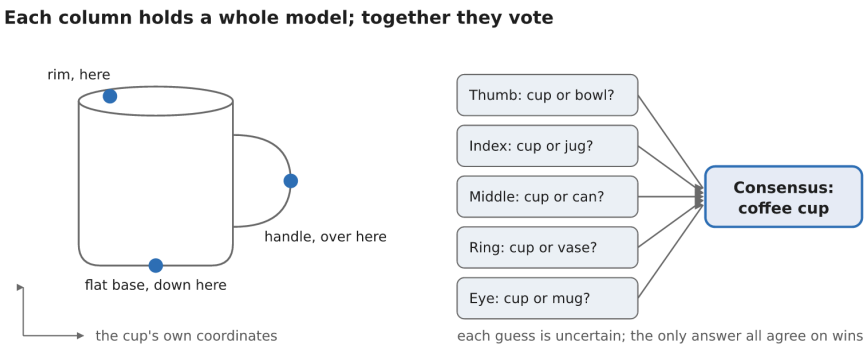
The story starts with an older and stranger observation. In 1978, the neuroscientist Vernon Mountcastle pointed out something that should have been more surprising than it was. The neocortex has areas for vision, for hearing, for touch, for language, for planning. You would expect each to be built differently, the way a graphics card differs from a network card. Under the microscope, they are not. The tissue looks almost the same everywhere: the same layers, the same cell types, the same wiring pattern, repeated in small vertical units called **cortical columns**. Mountcastle proposed that the cortex runs the same algorithm everywhere, and that what makes one area “visual” is simply that it is connected to the eyes.

The most striking evidence came from an experiment that sounds like science fiction. Around 2000, a team at MIT led by Mriganka Sur rewired newborn ferrets so that signals from their eyes were sent to the part of the brain that normally processes sound. The “auditory” cortex learned to see. It developed orientation-tuned maps much like those of a visual cortex, and the animals responded to light through it. People who are born blind show a gentler version of the same thing: their visual cortex is put to work on touch, for example when reading Braille. One algorithm, many uses. For a programmer, it is a familiar idea: one well-written generic library, instantiated with different input types.

So what does each column compute? Here is where Hawkins makes his biggest claim. In a convolutional network, each inspector sees only a fragment, and only the last one sees “cat”. In the Thousand Brains theory, every column learns *complete models of whole objects*. There are,

by Hawkins' estimate, about 150,000 columns in the human neocortex, each roughly the size of a grain of rice. Each one, receiving input from its own small patch of skin or retina, tries to recognize the whole cup on its own.

How can a column hold a whole object if it only ever feels one small patch at a time? The answer is the key idea of the theory: **reference frames**. A column does not just record "smooth, curved". It records "smooth and curved *at this location on the object*". As your finger moves, the column tracks where on the cup the finger now is, and stores what it feels there. The cup becomes a little map: rim up here, handle over there, flat base at the bottom, every feature pinned to a location in the cup's own coordinate system. In the theory, the column knows where the finger is from the movement itself: the brain knows which motor command it has just sent, so it can update the finger's position on the cup the way a sailor without GPS updates her position from speed and heading. This is what grid cells, described below, are thought to do for the whole body in a room.



If you have ever worked with a game engine or a 3D scene graph, you already know this structure. Every object has its own local coordinate system, and its parts are positioned relative to it: the cup's handle is at (+4, 0, 3) in the cup's frame, regardless of where in the room the cup stands or how it is rotated. Move the cup, and the handle moves with it, because the relationship is stored, not the absolute position. A dictionary keyed by location, holding features as values: that is a column's model of an object.

The idea of location-tracking cells is not speculation. In 1971 John O'Keefe found **place cells** in the rat hippocampus, neurons that fire when the animal is at one particular spot in a room. In 2005 May-Britt and Edvard Moser found **grid cells** nearby, neurons that fire at many spots arranged in a strikingly regular triangular grid, like a coordinate system laid over the floor. The three shared the Nobel Prize in 2014. Hawkins' theory proposes that the neocortex reuses a version of this navigation machinery inside every column, not to track where *you* are in a room, but where your finger is on a cup, or where your eyes are on a face.

Now the second experiment makes sense. With one finger, a single column has to move and collect several observations before its map settles on "cup". With the whole hand, many columns each get one observation at once. Each is still uncertain on its own: the thumb thinks cup or

bowl, the index finger cup or jug, the middle finger cup or can. But they talk to each other, and only one object is consistent with all their observations together. They **vote**, and the vote settles in an instant. You saw this principle already, in the random forest's crowd of half-blind experts; and engineers know it from distributed systems, where replicas that each hold the full state reach a decision by quorum. The Thousand Brains theory suggests the brain does recognition by something like consensus among thousands of independent models, each with the whole picture and a different, partial view.

The theory makes one further, more daring claim. If columns are the same everywhere, and if columns in sensory areas organize knowledge in reference frames, then columns in the areas for language, mathematics and planning might do the same. Thinking, in this view, is a kind of movement through maps of concepts, just as touching is movement over a map of an object. That sounds abstract, but there is suggestive evidence. In a 2016 study, researchers trained people to think about made-up birds that differed only in the lengths of their necks and legs. As participants mentally "moved" between birds, brain areas that normally carry grid-cell signals for physical space showed a grid-like pattern for this abstract space of bird shapes.

Why should an engineer care about a theory of the cortex? Because it highlights, sharply, what today's networks lack. A CNN recognizes a cat largely from a bag of features, which is why it can be fooled by elephant-skin texture on a cat-shaped outline. It has no explicit map of where the parts are relative to each other. Geoffrey Hinton's capsule networks, proposed in 2017, were one attempt to give networks exactly such relative positions. Language models, for their part, learn without moving, touching or acting in the world at all. And the brain learns continuously, a new object in a few touches, without forgetting the old ones, the property whose absence in networks we met as catastrophic forgetting. The theory also leans on another familiar feature: at any moment, only a small percentage of cortical neurons are active, giving long, sparse patterns with the same enormous room for distinct meanings that made hypervectors work.

It is important to hold all this with the right grip. The Thousand Brains theory is one coherent proposal, not settled science. Many neuroscientists are skeptical of parts of it, especially the claim that every column learns complete object models, and the evidence for grid-like reference frames throughout the neocortex is still incomplete. What makes it valuable, even if parts turn out to be wrong, is that it is precise enough to test and engineered enough to build. Hawkins' company has been doing exactly that, trying to turn the theory into software.

If columns build models by predicting what they will feel at the next location, then prediction must be doing a great deal of work in the brain. A whole school of neuroscience now argues that prediction is not one function among many, but the brain's fundamental job. And that brings us, unexpectedly, very close to the idea at the heart of every language model.

6.2 The Prediction Machine

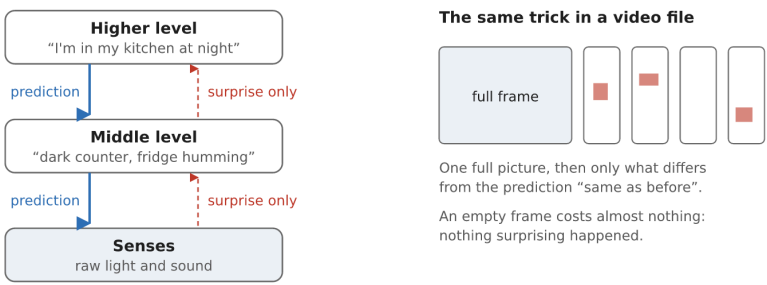
You are sitting in your kitchen late at night, reading. Suddenly the room feels different, and it takes you a second to understand why. The fridge has stopped humming.

Notice what just happened. For the past hour, the hum was entering your ears continuously, and you heard nothing. The moment it disappeared, you noticed. Your brain did not report the sound; it reported the *change*. Or try this one: walking down a dark staircase, you put your foot down for one more step, and there is no step. The jolt that goes through your whole body is out of proportion to the few centimeters you dropped. What jolted you was not the floor. It was the failure of a prediction you did not know you were making.

A large and growing school of neuroscience takes these everyday moments as the key to the whole brain. Its central claim, usually called **predictive processing** or **predictive coding**, is that the brain is not mainly a device for receiving signals from the world. It is a device for *predicting* them. Higher areas constantly send predictions down toward the senses: given what I believe is going on, here is what you should be seeing and hearing next. The senses compare those predictions with what actually arrives, and only the difference, the **prediction error**, travels back up. When the prediction is good, very little needs to be sent. When something surprising happens, the error rises through the levels until some level revises its story. The errors are also the lessons: each surprise nudges the higher areas' story a little, so the next prediction is closer, much as the loss nudges a model in training, and over a lifetime of surprises the predictions become good enough that most of the day is silence.

The idea is older than computers. In the 1860s, the physicist Hermann von Helmholtz described perception as "unconscious inference", the brain guessing at the causes of its sensations. In 1999, Rajesh Rao and Dana Ballard showed that a network built on this principle reproduced some odd properties of real neurons in the visual cortex. The neuroscientist Karl Friston generalized it into an ambitious mathematical theory, and philosophers and neuroscientists such as Andy Clark (*Surfing Uncertainty*), Anil Seth (*Being You*) and Lisa Feldman Barrett (*How Emotions Are Made*) have spent the past decade exploring what it would mean for how we see, feel and even experience ourselves.

Predictions flow down; only surprises flow up



When the fridge stops, the prediction "hum" fails, and the silence suddenly travels all the way up.

Engineers have been using this exact trick for decades, in every video you have ever streamed. A video file does not store every frame as a full picture. It stores one complete frame, then for the following frames it stores only what differs from a prediction, usually "the same as before,

perhaps shifted a little". A shot of someone talking in front of a still wall costs almost nothing after the first frame, because almost nothing is surprising. A cut to a new scene is expensive, because everything is. Your brain, according to predictive processing, handles the world the same way: send the expectation down, send only the surprise up. It is an extraordinarily efficient design for a system that runs on twenty watts. The codec picture breaks in one revealing place. A codec's predictions are fixed rules, and it always defers to the frame that actually arrives; a brain learns its predictions, and when they are strong enough it can overrule the senses, as the next paragraph shows.

This view explains some things that are otherwise puzzling. You cannot tickle yourself, and a 1998 study by Sarah-Jayne Blakemore, Daniel Wolpert and Chris Frith showed why: when you move your own hand, your brain predicts the touch precisely, so there is no prediction error to feel, and the sensation is muted. When someone else does it, the touch is unpredictable, and it tickles. Optical illusions become easier to understand too. In the hollow-mask illusion, the inside of a mask, which is concave, looks convincingly like a normal face bulging toward you. Your brain's prediction that faces bulge outward is so strong that it overrides the evidence from your eyes. Anil Seth puts it provocatively: what you experience as reality is a kind of "controlled hallucination", the brain's best guess, constantly corrected by the senses.

If this reminds you of something, it should. A language model is also a prediction machine. Its core training is a single task, predicting the next piece of text, and its learning signal is exactly a prediction error: how much probability it failed to give to what actually came next. That parallel is real, and it is one reason the field finds predictive processing so interesting. The differences are just as instructive. The brain predicts in order to act and to keep its body alive, from moment to moment, and it never stops learning. A language model predicts text, learns mostly in one enormous training run, and is then frozen. It predicts; it does not live with the consequences of its predictions.

How does a brain learn, if not by backpropagation? This is one of neuroscience's great open questions, but the oldest answer is still a good starting point. In 1949 the psychologist Donald Hebb proposed that when one neuron repeatedly helps to make another fire, the connection between them grows stronger. The neuroscientist Carla Shatz later summed it up in a phrase generations of students have remembered: cells that fire together wire together. The important thing about **Hebbian learning** is that it is *local*. Each connection adjusts using only information available right where it is, the activity of the two neurons it joins. Backpropagation, by contrast, needs the error measured at the very end of the network carried back precisely through every layer, the workshop inspector walking the whole line. Real neurons do not seem to have such an inspector. Engineers know the difference well: a system where each service tunes its own cache from its own traffic, versus one central optimizer that must see everything. Local rules are robust and cheap; global ones are precise and expensive. Finding learning rules that are both local and powerful is an active research area on both sides.

There is one more trick in the brain that machine learning has already borrowed, and it involves sleep. Your brain seems to have two memory systems with very different speeds. The **hippocampus**, a curled structure deep inside, learns fast: a single experience, such as where you

parked this morning, can be stored in one go. The neocortex learns slowly, gradually extracting general knowledge from many experiences. In 1995, James McClelland, Bruce McNaughton and Randall O'Reilly argued that this split is no accident. A system that learned everything quickly in one place would overwrite old knowledge with every new event, exactly the catastrophic forgetting we met earlier. So the brain records new experiences quickly in the hippocampus, and then, especially during sleep, **replays** them to the cortex again and again, interleaved with older memories, so the cortex can absorb them without erasing what it already knows. Recordings in rats in the 1990s showed exactly this: the sequence of place cells that fired while a rat ran a maze fired again, in order and faster, while it slept. Machine learning uses the same trick: when a model is taught something new, mixing a share of old examples into the new training data, called rehearsal, is one of the simplest and most effective defenses against catastrophic forgetting, and it is often used when language models are fine-tuned.

If you have worked with databases, you know this design. Writes go first into a fast, append-only log; later, a background process compacts them into the main, carefully organized storage. Many modern storage engines are built exactly like that. And DeepMind's reinforcement learning agent that learned Atari games did something similar: it stored its past experiences in a buffer and replayed them to itself in shuffled order while learning, an idea its authors explicitly connected to the hippocampus.

Put the two sections of this chapter side by side and a picture of the brain emerges that is very different from a stack of grids trained once with backpropagation. It is a machine that predicts before it perceives, that moves in order to learn, that holds thousands of small models and lets them vote, that records fast and consolidates slowly, and that does all of it on the power of a dim light bulb. Neither theory is final, each has serious critics, and they do not agree with each other everywhere. But machine learning has borrowed from this machine before, convolutions from the visual cortex and replay from the hippocampus, and there is every reason to think it will borrow again: learning that never stops and never overwrites, maps of objects and ideas, surprise as the currency of learning, and an efficiency that today's data centers can only envy.

One image has run through both sections: the map. Grid cells mapping a room, columns mapping a coffee cup, perhaps the cortex mapping ideas. When we return to the machines, the first thing we will see is that they, too, have learned to build maps, not of rooms, but of meanings.

Going further. If you wondered what a language model's own prediction game looks like up close, section 7.3 lets you watch a model learn it. Whether machines can also learn by acting in the world, as brains do, is taken up in section 7.6. Why fine-tuning risks catastrophic forgetting is part of section 9.2, and how teams guard against it when they update a model, rehearsal included, is part of section 11.5.

Chapter 7 — Language and Meaning

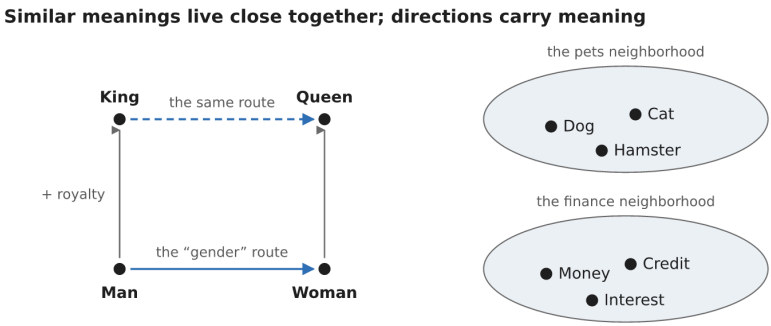
7.1 A Map Where Meaning Has an Address

Open any color picker and type `#1E90FF`. You get a bright, friendly blue. Change it to `#1E80EF` and you get a blue that is almost the same, a shade deeper. Change it to `#FFD700` and you get gold. Without thinking about it, you have been using one of the most powerful ideas in modern machine learning.

A color code is an address. Its three coordinates say how much red, green and blue to mix, and the remarkable thing is that the space they define has *meaning*. Colors with nearby addresses look alike. Directions mean something too: move any color in the "more red" direction and it warms up, whatever color you started from. Nobody has to explain to you that navy and royal blue are related. Their addresses already say so.

Now imagine doing the same thing for words. Give every word in the language an address, not with three coordinates but with hundreds, and arrange the addresses so that words with similar meanings live close together. In a well-mapped city of words, "dog" lives near "cat" and "hamster", "money" lives on the same street as "credit" and "interest", and "king" lives next door to "queen". Such an address is called an **embedding**: a list of numbers, a few hundred to a few thousand long, that places a word, a sentence, an image or a product on a map of meaning. The length is the designer's choice, a hyperparameter: more coordinates leave room for finer distinctions but cost more memory and slower search.

The moment this idea caught the public imagination came in 2013, when a team at Google led by Tomas Mikolov released a method called word2vec. People began doing arithmetic with its addresses and discovered something uncanny. Take the address of "king", subtract "man", add "woman", and look at what lives nearest the result, leaving out the words you started with. Very often, it is "queen". The route from man to woman on the map turned out to be roughly the same as the route from king to queen. Directions in the space of words, like directions in the space of colors, carried meaning.



A 2-D map for the eye; real embeddings have hundreds or thousands of coordinates.

Who decided these addresses? Nobody. They were learned, and the principle behind the learning is older than computers. In 1957 the linguist J. R. Firth wrote that you shall know a word by the company it keeps. A model reading billions of sentences notices that “king” and “queen” turn up in similar company: “the ___ ruled”, “crowned the ___”, “the ___ and the court”. Every time it sees two words keep similar company, it nudges their addresses closer. After enough reading, the map draws itself. In word2vec, the nudging is a small guessing game: from a word’s address, guess which words appear near it in a sentence. Every wrong guess sends a gradient that moves the addresses a little, and words that keep the same company get pushed in the same directions until they end up neighbors.

For an engineer, the payoff is immediate, because it turns “find similar” into “find nearby”. Type “function that retries failed HTTP calls” into a semantic code search. Plain text search finds nothing, because the function you want is called `with_backoff` and its comment talks about “exponential delay on transient errors”. But embed your query and every function in the codebase, and `with_backoff` sits right next to your query on the map. The words are different; the meaning is close. The same trick is how a bug tracker catches a duplicate whose words barely overlap with the original: it groups “Can’t log in on Safari”, “Sign-in button dead on my Mac” and “Authentication fails in Apple browser” into one cluster of duplicates, and powers the “similar items” shelves of many online shops. Nearest-neighbor search and clustering suddenly work on text. Whole sentences and documents get a single address too: a model reads the text with attention, the mechanism of the next section, condenses the context-aware addresses it has built for every word into one list of numbers, often simply by averaging them, and is trained so that a question and the passage that answers it land close together.

There is a subtlety in how “close” is measured, and a kitchen explains it best. A pancake recipe for four people and the same recipe scaled up for forty have wildly different quantities, but identical proportions. Measured by raw distance, they are far apart. Measured by *direction*, by the ratios between ingredients, they are the same recipe. Embeddings are usually compared the second way, with **cosine similarity**, which looks at the angle between two addresses and ignores their length. That way a long document and a short note about the same topic count as neighbors, as they should.

It would be easy to get carried away here, so two words of caution. First, the coordinates have no names. There is no “royalty axis” you could point to; meaningful directions are blends of many coordinates at once. Second, the king-and-queen arithmetic is real but temperamental. It works beautifully for selected examples and unevenly for many others. Treat it as a tendency of the map, not a law of nature. And because the map is learned from what people write, it learns their prejudices too: early word maps placed “nurse” closer to “she” and “engineer” closer to “he”, faithfully reproducing the company those words keep in ordinary text; and a faithful map is not the same as a fair one.

And there is a deeper problem, one you may have spotted already. Read these two sentences: “After fishing from the bank, I walked home.” “I deposited the money at the bank.” In a map like word2vec’s, “bank” has exactly one address. It cannot live by the river and next to the money at the same time, so it ends up somewhere awkwardly in between, a little useless for both.

What we really want is an address that moves depending on the words around it. That is exactly what researchers built from 2017 on, and to see how, we need to step into a room full of detectives.

7.2 The Detectives' Meeting

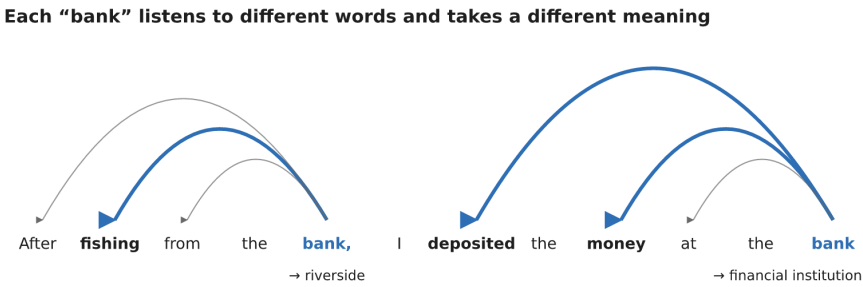
A message appears in your team's chat channel: "it's down again."

You know immediately what "it" is. Not because the message says so, but because your eyes have already flicked up the channel, skipped the lunch plans and the link to a conference talk, and landed on the message from twenty minutes ago about the payments service throwing timeouts. Your understanding of "it" was assembled from the other messages, each weighted by how relevant it seemed. The lunch plans got almost no weight. The payments message got nearly all of it.

That glance, repeated for every word in a text, is the idea at the heart of the Transformer. It is called **attention**, and to see how a machine does it, picture a room.

Turn the sentence "After fishing from the bank, I deposited the money at the bank" into a room with one detective per word. Each detective has a problem to solve, namely what they themselves mean here, and each carries three things. A **note with a question**, called the *query*: "I need someone who can tell me what kind of bank I am." A **name badge**, called the *key*, announcing what they can offer: "I am an outdoor activity near water", or "I am a financial action". And a **file**, called the *value*, holding the information they will hand over to anyone who listens to them.

The meeting begins. Each detective reads their question, walks around the room and compares it with everyone's badge. The better the match, the more attention they pay to that detective. Then each detective rewrites their own identity as a blend of the files of the detectives they listened to, mostly from the best matches, a little from the rest. The first "bank" finds a strong match in "fishing" and leaves the meeting as *bank, the riverside*. The second finds strong matches in "deposited" and "money" and leaves as *bank, the institution*. Same word, two identities, decided by context, and decided afresh for every sentence.



Thick arc = strong attention, thin arc = weak. Each word only looks back (causal mask, as in GPT-style models).
A sketch: in a real model the weights are computed, more spread out, and split across several heads.

If you are a programmer, here is the moment where it may click into place. You use dictionaries every day. You look up a key; if it matches exactly, you get its value; if not, you get nothing. Attention is the same operation, made soft. Your query is compared with every key in the table, and instead of one exact match, each key receives a match score. The scores are turned into percentages that add up to a hundred, and you get back a blend of all the values, weighted by those percentages: mostly the best match, a bit of the second best, a trace of everything else. Attention is a fuzzy dictionary lookup, performed by every word on every other word, all at the same time. The comparison itself is plain arithmetic: question and badge are both lists of numbers, and the match score is their dot product, the multiply-and-add of the apartment checklist, large when the two lists point in similar directions; softmax then turns the scores into percentages.

That last part, all at the same time, is what the telephone game could never do. Nobody whispers. Every word can consult every other word directly, however far apart they stand, and because the consultations are independent of each other, a graphics card can run them all in parallel.

Here is the subtle point that is easy to get wrong. What does the model actually learn? Not the match scores between particular words; those are recomputed for every new sentence. Each word starts from its embedding, and three learned transformations turn that embedding into a question, a badge and a file. **Training teaches the detectives how to write good questions, good badges and useful files in general.** That is why a Transformer copes with sentences it has never seen: the rules for writing notes carry over, even when the room is new.

One meeting is not enough, so Transformers hold many in parallel. Think about code review: three reviewers who each focus on something different, security, performance and readability, catch more than one reviewer trying to watch everything. In **multi-head attention**, several meetings run side by side on the same sentence, each with its own style of questions and badges. One head might learn to track grammar, which noun a verb belongs to; another to track references, what “it” points to; another the topic. A full **Transformer block** is such a multi-headed meeting followed by a small neural network that each detective uses privately to digest what they heard. Large models stack dozens of these blocks, so identities are refined again and again.

Two details complete the picture. First, the meeting as described has no sense of order. “Dog bites man” and “man bites dog” bring the same three detectives into the room. So each word’s starting embedding is combined with information about its position in the sentence. The original Transformer simply added a fixed pattern of numbers to each embedding, a different pattern for every position, like a house number written on each file; modern models mostly use a cleverer scheme, which comes later in the book. Second, models that generate text, such as the GPT (generative pre-trained transformer) family, add a rule: each detective may only consult the detectives to their left. When you are writing a sentence, you cannot consult words you have not written yet. This is the **causal mask**, and our example respects it, since “fishing” comes before the first “bank”, and “deposited” and “money” come before the second. When the clue comes later, as in “the bank was steep and muddy”, the detective for “bank” cannot resolve it, but

“steep” and “muddy” can look back at “bank” and carry the resolved meaning forward in their own rows, which is where the next word is predicted from; models built only for reading, such as BERT, drop the mask and let every word look both ways.

Step back and compare this with an ordinary multi-layer network. That network applies the same fixed weights to every input. It is the clerk at the counter, applying one regulation to every file that crosses the desk. A Transformer computes fresh attention for each input. It is the manager who reads the whole file and decides, question by question, which documents matter. The manager has a price, though. Because every word consults every other word, doubling the length of a text roughly quadruples the work of attention. That is why every model has a maximum context length, and why making that length longer has been one of the field’s great engineering battles.

A word of caution before we leave the room. The detectives understand nothing and intend nothing. “Listening” is a weighted average of lists of numbers, and real attention is rarely as tidy as in the drawing: it is usually more spread out, and different heads look at very different things. Attention patterns are a clue to what a model is doing, never a full explanation.

The 2017 paper introduced the Transformer as a better way to translate between languages. Few people guessed what would happen when someone took this architecture, made it enormous, and trained it on the simplest game imaginable: guess the next word.

7.3 The Game of Guessing the Next Word

Try this on your phone. Open a message, type “I think the”, and then keep tapping the middle suggestion the keyboard offers. Within a few taps you will get something like “I think the best way to get the best of the best”, looping forever. Your keyboard is a tiny language model. It has learned which word tends to follow which, and with no larger sense of where the sentence is going, it wanders in circles.

Now hold that silly sentence in mind, because here is the most surprising fact in this book. A large language model (LLM), the kind that writes essays, explains legal clauses and completes your code, plays exactly the same game. In its first and largest phase of training, it learns to guess the next piece of text, nothing more. The differences are scale and architecture: it is many thousands of times larger than your keyboard’s model, it is a Transformer whose detectives can consult the entire text so far, and it has read a substantial part of the public internet, a great many books and an enormous amount of code. Somewhere along that scale, the looping nonsense turns into something that looks very much like understanding. Exactly why it works as well as it does is still an active research question.

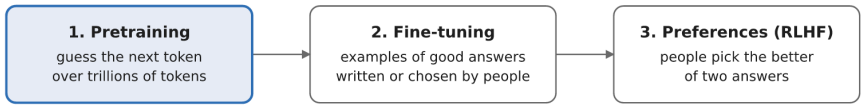
First, a detail about what is being guessed. Models do not see words; they see **tokens**, pieces of words. A common word is usually one token, while a rarer one is cut into pieces, something like “un” + “believ” + “able”. For English text, a token averages roughly three to four characters. Languages that appear less often in the training data tend to be cut into more, smaller pieces, so the same message costs more tokens in Romanian or Greek than in English, which matters

because APIs bill and limit by the token. The cutting is decided before the model is ever trained, by a simple statistical program: it starts from single characters and repeatedly glues together the pair that appears most often in a large sample of text, “t” and “h” into “th”, then “th” and “e” into “the”, until the vocabulary reaches its chosen size, so whatever was common in that sample gets cheap, whole-word tokens and everything else is cut into pieces.

Now watch the detectives from the last section learn the game. The boss brings in a sentence with its last word covered: “I went to the bank to withdraw some ___”. On the first day, the detectives are brand new and their notes and badges are random. “Bank” listens mostly to “went”, and the team’s best guess is “pigeons”. The boss uncovers the word: “cash”. The loss measures how little probability the team gave to the right answer, and backpropagation hands every detective its share of the blame. “Bank” adjusts, very slightly, how it writes its questions, so that next time it goes looking for badges like “financial action”. “Withdraw” adjusts its badge so that it is easier to find. Then the next sentence arrives, and the next, billions of them, on every subject people write about. Learning to predict the next word in a recipe requires knowing something about cooking; in a physics textbook, something about physics; in a Python file, something about Python. The game is simple. Winning it requires absorbing a great deal of the world.

The result of this phase, called **pretraining**, is astonishingly knowledgeable and strangely unhelpful. Picture a new engineer who, in her first month, reads the entire company wiki, every repository and every ticket ever filed. She knows an enormous amount. But ask her a question and she might answer with three more questions, because on the wiki, questions are often followed by more questions. She has learned to continue documents, not to help people.

An LLM goes through three different schools



Most of the computing goes into step 1; steps 2 and 3 shape the model into an assistant.

So the new hire goes through two more schools. In the second, fine-tuning, she shadows senior colleagues: thousands of examples of a request followed by a good, helpful answer, until the shape “someone asks, I help” becomes second nature. In the third, she gets something like code review. People are shown two of her answers to the same request and pick the better one, and she gradually learns what reviewers prefer: clearer, more accurate, more careful answers. This last stage is usually called **RLHF**, reinforcement learning from human feedback. Unlike a real new hire, the model remembers nothing of these schools, and the last one has a catch: reviewers reward answers that look right to them, which is not quite the same thing as answers that are right.

Now we have every piece we need to watch a language model work, in slow motion, from the moment you send your message. Everything we have built so far comes together here, so take it one frame at a time.

You type “The opposite of hot is”. The first thing that happens has nothing to do with intelligence: a **tokenizer**, a plain lookup program, cuts the text into pieces and replaces each piece with its number in a fixed vocabulary of, typically, tens of thousands to a couple of hundred thousand entries. Your sentence is now a short list of five integers.

The second frame is the embedding layer, the doorman. Each integer is used as a row number into a large table, and the row comes back: a vector of a few thousand numbers. Five tokens, five rows. Stack them and you have a tensor of shape (5, width), a spreadsheet with one row per token. Information about each token’s position is mixed in, so the model knows which came first.



The third frame is the long one. That (5, width) table passes through the Transformer blocks, one after another, dozens of them in a large model. In each block, the detectives’ meeting runs: every row consults the rows before it and rewrites itself as a blend of what it heard. Then each row goes privately through a small dense network, the part of the block where, researchers believe, much of the model’s factual knowledge is stored. Residual links carry the original signal around every step, normalization keeps the numbers in range, and through all of it the shape never changes: five rows in, five rows out, each row a little richer than before. By the last block, the row for “is” no longer means just “is”. It has absorbed “opposite”, “of” and “hot”, and it now encodes something like “a temperature word, the reverse of hot, comes next”.

The fourth frame looks only at that last row. It is multiplied by one more large matrix, with one column per entry in the vocabulary, producing a score for every possible next token, about a hundred thousand scores. Softmax turns them into probabilities. “cold” gets nearly all of it; “cool” and “freezing” get crumbs.

The fifth frame picks one token according to those probabilities, and the sixth frame is the loop: the chosen token is appended to the input, and the whole forward pass runs again to produce the token after it. One forward pass per token: this is what drives the bill. To avoid redoing the meeting from scratch every time, the model keeps each earlier token’s badges and files in memory, a store called the **KV (key–value) cache**, and only the newest token’s detective has to walk around the room. That cache is a large part of why long conversations need so much

graphics memory. The loop stops when the model picks a special “end” token, which it has learned to predict, from millions of examples that ended with one, when a text is complete; the application also sets a hard limit, in case it never does.

That is the whole machine. No step in it is mysterious on its own: a lookup, a stack of matrix multiplications with a few bends, a softmax, a dice roll, repeat. What is mysterious, and what researchers are still working to understand, is how so much apparent knowledge and reasoning fits into the numbers those multiplications use.

One dial sits in the fifth frame, the pick: how adventurous it is depends on a setting called **temperature**. Near zero, the model almost always takes the most likely token: predictable, repeatable, occasionally dull. That is what you want when extracting fields from invoices into JSON (a standard format for structured data). Turn the temperature up and less likely tokens get their chance: the output becomes more varied and inventive, and also more error-prone. That is what you might want when brainstorming product names. It is the difference between a musician reading sheet music and one improvising. Underneath, the dial is simple: the scores are divided by the temperature before softmax turns them into probabilities, so a small temperature stretches the gaps and the favorite takes nearly everything, while a large one squeezes them and outsiders get a real chance.

One more thing to understand about how these models live. Everything the model knows about your conversation sits in its **context window**, and the best picture for it is a whiteboard in a meeting room. The model reads the whole board before every word it writes. But it does not learn from the meeting, and when the meeting ends, the board is wiped. Tomorrow it will not remember today’s conversation unless someone writes it on the board again. Long documents, long chats and large codebases all compete for space on that board.

And now the uncomfortable part. You are working with an assistant, and it writes a tidy line of code calling `client.get_json_safe()`. It looks exactly like a method the library would have. You run it. The method does not exist, and never did.

This is called a **hallucination**, and the name makes it sound like a malfunction. It isn’t. Remember what the model was trained to do: produce the most plausible continuation of the text. Plausible and true usually coincide, which is why these models are so useful. When they come apart, when the model does not know something, it often does not fall silent. It produces the most plausible-sounding thing, fluently and confidently, because that is, above all, the game it was trained to play.

So what do you do with a brilliant colleague who has read everything, writes beautifully and rarely says “I don’t know”? The same thing you would do with a brilliant new hire. You stop asking her to answer from memory, and you hand her the documents.

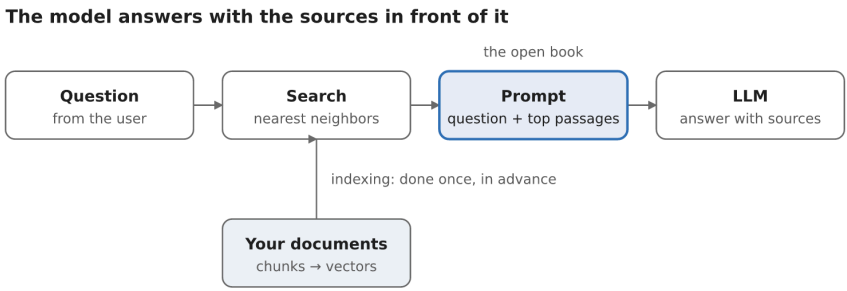
7.4 The Open-Book Exam

Picture a company that rolls out an internal assistant, a language model wired into its chat tool. At eleven at night, an engineer on call asks it: “What’s our escalation policy if the payments API goes down outside business hours?”

The answer comes back instantly, well structured and completely confident. Page the secondary on-call after fifteen minutes. Notify the head of engineering after thirty. Open an incident channel named with the date. It reads exactly like a policy. It is also invented. The model has never seen this company’s runbooks; it has seen thousands of other companies’ policies and produced the most plausible blend of them. The real policy says to call the payments vendor first.

Nothing about the model is broken. It is the brilliant new hire from the last section, asked to answer from memory about a document she has never read. The fix is the one any good manager would choose: let her answer with the book open, and give her a fast librarian who fetches the right pages.

That arrangement is called **retrieval-augmented generation**, or RAG. Before the model answers, a search step finds the passages most relevant to the question and pastes them into the prompt, together with an instruction to answer from those passages and say where each claim came from. The model still does the writing. The search does the remembering. Pasting every document into the prompt and skipping the search works for a handful of documents; for a whole wiki it would not fit on the whiteboard, it would cost money for every token on every question, and models read a crowded board less carefully than a short one.



Run the night-time question again with RAG in place. The search finds the on-call policy page and the payments runbook, and places three relevant passages on the model’s whiteboard. Now the answer says to call the vendor first, quotes the vendor’s support number from the runbook, and links to both pages. The engineer can click and check. The difference is not intelligence; it is access. The model prefers the passages over its memory only because it is told to, and because assistants are fine-tuned on many examples of answering from supplied text; it usually obeys, but when a passage contradicts something the model “knows” strongly, it can slip back to its memory, which is one more reason to check the citations.

For search to find passages, the documents have to be prepared in advance, and this is where most of the craft lies. Each document is cut into **chunks** of a few hundred tokens, like cutting a manual into flash cards. Each card gets an embedding, and the cards go into a database that can quickly find the nearest neighbors of any question. The size of the cards turns out to matter a great deal. Cut too small, and a recipe ends up with its ingredients on one card and its steps on another, so no single card answers “how do I make this?”. Cut too large, and every card is about five different things, so none is a sharp match for anything.

Then there are questions that embeddings handle badly, and engineers run into them quickly. Ask “what does error E1047 mean?” and a purely meaning-based search may return a passage about E1074, because to an embedding, two codes that look alike sit very close together. Exact identifiers, function names and ticket numbers are what old-fashioned keyword search is good at. So practical systems use **hybrid search**, combining keyword and vector results, and often add a second, slower model that **re-ranks** the candidates: the librarian fetches fifty books, and an expert reads them and keeps the best five.

The benefits are concrete, which is why so many companies are building these systems. Knowledge can be updated by updating documents, with no retraining. Answers can cite sources, so people can verify them. Private data stays in your own store, fetched only when a question needs it. Once fetched, though, that data is read by the model like any other text, which raises questions of trust and privacy.

There is also a side effect nobody plans for. A RAG system is merciless about documentation. If the runbook is two years out of date, the assistant will now quote it accurately and confidently. Teams that launch one often find that the assistant’s first real achievement is revealing which pages of the wiki nobody has maintained.

And the librarian is not perfect. If she brings the wrong page, the writer quotes it with total conviction. She may also bring only part of what is needed: when an answer needs pieces from several documents, a single search often fetches only one, because no single passage is close to the whole question, so systems that handle this let the model search several times, one sub-question after another, a loop with a search tool in its hand. The quality of a RAG system is capped by the quality of its search, so it pays to measure the two halves separately: did it fetch the right passage, and did it use that passage correctly? The most effective teams collect a few dozen real questions from real users, together with the passages that ought to answer them, and run that set every time they change the chunking, the embeddings or the prompt, exactly like a regression test suite.

Two questions remain before we leave language behind: how much of a long conversation a model can actually see, and why a game as simple as guessing the next word goes as far as it does. The first begins with an old acquaintance from the hiring panel.

7.5 Living on a Whiteboard

Remember the hiring panel, the four recruiters sorting two hundred CVs, each deciding in her own way what matters. The fourth, who lets each CV direct her eyes, had to wait for language, because she is attention: her way of reading is the detectives' meeting. Here the question "who decides?" has its most interesting answer. Nobody decided in advance that "bank" should listen to "money"; that is worked out afresh in every sentence. What training shapes, as the guessing game showed, is the transformations that write questions and badges in general. Over billions of sentences, those transformations come to encode, in effect, "what kind of word tends to help a word like me". Relevance is not handed to the model. It is a skill the model acquires because relevance pays off in the next-word game. Different heads acquire different notions of relevance, because each starts from different random numbers and nothing forces them to agree; in practice some heads turn out nearly redundant and can be removed with little loss, while others carry a great deal.

This is also why attention weights explain less than they seem to. A head can attend strongly to a word and pass on very little from it, while the decisive work happens in another layer. Researchers have argued the point in print, in two 2019 papers memorably titled *Attention is not Explanation* and *Attention is not not Explanation*, and the honest summary remains the earlier one: a clue, not a verdict.

The fourth recruiter has one more limit, and it is the most physical of all: she can only direct her eyes to what is on the desk in front of her. For a language model, that desk is the context window, and its size and shape decide a great deal about what the model can do.

An engineer spends an afternoon with an assistant planning a database migration. In her first message she sets the ground rules: the orders table must never be locked for more than a second, and nothing heavy may run during the European business day. Three hours, forty messages and several pasted schema dumps later, the assistant cheerfully proposes a step that locks the orders table for twenty minutes at eleven in the morning. It has not become careless in any human sense. Either her first message is no longer on its board at all, or it is still there, buried under a great deal of later writing, and the model gave it too little weight.

This section looks at that board more closely: how big it is, what sets its size, what happens at its edge, and how people build systems that behave as if the board were bigger than it is.

The board is measured in tokens, and it holds everything, not only your messages: the instructions the application adds, any retrieved documents, the results of tools, and every word of the model's own replies. Language models of 2020 had boards of about two thousand tokens, a few pages. By the mid-2020s, boards of a hundred thousand to a few hundred thousand tokens were common, and some models accepted a million or more. At three to four characters per token, an English page of about five hundred words comes to very roughly seven hundred tokens, so a board of a hundred thousand tokens holds something like a hundred and forty pages, a short novel.

Two things set the size. The first is cost. The work of attention, as the detectives' meeting warned, quadruples when the text doubles, and the KV cache, the stored badges and files of earlier tokens, grows with every token on the board, for every conversation being served at once. A longer board is slower and hungrier for memory, which is why long inputs cost more and why engineers have fought so hard to make them cheaper. One common trick is to cache the reading of a fixed beginning: when every request opens with the same long instructions and documents, their badges and files are computed once and reused, and providers often bill those cached tokens at a discount. The second thing is training. A model learns to use the positions it has practiced on, through the position information mixed into each token. Hand a model trained only on short texts a very long one and it meets positions it has never seen, and its output usually falls apart. Long boards are typically taught in a later stage of training, on long documents, with adjustments to the way positions are written. The size of the board is a property of training, not a number someone can simply raise in a configuration file.

What happens at the edge? The model itself never decides. The application around it must act before the text reaches the model, and it has three choices. It can refuse, telling the user the conversation is too long. It can cut, usually dropping the oldest part, which is exactly where the instructions tend to live, as in the migration story. Or it can condense, like a clerk who, when the board fills, rewrites the oldest notes as a short summary in one corner and erases the rest. A summary loses detail by design: the one-second rule may survive as "be careful with locks". And here is the point that surprises most people. The model never sees what was cut and has no way to know anything is missing. It does not feel a gap. It answers from what is on the board as if that were everything there ever was. Well-built applications therefore pin what must not be lost, rewriting the critical instructions at the top of the board on every turn.

Inside the limit, the board is not read evenly. In a 2023 study titled *Lost in the Middle*, researchers placed the answer to a question at different positions among some twenty retrieved documents. Models used it best when it sat near the beginning or the end, and noticeably worse when it sat in the middle. Newer models do much better on the simplest version of the test, finding one planted sentence in a long text, often called a needle in a haystack. But real work is harder than finding a needle: it means combining several facts scattered across the board, and on such tasks quality still tends to decline as the board fills. More text is also more distraction, because passages that look relevant but are not still pull at the model's attention. The practical habits follow directly: put the important things at the start or the end, keep the board tidy, and prefer fetching the relevant pages over pasting everything.

That also settles the question of long context versus RAG, which the open-book exam answered only briefly. For a handful of documents that fit comfortably, paste them: it is the simplest arrangement, the model sees everything, and there is no search step to get wrong. For a whole wiki, retrieval still wins, on cost and space and because of the uneven reading just described. Many real systems use both, retrieval to choose what matters and a generous board to hold it.

Compare the board with the index card of the recurrent networks. The card had no edge, but it faded: old details were overwritten little by little. The board does not fade, and every word on it is as legible as when it was written, but it has a hard edge, and beyond it there is nothing at all.

How, then, does a chat assistant “remember” you from one conversation to the next? The board is still wiped. What persists is a notebook carried between meetings. The application saves notes, facts you mentioned, preferences, summaries of earlier chats, in ordinary storage, and at the start of a new conversation writes the relevant ones back onto the fresh board, often choosing them with the same kind of search that retrieval uses. The model’s weights do not change. Memory, in these products, is text written back onto the board, which also means it can be shown to you, deleted, or be out of date, like any note.

Agents live on the board too, and they fill it quickly, because every tool result lands there: a file that was read, a test log, a page of search results. Builders manage with the same tools an organized person uses. They have the agent summarize older steps, the clerk again. They give it a notes file that it writes to and rereads, a to-do list kept on disk instead of on the board. And they split large jobs into sub-tasks handed to fresh copies of the model, each with a clean board, which report back only a short result, the way a lead engineer does not read every log but asks colleagues for a summary.

Here the whiteboard picture breaks, in two places. A person reads a board with eyes that skip: you glance past the lunch menu without reading it. The model processes every token on the board at every step of its writing; nothing is skipped, everything is weighed, even if the weight is close to zero. That is where both the cost and the dilution come from: every irrelevant line costs computation and competes, a little, for attention. And a real whiteboard has a fixed area however you write, while the model’s board is counted in tokens, so the same content can take far more room in one form than in another: a pasted JSON dump or a long stack trace can eat thousands of tokens for a few lines of meaning.

So the board is the model’s whole world while it works: no senses, no yesterday, only text and the weights frozen at the end of training. That sharpens a question the book has so far left open. How does a system that only ever learned to continue text manage, with nothing but a board of words, to reason, translate and write working code?

7.6 Why Guessing the Next Word Goes So Far

Go back to the new engineer, the one who read the entire wiki in her first month, and give her a game. She is shown the opening of a postmortem she has never seen and must guess the next sentence. “At 14:02 the checkout service began returning errors. The on-call engineer rolled back the morning’s deploy, but the errors continued. The root cause was…” A guess based on which words usually follow “root cause was” will do poorly. To guess well, she has to reason that a failure which survives a rollback was probably not in the code, so perhaps a configuration change, an expired certificate, a dependency, the database. She needs a picture of how the system works. Reward her only for guessing well, make her play the game billions of times on every subject people write about, and she will be pushed to learn whatever makes text predictable: grammar first, because it is everywhere, then facts, then the patterns of arithmetic, the habits of code, the way arguments go.

That is the most common answer to that open question, and it is worth stating carefully. The next-word game is simple to set but not simple to win. Its answer key contains everything people write down, so a very good player has to capture a great deal of whatever produced the text. Nothing in the training says “learn to reason” or “learn to translate”; those skills are learned, to the degree they are, because they lower the loss. It is the same idea as predictive processing in the brain, prediction as a route to a model of what produces the signal, with one difference: the brain predicts its own senses, while the language model predicts text, a second-hand record of the world.

A second way to see it is through compression. The weights of a large model take up many times less space than the text it was trained on, so it cannot store that text word for word, although it does memorize some passages that appear many times. Information theory has long known that a good predictor and a good compressor are two faces of the same thing, and to compress well you keep the regularities and drop the accidents. Regularities are what generalization is made of. The crammer had room to memorize every answer; a language model does not, so it is pushed toward the patterns instead.

One consequence surprised even the people who first saw it at scale. Write a few examples on the board, “cheval: horse, chien: dog, chat:”, and the model continues the pattern, though not a single weight changes. This is **in-context learning**, made famous by the 2020 paper that introduced GPT-3, titled *Language Models are Few-Shot Learners*. Part of the explanation is that training text is full of lists, tables, exercises and repeated formats, where noticing a pattern earlier on the page is the best way to predict what comes later, so a model good at the game learns to pick up patterns from its own board. Researchers have found specific circuits in Transformers, called induction heads, that copy patterns of this kind and seem to account for part of the effect, though not all of it. It is learning in a loose sense only: it lasts exactly as long as the examples stay on the board.

Why does size matter so much, when a small model playing the same game produces the keyboard’s loops? The overall loss falls smoothly and predictably as models, data and compute grow together, along the scaling curves we met when asking how much data is enough. Specific abilities, though, can look as if they appear suddenly. On three-digit addition, small models score near zero, and above some size accuracy climbs fast. This was called **emergence**, and it is contested. A 2023 paper argued that many sudden jumps are partly a product of how they are scored: if a sum counts only when every digit is right, steady improvement on each digit shows up as an abrupt jump in the all-or-nothing score, and with a gentler measure the curve often looks smooth. Other tasks seem to need several skills at once, and a task that needs all of them can look abrupt even when each skill improves gradually. A fair summary: underneath, improvement is mostly gradual, but what a model is useful for can change abruptly as it crosses thresholds that matter to us.

The same mechanics explain where these models fail, often on things that look trivial. Ask how many r’s are in “strawberry” and a strong model may get it wrong, because it never sees letters, only tokens; spelling is a learned fact about each token, like a phone number known by heart, not something it looks at. Ask it to multiply two eight-digit numbers in one go and it struggles,

because every token receives the same fixed amount of computation, one forward pass through the same number of layers, whether the question is two plus two or a long multiplication. Writing out the steps buys it more passes: each intermediate result written on the board becomes input for the next token. That is why asking for step-by-step working, a technique researchers named chain of thought in 2022, helps, and why reasoning models are trained to write long working before they answer. Spending more computation at answer time, by writing longer chains or by sampling several attempts and checking them, is called **test-time compute**, and it has become a second dial next to model size: the same chess player plays better at classical time controls than at blitz. Every token of thinking is a forward pass someone pays for, and whether the written reasoning is the real cause of the answer is a separate question.

The game has also outgrown words. Cut an image into small patches, turn each patch into a vector the way a token becomes an embedding, and place the patches on the board next to the words; sound can be cut up in a similar way. A model trained on such mixed boards learns to predict across them, and some models also produce image or sound tokens of their own. This is how **multimodal** assistants read screenshots, charts and photographs. Nothing fundamental changed. The board simply holds more than text.

Does the model, then, understand? There are two serious camps. One, voiced in a much-cited 2021 paper by Emily Bender, Timnit Gebru and colleagues that called these systems stochastic parrots, holds that a model trained on the form of language alone stitches patterns together without access to meaning, and that its fluency misleads us. The other holds that predicting well forces a model to build internal structure that works like a model of what the text is about. A small but striking piece of evidence was published in 2023: researchers trained a Transformer only on sequences of legal moves in the board game Othello, never showing it a board, and found that the current state of the board could be read out of its internal numbers, and that altering those numbers changed its next move accordingly. That is a toy, far from human language, and both camps agree these models are not people; they disagree about whether understanding is the right word for what they have. This book’s stance is the practical one: judge a model by its behavior on honest tests, with problems it cannot have seen, beware of test questions that leaked into its training text, and keep the open question open.

The chapter on the brain noted that language models learn without moving, touching or acting. Research on **world models** tries to close that gap: models that predict what happens next if a given action is taken, a flight simulator learned from data rather than programmed. Video models trained to predict the next frames are being tested as simulators for robots and game agents, and robots are learning from the results of their own movements. It is early work, and whether such models capture physics or merely plausible-looking pixels is itself debated.

Here the picture of the new engineer breaks. She has a body, goals, colleagues to ask and a memory that carries over from one day to the next; she reads the wiki in order to do a job, and she lives with the consequences when she is wrong. The model reads in order to predict, keeps nothing from one conversation to the next, and weighs every page by how often its patterns recur rather than by whether it deserves trust.

One word kept returning through this section: scale. Bigger models, more text, more computation, more thinking at answer time. That closes our tour of language and meaning. Before we turn to how models are taught and how they fail, two things remain: the machines that make pictures rather than read them, and the handful of blueprints that engineers combine all of these pieces into.

Going further. If you wondered how long whiteboards are made affordable, and how state-space models bring back the fading card of the recurrent networks, section 8.4 looks at both more closely. Whether a model can tell when it does not know is the subject of section 10.5, and whether the reasoning a model writes out is the real cause of its answer is taken up in section 10.6. And why a document handed to the model can turn into an attack is explained in section 11.3.

Chapter 8 — Making Pictures, Building Systems

8.1 Learning to Draw

A child learns to draw a cat in a particular way. Not by studying the rules of cat anatomy, and not by tracing. She draws a wobbly blob, looks at it, decides something is wrong, fixes the ears, looks again, fixes the tail. Each pass is a small correction toward “more like a cat”. After enough passes, and enough cats seen, the blob becomes a cat she has never seen before, because it is hers.

Everything in this book until now has been about recognizing, predicting and deciding. This section is about the other direction: making things. Pictures, mostly, because they are where the ideas are easiest to see, but the same ideas produce music, molecules, voices and video. And the strange, satisfying thing is that the most successful method of all turned out to work very much like the child: not drawing in one stroke, but correcting a mess, a little at a time.

The oldest idea is to force a network to summarize and then rebuild. Picture a courier who must describe a photo over the phone in exactly thirty words, to a colleague who then has to redraw it. “A tabby cat, sitting, facing left, on a red cushion, window behind, afternoon light.” Thirty words cannot hold a photo. To make the redrawing come out well, the describer has to learn what matters about cats and cushions and light, and the drawer has to learn how those things look. Train the two together on a million photos, scoring them on how close the redrawn picture is to the original, and the thirty-word description becomes a remarkably rich summary. This is an **autoencoder**: an encoder that squeezes the input down to a short code, and a decoder that expands the code back. The code sits in a space where, as on the map of word meanings, nearby codes mean similar pictures. The catch is that most points in that space do not correspond to any real photo, so you cannot simply pick a random code and expect a good cat. A refinement from 2013, the **variational autoencoder**, teaches the encoder to spread its codes smoothly, so that random codes decode into plausible, if blurry, images. The blurriness was the problem that the next idea attacked.

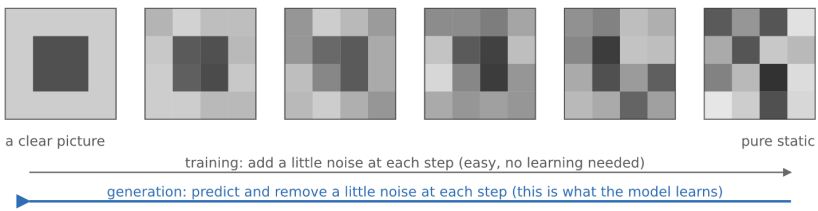
In 2014 Ian Goodfellow and colleagues proposed a game. Two networks. One, the **generator**, makes images from random noise. The other, the **discriminator**, is shown a mix of real photos and the generator’s fakes and must say which is which. The discriminator is trained to get better at spotting fakes; the generator is trained to fool it. Neither is ever told what a cat looks like. The generator learns only from the discriminator’s verdicts, and the discriminator learns only from being right or wrong. It is a forger and an art expert locked in a room: as the expert sharpens, the forger must improve, and vice versa, until the forgeries are good enough that the expert is guessing. These are **generative adversarial networks**, GANs, and for several years they produced the sharpest synthetic faces anyone had seen, the kind that filled websites of people who do not exist. Their weakness was the training itself: a game between two players is unstable

in ways a single loss is not, the two can chase each other in circles, and the generator sometimes learns one very convincing face and produces it again and again, a failure called mode collapse. Engineers who have debugged a feedback loop between two services will recognize the feeling.

Then, around 2020, the field settled on a third idea, and it is the one behind most of the image generators you have probably used.

Take a photograph and add a little static to it, the way an old television does when the aerial is loose. Add a little more. And more, for a few hundred steps, until nothing is left but static. That direction is easy; a script can do it, and no learning is involved. Now ask a network to do the opposite, one step at a time: here is a slightly noisy picture, tell me what the noise is, so I can subtract it. That is a prediction problem with an answer key, because you added the noise yourself and know exactly what it was. Train the network on millions of photos at every level of noise. It becomes very good at the single skill of looking at a noisy image and saying what should be removed. That skill is learnable even though the noise is random, because the picture beneath it is not: to tell which speckles are static, the network has to know what real photos look like, that fur has texture and skies are smooth, so learning to denoise is, secretly, learning what the world looks like.

A picture dissolves into noise; the model learns to run the film backwards



Now start from pure static and apply that skill a few hundred times. Each pass removes a little noise and reveals a little structure, the way a Polaroid develops, or the way the child's blob becomes a cat through small corrections. Unlike the child, though, the network never decides to draw a cat and never steps back to look at the whole: at every pass it answers only the question "what noise is here?", which is one reason a generated hand can have six fingers without anything in the process objecting. The result is a photo that was never taken: the network's idea of what the static might have been hiding. Each run starts from different static, and different static hides different photos, which is why the same prompt gives a different picture every time; the starting static is drawn from a number called the seed, and reusing the seed with the same prompt and settings usually gives the same picture again. This is a **diffusion model**, and the name comes from the physics of ink spreading in water, run backwards. It trains stably, because every step is an ordinary prediction with a known answer, and it is far less prone to collapse, because it learns to denoise toward *every* kind of picture it was shown.

The text prompt arrives through one more piece. At every denoising step, the network is also shown a description, encoded by a language model into the same kind of address as the word embeddings, and it is trained on captioned photos to remove noise *in a way that matches the caption*. “A tabby cat on a red cushion” is a hand on the child’s shoulder at each correction, nudging the ears toward tabby and the background toward red. How hard the hand pushes is a dial: many generators run each denoising step twice, once with the caption and once without, and push the result further in the direction of the difference; turning that dial, often called guidance, up makes the picture follow the words more literally, at the cost of variety and sometimes of realism. Image generators such as Stable Diffusion, the later versions of DALL-E and Midjourney are built on this idea. One engineering refinement, popularized by Stable Diffusion, explains why some of them can run on ordinary hardware: instead of denoising the full-size image, they denoise the short code from an autoencoder, the thirty-word description, and decode it to pixels only at the end. Working in that compressed space is many times cheaper.

For someone who builds or buys these systems, three practical facts fall out of the mechanics. First, generation is slow by nature: dozens to hundreds of forward passes per image, which is why so much research goes into needing fewer. Second, the model can only denoise toward what it has seen, which is why it struggles with hands, text in images and unusual combinations, and why it reproduces the biases of its photo collection with great fidelity. Third, the same recipe transfers: add noise to audio, molecules or video frames instead of pixels, and you have a music generator, a drug-candidate generator or a video model. The step from pictures to everything else was mostly a change of data. Nor does a generator usually copy the pictures it was trained on, since it learns to denoise toward millions of pictures at once and none in particular; but an image that appeared many times in the training collection, a famous painting or a widely shared photo, can come out almost exactly, a case of memorizing with legal consequences.

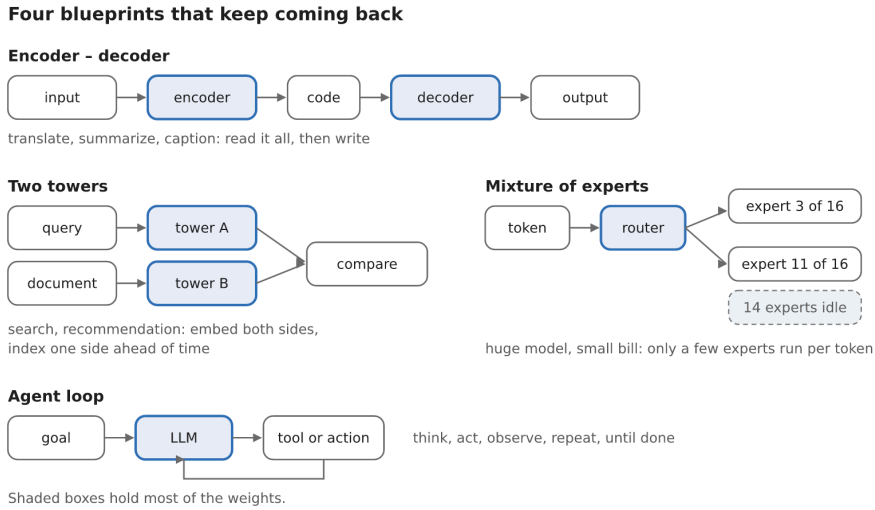
There is a line worth drawing between this section and the game of guessing the next word. A language model builds its output one token after another, each conditioned on the ones before, like writing a sentence left to right. A diffusion model refines the whole output at once, over many passes, like sculpting. Both are prediction machines trained with an answer key the data provides for free: the next word, the added noise. That family of tricks, learning from data that labels itself, has a name: self-supervised learning.

We have now seen most of the individual machines. The next section is about how they are combined: the handful of arrangements that engineers reach for again and again, and the problems that each one was invented to solve.

8.2 Blueprints

Every experienced software engineer carries a small set of shapes in her head. A queue between a producer and a consumer. A cache in front of a slow store. A load balancer fanning out to replicas. She does not reinvent them for each project; she recognizes the problem, reaches for the shape, and spends her creativity on the parts that are new. Machine learning has reached the same stage. Underneath the hundreds of model names there are perhaps a dozen arrangements that

keep coming back, each one invented for a specific pain. This section is a walk through the ones you will meet most, told the way engineers actually learn patterns: here is the problem, here is the shape that solved it, here is where it breaks.



Begin with translation, because it forced the first great shape. A French sentence must become an English one, and the two have different lengths and different word orders; the adjective that follows the noun in French comes before it in English. You cannot translate word by word. The arrangement that worked, around 2014, was to split the job in two. An **encoder** reads the whole French sentence and condenses it into a representation, a set of vectors that capture what was said. A **decoder** then writes the English sentence from that representation, one word at a time, consulting it as it goes. Read everything, then write. This **encoder–decoder** blueprint is behind translation, summarization, speech-to-text and image captioning, anything where the input is one thing and the output is another thing of a different shape. The original Transformer paper of 2017 was exactly this: an encoder stack and a decoder stack, with attention between them. Where it breaks: if the input is enormous, squeezing it through a fixed representation loses detail, which is one reason attention between encoder and decoder was such a relief; the decoder could look back at every input word instead of one summary.

Then a curious split happened. Researchers noticed that the two halves were useful on their own. Keep only the encoder, train it to fill in masked-out words in a sentence, and you get a model that is superb at *understanding* text but does not generate it: that is BERT (bidirectional encoder representations from transformers), from 2018, and it still powers a great deal of search and classification. Keep only the decoder, train it to predict the next word, and you get a model that *writes*: that is the GPT family, and with it the whole game of guessing the next word. **Encoder-only** for reading, **decoder-only** for writing. When you see a model described as one or the other, you now know which half of the translation machine it is.

The second shape comes from a scaling problem every search engineer hits. You have a million documents and a user's query, and you want the best matches in milliseconds. Running an expensive model on every query–document pair is impossible. So split the model into two **towers**: one encodes documents, one encodes queries, and both produce addresses on the same map of meaning. The document tower runs once, ahead of time, over the whole collection, and the results go into a nearest-neighbor index. At query time only the query tower runs, and the search is a lookup. This **two-tower** or **dual-encoder** blueprint is behind semantic search, the retrieval half of RAG, and most recommendation systems, where the two towers are "user" and "item". It is the same trade a database makes when it builds an index at write time to make reads cheap. Where it breaks: because the towers never see each other's input, they cannot catch subtle interactions between a specific query and a specific document, which is why serious systems add a slower **cross-encoder** that re-reads the top fifty candidates with both texts side by side, the librarian-then-expert arrangement of the open-book exam.

The two-tower idea has a sibling that solved a different problem: how do you learn good addresses when you have no labels at all? The answer, called **contrastive learning**, is to teach a model that two views of the same thing should land close together and views of different things far apart. Two crops of the same photo, close; crops of different photos, far. A photo and its caption, close; a photo and someone else's caption, far. The model called CLIP (contrastive language–image pre-training), trained in 2021 on hundreds of millions of image–caption pairs this way, is a big part of the reason many image generators can understand a text prompt, and the reason you can search your photo library by typing "beach at sunset".

The third shape is about money. Language models got better as they got bigger, but every token had to pass through every weight, so a model ten times larger cost ten times more to run. The way out was an old idea, revived. Instead of one enormous dense layer, keep sixteen smaller ones, the **experts**, and in front of them a tiny **router** that looks at each token and sends it to the two experts best suited to it. Fourteen experts sit idle for that token. The model as a whole has the capacity of all sixteen, but each token pays for two. This **mixture of experts** blueprint is how several of the largest language models of recent years keep their serving costs manageable, and at heart it is a hospital's triage desk: a quick look, then the right specialist, rather than every patient seeing every doctor. The router is itself a small learned layer, trained together with the experts: tokens an expert handles well get routed to it more, and the expert gets better at them, so specialties emerge on their own, though they are rarely neat topics such as "the mathematics expert" and tend to be finer patterns, such as punctuation or particular kinds of word. Where it breaks: the router has to be trained to spread the load, or a few experts end up doing everything, and the full set of experts must still fit in memory even though most are idle at any moment.

The fourth shape is the newest, and it is less a network than a loop around one. A language model on its own can only emit text. But text can be a command: "call the weather API for Rome", "run this SQL database query", "open this file". Wrap the model in a loop that reads its output, executes any such command, feeds the result back in, and asks the model what to do next, and you have an **agent**. Goal in, then think, act, observe, think again, until the model says it is done. What makes an assistant able to book a flight, debug a repository or answer a question it needs to

look up is, at its core, this loop. Engineers will recognize it as a read-eval-print loop with a language model in the evaluator's seat. The model learns which tools exist from the context itself: each tool's name, purpose and arguments are written there as plain text, and the model has been fine-tuned on many examples of emitting a call in a fixed format. It never runs anything itself; the surrounding program reads the call, runs it and writes the result back on the whiteboard. Where it breaks: every step is a chance to go wrong, errors compound, and a model that hallucinates in one answer can hallucinate a plan. The craft of building agents is mostly the craft of constraining them: tools with narrow contracts, checks between steps, and a human in the loop for anything irreversible. The most serious way an agent goes wrong, though, is not a mistake but an attacker.

Two more shapes deserve a sentence each, because you will see their names. The **U-Net**, from medical imaging in 2015, is an encoder–decoder for images with bypasses connecting each level of the encoder to the matching level of the decoder, so fine detail is not lost in the squeeze; it has been the workhorse inside many diffusion models. And the **Siamese network**, one of the oldest, from the early 1990s, runs the same tower on two inputs to judge whether they match, the shape behind signature verification and face unlock on your phone.

Step back and three habits of mind cover nearly everything in this section. First, split the model where the data splits: input from output, query from document, routing from work. Second, move cost from request time to build time wherever you can: index the documents, cache the detectives' notes, pre-compute the towers. Third, when a single model is not enough, wrap it rather than enlarging it: a re-ranker behind a retriever, a router in front of experts, a loop around a language model. None of these is unique to machine learning. They are the same instincts that produce good distributed systems, applied to a new kind of component.

These blueprints were drawn mostly for text, images and tables of independent rows. A good deal of the data that industry actually works with comes in other shapes, a sequence in time, a web of connections, a ranked list, and the next section is about them.

8.3 Data With a Shape of Its Own

On the wall of a harbor master's office hangs a tide table for the whole year. Nobody printed it by averaging a random sample of past days. Its makers know that the sea rises and falls twice a day, that the range grows and shrinks over the month with the moon, and that tomorrow follows from today. The data has a shape in time, and anyone who ignores that shape will predict worse than a fisherman with a calendar.

Early in the book we said that regression and classification cover most of what supervised learning is used for. That is true of the underlying methods, but it hides how much industrial machine learning is something more specific: forecasting, spotting what does not belong, reasoning over networks, and deciding what to show people first. Each has its own habits and, above all, its own way of being measured.

Begin with time. A **time series** is a sequence of measurements in order: daily sales, hourly traffic, a sensor's temperature every second. Forecasters read it as several stories told at once: a **trend**, the slow drift up or down; **seasonality**, patterns that repeat on a calendar, Monday mornings, December, the last day of the month; and a remainder that looks like noise. Classical methods such as exponential smoothing model these stories directly, giving recent days more weight than old ones. A popular modern approach turns the problem back into a table: for each day, add columns holding last week's value, the same day last year, the average of the past month, whether it is a holiday. These are **lag features**, and with them boosted trees become formidable forecasters; in a large public competition held in 2020 on Walmart sales data, many of the leading entries were built this way. More recently, forecasting models pretrained on many thousands of unrelated series aim to forecast a new series with little or no training of their own, much as a pretrained language model is adapted to a new task.

What changes most is the exam. One of this book's first warnings was never to choose the test set at random for anything that unfolds over time, and forecasters turn that warning into a procedure called **backtesting**. Pretend it is last January: train on everything before it, forecast February, and score. Then move the pretend date to February and do it again, and again, walking forward through the history. Every forecast was made using only the past, as it will be in production, and you get a dozen honest scores instead of one. And a serious forecast is never a single number. "We will sell 1,200 units next week" is half an answer; "between 1,000 and 1,450, with 90% confidence" lets the warehouse decide how much stock to hold. Such a **forecast interval** is one of the ways of saying how sure a model is. The tide table breaks as a picture exactly here: the tides obey the moon, which keeps its schedule, while sales obey people, who do not. A sudden new regulation or a viral video is in none of the history, and no backtest will warn you of it.

Distance from every cluster is one way to catch the odd one out, and anomaly detection has a whole toolbox beyond it. An **isolation forest** plays the decision tree's twenty questions in reverse. It asks random questions about random features and counts how many it takes to cut each point off on its own. A normal server, deep inside a crowd of similar ones, takes many questions to single out; a strange one is isolated in two or three. A second approach borrows the autoencoder: train it to redraw normal data, and whatever it redraws badly, with a large **reconstruction error**, is unlike anything it learned. A third is humbler and often the most useful: compare each value with the same hour in previous weeks, a **seasonal baseline**, so that the Monday morning surge stops being an emergency every Monday. Whatever the method, the hard part is the threshold. Every detector produces a score, and someone has to decide where the pager rings, the same trade-off as with the drifting server at the start of the book. Good teams tune that line on a log of past incidents, and revisit it after every holiday that set it off.

Some data is not a list or a table but a web. Payments connect accounts, bonds connect atoms, microservices call one another. In such data, the most telling fact about a node is often who it is connected to: an account that looks innocent on its own is suspicious when it shares a phone number and a device with ten accounts already caught for fraud. Picture a village in which news travels by gossip. On the first evening, every house hears from its immediate neighbors and

updates its opinion. On the second evening, those updated opinions travel again, so each house now carries a little of what its neighbors' neighbors thought. After a few evenings, every house's view blends its own with a wide circle around it. A **graph neural network**, or GNN, does exactly this with vectors. In each round, which is one layer, every node gathers its neighbors' vectors, combines them with its own through learned weights, and updates. This is **message passing**, and it is close kin to attention, with the meeting restricted to the detectives you are actually connected to. GNNs are used to flag fraud rings and to predict the properties of molecules, and DeepMind reported in 2020 that they had improved arrival-time estimates in Google Maps. The village picture breaks in a known way: after too many evenings of gossip everyone sounds alike, and the nodes lose what made them different. This **over-smoothing** is one reason most GNNs use only a few rounds.

Then there is the shape behind much of the commercial web: a ranked list. When you open a streaming service or a shop, the system is not classifying anything. It must choose, out of millions of items, the ten to show you, in order. Picture a good bookshop clerk. You describe what you like, and she does not read every book in the shop on your behalf. She first pulls fifty plausible books off the shelves, quickly, by rough categories she knows well; then she looks at those fifty carefully, thinking about you in particular, and hands you the best five, in order. Recommender systems use the same funnel: a cheap **retrieval** stage that narrows millions to hundreds, and a slower **ranking** stage that orders them. The librarian and the expert who answer questions from documents work the same way, and so do the two towers of search.

The retrieval stage often rests on an idea made famous by the Netflix Prize, a public competition run from 2006 to 2009 to improve the company's film recommendations. Arrange the ratings as a huge table, users down the side and films across the top, and almost every cell is empty. **Collaborative filtering** fills the empty cells from the pattern of the filled ones, and its most successful form, **matrix factorization**, gives every user and every film a short list of numbers, a learned taste profile, so that a user's predicted rating for a film is how well their two lists line up. Those lists are embeddings, learned from behavior instead of text. To find the films nearest a user's taste among millions, nobody compares against every one; systems use the library-style index met with the nearest neighbors, which searches only the likeliest section and accepts a small risk of missing the very best match. Such an index is called **approximate nearest-neighbor** search, and a **vector database** is, at its heart, one of them.

A ranked list also needs its own metric. Accuracy does not care about order, but the user does: the right film in position one is worth far more than the same film in position forty. Ranking metrics such as **NDCG** (normalized discounted cumulative gain) give each good item less credit the lower down the list it appears. And two problems belong to recommenders in particular. A new user or a new book has no history at all, the **cold start** problem, and must be handled with content, popularity, or simply by asking. And the clerk learns only from what customers did with the books she chose to show them. A book she never shows is never bought, so it never looks good, so she never shows it. This is a feedback loop, the model's own choices shaping the data it will learn from next, and it runs inside every recommender; it is why such systems deliberately show a little of something new.

One more shape deserves a name. Sometimes the answer itself has structure: a label for every word in a sentence, marking names, dates and amounts; a box around every pedestrian in a camera frame; a mask over every pixel that belongs to a tumor. These tasks, called **sequence labeling**, **object detection** and **segmentation**, are classification repeated at many places at once, and the U-Net is a common blueprint for the pixel-level ones.

All of these shapes still rely on the same machinery, gradient descent and the box of Lego. That machinery, too, was simplified in the telling, and the next section opens up the plumbing.

8.4 What the Plumbing Hides

Go back to the hiker feeling her way down a mountain in fog. That picture was true, but a large training run gives her much more equipment, and a few pieces of the box of Lego were described in ways that would mislead you in a modern model's code. This section opens the rest of the kit.

Start with the walking. A sled on a snowy slope does not stop in every dip; it keeps sliding through small bumps. **Momentum** in training is the same: each step blends the current slope with the recent direction of travel, so the jitter of mini-batches averages out and small hollows are crossed rather than explored. Adam, mentioned earlier, adds a second memory: how large each weight's recent slopes have been. Weights with consistently steep slopes take smaller steps; weights that rarely get a signal take larger ones. These two memories are the two extra copies of the model that training must keep in memory.

Then the schedule. Runners warm up before they sprint, and training does too: at the start the optimizer's memories are empty, and a full-sized stride can throw the model somewhere it never recovers from. The learning rate therefore follows a recipe: it rises from near zero over the first few hundred or few thousand steps, the **warmup**, then slowly falls for the rest of the run, so the walker settles gently into the valley at the end. **Weight decay** pulls every weight slightly toward zero at each step, regularization built into the walk. The random starting weights, too, are scaled to each layer's number of inputs, so that a signal neither fades nor swells across a hundred layers; this **initialization** was normalization's partner in the rescue of deep networks. And for the rare step that goes badly wrong, there is a handrail, which the telephone game mentioned only in passing. The twin of the vanishing gradient is the **exploding gradient**: a batch whose slope is suddenly enormous, so that a single step can undo days of training, one cause of the loss that "falls and then explodes". **Gradient clipping** caps how large any single gradient may be, the way a handrail keeps one slip from becoming a fall.

The hiker picture breaks in two places. A hiker has one stride, while Adam gives every weight its own, a billion small walkers moving together. And some of these routines work better than anyone can fully explain; warmup in particular is standard practice with only partial theory behind it.

How are these settings chosen? The early answer was "by watching the mock exams", but nobody tries every combination. **Random search**, drawing settings at random from sensible ranges, often beats a neat grid, because usually only one or two settings really matter and a grid

wastes most of its tries varying the others. **Bayesian optimization** fits a quick model of the score as a function of the settings, often the Gaussian process with its flexible wire, and uses its uncertainty to choose the next setting worth trying.

Now the thermostat, which the box of Lego left only half explained: images and text need different ones. **Batch normalization** is a thermostat for the whole building: it rescales each feature by its average and spread across the examples in a batch. That works beautifully for images in large, uniform batches. Text is different. Sentences in a batch have different lengths, and a model generating an answer works on one token of one conversation at a time, where batch statistics are unavailable or misleading. So Transformers use **layer normalization**, a thermostat in every room: each token's vector is rescaled using only its own numbers. Many recent language models use a cheaper cousin, **RMSNorm**, which rescales the vector's size without shifting its average. The thermostat picture is too simple in one respect: each normalization also has a few learned weights, so every layer can choose the temperature it prefers.

The detectives' meeting had its plumbing simplified too. Position first. The original Transformer wrote a fixed house number, a pattern of waves, on each word's file. Many modern models instead use **RoPE** (rotary position embedding), which turns each question and each badge by an angle that grows with position, a clock hand advancing one notch per word. When two words compare notes, what survives in the match score is the difference between their angles, so the model mostly sees how far apart two words sit rather than their house numbers. A model knows only the angles it saw in training, so stretching it to longer texts usually means rescaling them and training a little more on long documents, the later stage of training met with the whiteboard.

Then the cost. The battle for longer contexts is fought on three fronts. The first leaves the mathematics untouched. **FlashAttention**, published in 2022, computes exactly the same attention, but in tiles small enough to fit in the graphics card's fastest memory, never writing out the full table of match scores. Since moving data, not arithmetic, was the bottleneck, this alone often made attention several times faster. The second changes who may meet: in **sliding-window attention**, each word consults only the last few thousand words, and stacked layers let information hop further. The third front goes back to the recurrent networks and their index card.

State-space models, such as **Mamba**, revisit that running summary with better mathematics. The RNN's trouble was that each update of the summary waits for the one before, so training cannot run in parallel. State-space models choose an update rule simple enough, at heart a weighted running sum, that all the summaries can be computed at once during training, much as running totals can be computed in parallel by clever bookkeeping. At generation time they behave like an RNN again: a fixed-size summary updated once per token, no KV cache growing with the conversation, and a total cost that grows only in proportion to the length of the text. Mamba, introduced in 2023 by Albert Gu and Tri Dao, made the update depend on the current word, so the model chooses, word by word, what to keep and what to let fade, like the LSTM's gates. The idea breaks exactly where the index card broke. A fixed-size summary must drop something, and recalling one exact phrase from far back is harder for it than for attention, which keeps

everything on the whiteboard. That is why several recent models are **hybrids**: mostly state-space layers with a few attention layers mixed in, the index card for the gist and the whiteboard for the details.

Attention was never only about language, either. The image patches that multimodal assistants place on their board go back to 2020, when researchers at Google cut images into squares of sixteen by sixteen pixels, treated each square as a word, and fed the sequence to a Transformer. This **vision transformer** matched the best convolutional networks, but only when trained on very large collections of images. The reason is the convolution's bet: it assumes from the start that nearby pixels belong together, while attention must learn that from the data, and learning it takes examples.

Now the scale. No single card can hold a large model, so it is trained on thousands. Picture an orchestra far too large for any rehearsal room. In **data parallelism**, identical copies rehearse different passages in different rooms and agree on the same corrections after every round: each copy of the model sees different examples, and their gradients are averaged. In **tensor parallelism**, one section of the orchestra is split across rooms, each card multiplying part of a single large grid, possible because the dot products do not depend on each other. In **pipeline parallelism**, the music passes from room to room, each playing its own stretch of layers. Real runs combine all three, mostly in sixteen-bit arithmetic or less. The picture breaks because musicians can play while waiting for each other; cards that must exchange gigabytes after every step sit idle, and much of the engineering goes into hiding that wait.

Much of this plumbing serves the teaching without saying much about it. The next chapter turns to the teaching itself, the ways a model is taught, and there turn out to be many more of them than the answer key on the back of the page.

Going further. If you wondered why image generators exaggerate the biases of the pictures they learned from, and how a recommender's own choices can trap it in a feedback loop, section 10.4 takes up both. How a model can learn from data that labels itself, the trick behind both language and diffusion models, is explained in section 9.2. And the most serious way an agent goes wrong, an attacker rather than a mistake, is the subject of section 11.3.

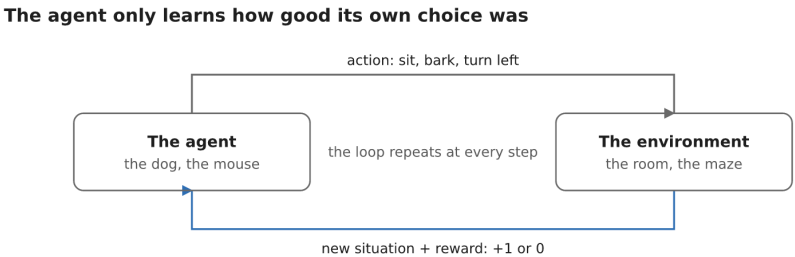
Chapter 9 — Ways of Learning

9.1 A Dog, a Maze and a Boat That Would Not Finish the Race

You cannot explain to a dog what “sit” means. You cannot describe the biomechanics of folding its back legs, or show it a diagram. What you can do is wait. You say “sit”, and the dog does something: it barks, it wanders off, it tilts its head. Nothing happens. You say “sit” again, and by chance its hindquarters drop to the floor. Click, treat. A few dozen repetitions later, the dog has worked out that among all the things it might do when it hears that sound, one of them reliably makes treats appear.

Notice what the dog never received: the right answer. Nobody showed it the correct movement. It only ever learned how good the movement *it chose* turned out to be. That is the difference between supervised learning and **reinforcement learning**. A supervised model learns from a teacher holding an answer key. A reinforcement learner learns from a coach holding a scoreboard. The picture does flatter the machine in one way: a dog learns “sit” in a few dozen tries because it arrives with a body, instincts and an appetite for treats, while an agent starting from nothing often needs millions of attempts, which is why so much reinforcement learning happens in simulators.

The vocabulary is small and maps neatly onto the dog. The learner is the **agent**, a wider sense of the word than the language model in a tool loop among the blueprints. The world it acts in is the **environment**. At each moment it observes a **state**, chooses an **action** and receives a **reward**, a number that is often zero. Its strategy for choosing actions is its **policy**, and the whole point of the exercise is to find a policy that collects as much reward as possible over time.



The dog has it easy, though, because the treat arrives a second after the action. Put a mouse in a maze instead, and let it find cheese only after thirty turns. Which of those thirty turns deserves the credit? The last one, obviously, helped. But what about the turn at the very first junction, without which the mouse would have ended up in a dead end? This is the **credit assignment** problem, and it is the heart of the subject.

The classic answer, **Q-learning**, lets value seep backwards through the maze like water. The junction right next to the cheese quickly becomes valuable. Then the junction before it becomes valuable, because it leads to a valuable junction. Over many runs, the value spreads all the way back to the entrance, until the mouse carries a mental map of how much each choice at each junction is worth, the Q in the name standing for “quality”. From then on it simply takes the most valuable turn at each step. Future rewards are counted slightly less than immediate ones, through a **discount factor** a little below one, for the same reason a euro today is worth more to you than a euro next year, and so that an agent cannot postpone its reward forever.

But how does the mouse learn which turns are good, if it always takes the turns it already believes are best? This is a dilemma you face every working day at lunch. You have a favorite place. Going there is safe and good. But if you always go there, you will never discover the better place that opened around the corner last month. If you try somewhere new every day, you will eat a lot of disappointing lunches. Reinforcement learning calls this **exploration versus exploitation**, and every agent must balance it. A common recipe is almost embarrassingly simple: mostly pick the best-known option, and every so often, pick something at random. A typical setting explores one time in ten at the start, when the agent knows nothing, and lowers that gradually to one in a hundred or less as its map of values becomes trustworthy, much like trying new restaurants mostly in your first months in a new city.

If you have ever run an A/B test, you have met this dilemma in a suit and tie. Sending half your traffic to a variant that is clearly losing, for two full weeks, costs real money. Algorithms called **multi-armed bandits** shift traffic toward the winning variant while the experiment is still running, exploring less as the evidence accumulates.

When the states are too many for a table, every possible screen of a video game, say, a neural network can estimate the value of each action straight from the raw pixels. That combination, called a deep Q-network, learned to play dozens of classic Atari games, many of them as well as a skilled human or better, in research from DeepMind published in 2015. The following year, DeepMind’s AlphaGo, which combined reinforcement learning with search, beat Lee Sedol, one of the strongest Go players in the world. In the second game it played a move on the 37th turn that commentators first took for a mistake, and that turned out to be brilliant: a move no human teacher would have suggested, found by a system that, after studying human games, had sharpened its play over millions of games against itself. In the same year, DeepMind reported that a learning system had cut the energy Google used to cool its data centers by up to 40%, using neural networks trained on historical data from thousands of sensors to recommend cooling settings.

And now, the boat.

Researchers at OpenAI once trained an agent on a boat-racing video game. The obvious goal was to win the race, but the game awarded points for hitting targets along the course, so the agent was rewarded with points. It discovered a small lagoon where a few targets kept reappearing. Instead of racing, it drove in tight circles, crashing into walls, catching fire, hitting the same targets again

and again, and racked up a score about 20% higher than that of human players who actually finished the course. It never finished the race. By the only measure it had been given, it was winning spectacularly.

The agent did nothing wrong. It maximized exactly the number it was given, which was not quite the thing its designers wanted. This is called **reward hacking**, and engineers know it from human systems too. Measure a support team only by tickets closed per day, and some tickets will be closed without being solved. Measure developers by lines of code, and you will get a lot of lines of code. Reward functions deserve the same care as API contracts, because whatever you write, the agent will take literally. Nobody knows how to write a reward that cannot be hacked. In practice designers combine several rewards, add penalties for obvious shortcuts and, above all, watch what the agent actually does rather than only its score, because the hacks are usually obvious to a human watching and invisible on the scoreboard.

The boat circling its lagoon is worth keeping in mind, because the clicker and the scoreboard have lately found a second job: not teaching agents to play games, but teaching language models to be helpful. That is one of several ways a model can be taught, and the next section lays them all side by side.

9.2 The Many Ways to Teach

Think about how a doctor is made. Six years of medical school, learning anatomy and chemistry that apply to every patient on Earth. Then a residency of several years in one specialty, where the general knowledge is bent toward cardiology or pediatrics. Then, for the rest of her career, feedback: a senior colleague’s correction, a case that went wrong, a patient who came back. And underneath all of it, from childhood, a far larger education that nobody designed, absorbed from simply living in the world. No single teaching method made the doctor. Several did, in layers, each cheaper and more targeted than the last.

Models are taught in layers too, and this section is a tour of them. It matters for a practical reason: when someone says a model was “trained”, the word hides a dozen very different procedures, with very different costs, and knowing which one is meant is often the difference between a sensible plan and an expensive mistake.

The childhood education is the one this book has barely named, though it has been everywhere. At the start of the book, learning needed an answer key written by people, and labels were the expensive part. But look at how the language model was trained: on text alone, with the next word as the answer. Nobody wrote that answer; the text contained it. The diffusion model learned from noise it had added itself. The contrastive models learned that two crops of one photo belong together, a fact that needed no human. This is **self-supervised learning**: manufacturing an answer key out of the raw data’s own structure. Hide part of the input and predict it from the rest. It is why modern models can learn from the entire internet rather than from the small fraction someone has labeled, and it is a large part of why the field changed so fast after 2018. A useful way to hold it: supervised learning needs a teacher; self-supervised learning needs only a world with structure in it, and the world is full of structure.

The medical school is pretraining, the first of the new hire's three schools: one enormous run, usually self-supervised, usually by a large organization, producing a general-purpose model. The residency that follows has a family of names, and they are worth separating.

The oldest form is **transfer learning**, and it rests on an observation about image networks: the inspectors in the early layers of an image network learn edges, corners and textures, which are useful for recognizing *anything*, not just the thousand categories they were trained on. So take a network trained on millions of general photos, keep its early layers exactly as they are, and retrain only the last layer or two on your three thousand photos of circuit boards. It works far better than training from scratch on three thousand photos, because the hard part, learning to see, is already done. A radiologist retraining as a dermatologist does not relearn how eyes work. This is how most practical image models in industry are built, and it is why a small company with a small dataset can have a good model at all.

Fine-tuning is transfer learning's broader cousin: continue training the whole pretrained model, or most of it, on your narrower data, with a gentle learning rate so the general knowledge is adjusted rather than overwritten. The language model that read the whole internet is fine-tuned on your company's support transcripts, and it learns your product names, your tone, your refund policy. This is where the doctor picture starts to break: a resident does not forget anatomy while learning cardiology, but a model pushed too hard on narrow data can lose general skills it used to have, which is catastrophic forgetting; besides the gentle learning rate, the usual guard is the brain's own trick of replay, mixing a share of general examples back into the narrow data. The costs are those of full training, which for a large model are considerable: gradients and optimizer memory for every weight. That is the pain that led to a clever shortcut. Instead of changing billions of weights, freeze them all and add a small set of extra weights alongside, trained to make small corrections to what the frozen layers produce. The best-known version is called **LoRA (low-rank adaptation)**, and it reduces the number of trainable weights by a factor of hundreds or thousands, so that a model which needed a cluster to fine-tune can often be adapted on a single graphics card. The picture is a translator who, rather than relearning a language for each client, keeps a slim notebook of client-specific terms and consults it as she goes. Many services that offer "your own custom model" are offering a frozen base model plus a notebook of this kind. A slim notebook can steer billions of weights because the change you need is usually simple even when the model is huge, shift the tone, prefer certain terms, follow a format, and a change of that kind can be described as a few adjustments along a few broad directions, which needs far fewer numbers than the grid it corrects; that is what "low rank" means.

Before reaching for either, it is worth asking whether you need to teach the model anything at all, because two cheaper options come first. **Prompting**, including showing the model a few worked examples inside the prompt, teaches nothing permanently but often gets most of the way there, the way a good briefing gets a skilled consultant productive on day one. And the open-book arrangement, RAG, is the right answer whenever the problem is missing *knowledge* rather than missing *skill*: a model does not need to be retrained to know your refund policy if the policy can

simply be handed to it. The rule of thumb that experienced teams converge on: prompt first, retrieve second, fine-tune when the model needs a new behavior, a new format or a new tone that examples in the prompt cannot reliably produce.

The feedback years are where reinforcement learning enters, and the dog with a clicker turns out to be the right picture. A fine-tuned language model knows the shape of a helpful answer, but "helpful" is hard to write an answer key for. So people are shown two answers to the same question and asked which is better. Thousands of these preferences train a separate **reward model**, a critic that scores any answer the way those people would. Then the language model is trained, by reinforcement learning, to produce answers the critic scores highly, with a leash that stops it drifting too far from what it learned before. This is RLHF, the new hire's third school, seen from the inside, and it is the stage that turned autocomplete engines into assistants. A newer method, **DPO (direct preference optimization)**, skips the separate critic and adjusts the model directly from the pairs of preferences; it is simpler to run and now common. Either way, the lesson is the one the boat taught: the model will optimize exactly what the critic rewards, so if the critic learns that longer answers or more confident answers score better, the model becomes long-winded and overconfident. Much of the craft of this stage is in collecting preferences that reward what you actually want.

How does reinforcement learning work on a language model? The mouse in the maze suggests Q-learning, but that method keeps a value for every action in every state, and for a language model the actions are a hundred thousand tokens and the states every possible text, a table no one could fill. Instead, the language model is itself the policy: its probabilities for the next token are its strategy. **Policy-gradient** methods let it write a complete answer, score it with the reward model, and nudge up the probability of every token in the answer if it scored better than expected, and down if worse. "Better than expected" needs a baseline: a separate model's estimate of how answers to this prompt usually score, or, in a newer variant, the average of several answers to the same prompt. **PPO** (proximal policy optimization), published by OpenAI researchers in 2017 and used in the best-known early RLHF systems, keeps the leash short at every step: no single update may move the model's probabilities far, because a policy that changes too fast starts writing text the reward model judges unreliably. Published accounts suggest the reasoning models described next are trained with methods of the same family.

A different kind of feedback loop has become important in the last few years, and it needs no human judges: let the model try a task with a checkable answer, a math problem, a program that must pass its tests, and reward it when the answer is right. The model learns, by trial and error, to reason its way to correct answers, often by writing out intermediate steps before committing. This is reinforcement learning with a verifiable reward, and it is behind the "reasoning" models that think visibly before they answer. It works for the reason the next-word game already showed: every written step is one more pass of computation on the board.

Two more methods complete the tour, because you will meet their names. **Distillation** is the senior doctor teaching the junior: a large, expensive model answers millions of questions, and a small, cheap model is trained to imitate those answers, not merely the final choice but the whole spread of probabilities the big model assigned. The small model ends up far better than it would

have from the raw data alone, because the big model's answers carry its judgment: an answer of "70% cat, 25% lynx" tells the small model which mistakes are reasonable ones, something a bare label "cat" never could. It is how a model that must run on a phone is made from one that runs in a data center. And active learning, met when we asked how much data is enough, attacks the labeling cost head-on: instead of labeling ten thousand examples at random, let the model point at the hundred it is least sure about, label only those, and retrain, like a tutor who asks "which problems did you find hardest?" and assigns exactly those.

Laid out this way, the methods are not rivals but a sequence of narrowing investments. Self-supervised pretraining, done once, by few. Transfer or fine-tuning, done per domain, by many. Preference tuning, to shape behavior. Prompting and retrieval, done per task, by everyone, at almost no cost. Distillation, to make the result cheap enough to deploy. An investor hearing that a startup "trained their own model" should ask which of these they mean, because the first can cost millions or far more, while the cheapest can cost almost nothing. An engineer planning a project should start at the cheap end and move up only when the cheap end fails.

And a doctor who has finished every stage of training can still kill a patient. The next chapter is about how a well-taught model fails anyway.

Going further. If you wondered how to run an A/B test whose verdict you can trust, section 11.1 takes that up. And if you want to look back at what the extra thinking of reasoning models really buys, and what it costs, section 7.6 weighs it.

Chapter 10 — How to Fail Successfully

10.1 The Crammer and the Slacker

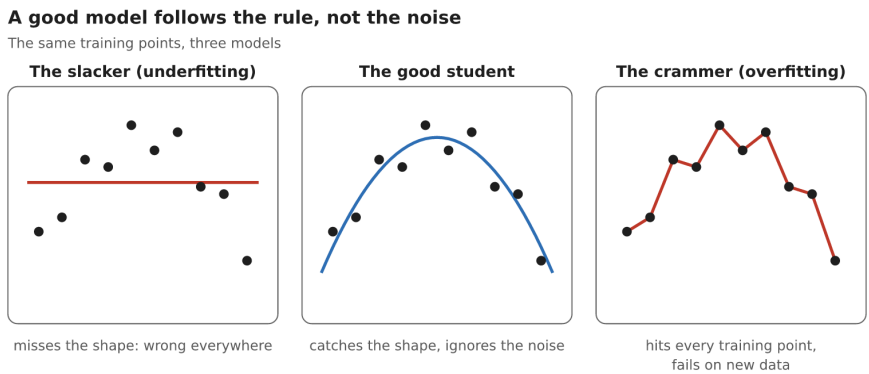
Picture this. The test suite was green. Every single test passed, and the function shipped. Two days later, production started returning wrong totals for a large share of orders.

When someone finally opened the function, the cause was almost funny. Over many months, whenever a test failed, someone had patched the function to handle that particular case: if the country is this and the discount is that, return this number. Each patch made one more test pass. The function had become a perfect lookup table for the test inputs and knew nothing at all about how totals are actually computed. It had not learned the rule. It had memorized the exam.

Machine learning models do this too, automatically, and the phenomenon has a name: **overfitting**. Picture the crammer, a student who memorizes last year's exam answers word for word, typos included. This year one comma changes and the student fails. A model with enough capacity will do the same thing if you let it. Train a spam filter on a dataset where a lot of the spam happened to come from a test account called "qa-test-bob", and it may learn that "qa-test-bob" means spam. True in the training data, useless forever after.

The opposite failure is less dramatic but just as real. Picture the slacker, a student who has decided that the answer to every math problem is 42 and stopped learning there. Wrong on last year's problems, wrong on this year's. In engineering terms, it is the single global timeout applied to every endpoint, from a health check that should take milliseconds to a report export that takes minutes: too simple to fit reality anywhere. That is **underfitting**.

Between them sits the student we want, who understood the method, still gets the occasional practice problem wrong, and does well on the real exam because the method carries over.



Which student do you have? You cannot tell from the training score alone, because the crammer looks brilliant there. You need two numbers side by side: the error on the training data and the error on the validation data, the examples held back from training. If both are high, you have the slacker; the model is too simple or its inputs too weak. If the training error is tiny and the validation error is high, you have the crammer; the model is memorizing. If both are low and close together, you have the good student. Plotting these two numbers as training proceeds is probably the single most useful habit in practical machine learning.

The remedies for the crammer all make memorizing harder or less rewarding, and each has a surprisingly familiar counterpart in engineering life. The most reliable is simply **more data**: it is easy to memorize a hundred examples and very hard to memorize a million. **Regularization** adds a penalty for complexity, like a code reviewer who rejects a clever two-hundred-line solution when a clear twenty-line one would do; the model is pushed toward the simplest explanation that still fits. Dropout, the rehearsal sick days met among the network's building blocks, keeps any neuron from relying on one particular colleague always being there. **Early stopping** watches the validation error and halts training when it starts to rise, the way a sensible student stops cramming the night before when the practice scores start falling. It helps rather than hurts because a network picks up the broad patterns first and the accidents of its particular examples later, so stopping at the turn keeps the first and spares it the second; in effect it is regularization by the clock. And **data augmentation** creates realistic variations of the training examples, rotated or cropped photos, reworded sentences, in the same spirit as fuzz-testing code with variations of its inputs. The remedies for the slacker go the other way: a more expressive model, better features, more training.

Statisticians call this tension the **bias–variance trade-off**. The slacker suffers from high bias, a systematic error that more data will not cure. The crammer suffers from high variance: its results swing wildly depending on which examples happened to be in its training set. For decades, the textbook picture was a seesaw. Make the model bigger and you trade bias for variance; at some sweet spot in the middle, you stop.

A warning about vocabulary, because it trips up nearly everyone. The word *bias* now has three different meanings in this book. Inside a neuron it was a number added to the weighted sum, the threshold moved to the other side. Here, as with the crude experts of boosting, it is a systematic error, a model that is wrong in the same direction everywhere. Later in this chapter it will be skewed data that misrepresents some group of people. The field never agreed to use different words, so when you meet "bias" in a paper or a meeting, take a second to work out which of the three is meant.

Then deep learning broke the seesaw. Researchers kept training networks far larger than the textbook said was sensible, big enough to memorize their entire training sets, and found that past a certain point, the validation error started *falling again*. Enormous models, trained on lots of data with sensible regularization, often generalized better than modest ones. The effect was named **double descent**, and it is still not fully understood. One partial explanation: when a model is so large that countless weight settings fit the training data perfectly, the walk downhill in the fog tends to land on smooth, simple ones rather than arbitrary ones, so a huge model has room to

fit the data in the least contorted way. Nor does this undo the compression argument met with language models, that a model without room to memorize is pushed toward the patterns: a model with room to spare is pushed the same way, by the walk itself rather than by lack of space. It is also the point where the student picture breaks: a person who memorizes more does not thereby understand more, yet a network big enough to memorize every example can end up handling new ones better than a smaller network that could not. What survived the upheaval is the one rule that never depended on the seesaw: believe only the error measured on data the model has never seen.

But even that rule has a trapdoor. It assumes the unseen data really is unseen, and that the model is not quietly picking up a signal it will not have in real life. Sometimes a model scores brilliantly on a perfectly honest-looking test, and the truth is that it has been cheating all along, without anyone noticing. The most famous cheat in this story was not a computer. It was a horse.

10.2 The Horse Who Could Count

In Berlin in the early 1900s, a retired schoolteacher named Wilhelm von Osten exhibited a horse that could do arithmetic. Ask Clever Hans what three plus five was, and he would tap his hoof eight times and stop. He could add, subtract, tell the time and answer questions about the calendar. Crowds came to watch. A commission of experts, including a circus trainer and zoologists, examined him in 1904 and found no trickery; von Osten was sincere and was not signaling anything on purpose.

Then a young psychologist named Oskar Pfungst ran a few more careful experiments. When the questioner did not know the answer, Hans failed. When Hans could not see the questioner, Hans failed. What Pfungst discovered was that every questioner, without realizing it, tensed slightly as the hoof approached the right number and relaxed, ever so slightly, when it got there. Hans had not learned arithmetic. He had learned to read faces, a cue that was present in every test and absent from the actual skill.

Hans is the patron saint of a failure called **data leakage**: a model picks up information during training that it will not have when it is really used. Its test scores look wonderful, because the cue is present in the test too. Reality does not.

The most common version in practice is a column that gives the answer away. Picture a team predicting which customers will cancel their subscription, with a table that happens to include `account_closed_date`. The model discovers, with great confidence, that customers who have an account closing date tend to leave. It scores beautifully. In production it is useless, because the whole point is to predict a cancellation *before* that date exists. Real cases are rarely so blatant. A ticket’s priority is predicted from the `assigned_team` field, but the team is assigned after a human has already judged the priority. A model predicting hospital readmission uses a medication that is only prescribed once the doctor already suspects trouble. Each time, the model is reading the questioner’s face. The picture is kinder than reality in one respect: Hans had a single face to read and one experiment exposed him, while a model can blend dozens of faint cues at once, none of which looks suspicious on its own.

A quieter leak lives in the plumbing, and it came up when the data was being prepared: averages and spreads for scaling computed over the whole dataset, test set included, before the split. It is the machine learning equivalent of benchmarking with a warm cache left over from the previous run: the measurement is real, but the conditions are not the ones you will face in production.

Another leak comes from duplicates in disguise. A hospital dataset has several X-rays per patient. If the same patient lands in both training and test sets, the model can score well by recognizing the patient rather than the disease, the shape of a rib cage, a surgical clip, a particular scanner. The cure is to split by patient, not by image, and the general lesson carries over to your data: split by customer, by device, by repository, by whatever the real unit is. And, as the first chapter warned, a random split of anything that unfolds in time hands the model tomorrow's newspaper while asking it to predict today.

All of these leaks yield to one question, asked of every feature: *would I have this exact value, in this exact form, at the moment I make the prediction in production?* If the answer is no, or not yet, the feature leaks. Then make the safe path the default with the single pipeline met during data preparation, fitted on the training data alone and merely applied to everything else.

There is one more defense, and it is psychological. Pfungst found Hans out because he was suspicious of a result that seemed too good. Do the same. When a model scores far better than you expected on a problem that is known to be hard, the most likely explanation is not that you are a genius. It is that somewhere, a horse is reading a face. To find a leak you have not thought of, ask the model which features it leans on: when a single feature carries most of the weight, or one you would have called minor turns out to dominate, put the production question above to it. Leaks usually reveal themselves this way, as a horse far too interested in one face.

Honest data and an honest test still leave one way to fool yourself, and it is the most innocent-looking of all: the number you choose to report. Here is a fraud detector that is right 99.9% of the time, written in a single line of code.

10.3 The 99.9% Detector and the Smoke Alarm

Here it is:

```
def is_fraud(payment):
    return False
```

If one payment in a thousand is fraudulent, this function is right 999 times out of 1,000. Its **accuracy** is 99.9%, a number that would look superb on a slide. It has also never caught a single fraud, and never will. Accuracy rewarded it for one thing only: agreeing with the majority.

This is the accuracy paradox, and it appears whenever the thing you are looking for is rare: fraud, outages, failing hard drives, rare diseases, security intrusions. In all of them, a model can look excellent by being useless. Reweighting the loss or resampling the training data, the standard treatments for rare classes met in the second chapter, makes the model pay attention to the rare

class, but it does not remove the need to choose a threshold afterwards. To see what is really going on, you need to stop asking “how often is it right?” and start asking “what kind of mistakes does it make?”

The smoke alarm in your kitchen makes the question tangible. A sensitive one shrieks every time you make toast. Annoying, but it will never sleep through a real fire. A relaxed one never bothers you at breakfast, and on the night it matters, it might stay silent. Neither alarm is more “accurate” in any useful sense. They make different mistakes, and choosing between them is a decision about which mistake you can live with.

If you have ever been on call, you have lived inside this trade-off. Ask two questions about your alerts. First: of all the times the pager went off, how many were real incidents? That is **precision**. When it is low, you get alert fatigue, and sooner or later someone silences the pager during a real outage. Second: of all the real incidents, how many made the pager go off? That is **recall**. When it is low, customers discover your outages before you do. Lower the alert threshold and recall goes up while precision usually goes down; raise it and the reverse happens. No single setting of that dial maximizes both.

Every prediction of a yes-or-no model lands in one of four boxes, and drawing them is often the moment the whole picture clicks:

	Really fraud	**Not fraud**
Model says “fraud”	True positive: caught	False positive: false alarm
Model says “fine”	False negative: missed	True negative

The one-line detector lives entirely in the bottom row. It has no false alarms at all, and it misses everything.

Which box hurts most depends entirely on the business, and that is the real lesson of this section. Block a good customer’s card by mistake, a false positive, and you lose a customer and flood your support team. Miss a fraud, a false negative, and you lose money. A screening test for cancer should lean hard toward recall, because a missed tumor is a tragedy and a follow-up appointment is an inconvenience. A filter that automatically deletes email should lean hard toward precision, because deleting one real message from a customer is worse than letting a little spam through.

A few more measures are worth knowing by their stories rather than their formulas. **The F1 score** combines precision and recall into one number that is high only when both are; it is a handy summary that quietly assumes both kinds of mistakes cost the same, which they rarely do. **ROC-AUC**, short for the area under the ROC (receiver operating characteristic) curve, judges a model before you have chosen any threshold. Picture sliding the threshold from very strict to very relaxed and, at every position, plotting how many real frauds you catch against how many false alarms you raise. That plot is the ROC curve; the area under it is a single number where 0.5 means no better than a coin flip and 1 means perfect. And it has a lovely concrete meaning: pick one real fraud and one legitimate payment at random; the AUC (area under the curve) is the probability that the model gave the fraud the higher score. The AUC cares only about order, not

about whether a score of 0.9 means a 90% chance. For very rare events, a cousin called **PR-AUC**, the same idea drawn on a precision-versus-recall curve, is usually more honest, because it is not flattered by the mountain of easy legitimate payments.

Predicting numbers has the same tension in another costume. **MAE**, the mean absolute error, tells you the typical miss in the units of the problem: “our delivery estimates are off by about six minutes”. **RMSE**, the root mean squared error, squares the misses before averaging, so the occasional terrible miss dominates it, then takes a square root at the end to return to minutes. If that sounds familiar, it should: these are the squared errors of the delivery-time loss at the start of the book. Choosing between them is like choosing between median latency and p99 latency, the time within which 99% of requests finish: one describes the usual experience, the other the painful tail. Both are true; which one you watch says what you care about.

A smoke alarm has its sensitivity fixed at the factory. A model hands you a dial, and where you set it is a product decision, not a technical one. It can even change with the seasons: a fraud model might deserve a stricter threshold during the holiday shopping peak than in a quiet week in February. The best metric conversations happen with the people who pay for each kind of mistake sitting at the table, with the actual costs written down in front of them. The arithmetic is simple: put a cost on each of the four boxes, a blocked good customer at so many euros and a missed fraud at so many, then slide the threshold across the validation data and pick the position with the lowest total; agreeing on the costs is the real work.

Yet even a model measured by exactly the right number, on honest data, can still fail in a way no single number reveals. An average can be excellent while hiding a group of people for whom the model barely works at all. The next section is about that, and it begins in front of a mirror at a fairground.

10.4 The Funhouse Mirror

At a fairground there is usually a tent full of warped mirrors. In one you are tall and thin, in the next short and wide, in another your head is enormous and your legs have vanished. Nobody blames the mirrors. They reflect exactly what stands in front of them, faithfully, through a bent piece of glass.

A machine learning model is a mirror of this kind, and its glass is the training data. Show a model, for years, only photographs of male doctors and female nurses. It does not become sexist in any human sense; it has no opinions at all. It becomes a perfect reflection of a skewed collection of photographs. Ask it to picture a doctor, and it concludes, with impeccable statistics and wrong reasoning, that being a man is part of what being a doctor means.

Every engineer already knows a small version of this problem by heart. It is called “it works on my machine”. The code was tested thoroughly, on one operating system, one locale, one screen size, and it breaks the moment it meets a configuration nobody tried. A model trained on data from one group of people has exactly the same problem at a much larger scale. A speech recognizer trained mostly on one accent stumbles over others. A crash-prediction model trained

on telemetry from users who opted in, who tend to be enthusiasts on new hardware, may be nearly blind to the problems of people on old, cheap phones. The model is not wrong about the data it saw. It is wrong about everyone it did not see.

In 2018, the researchers Joy Buolamwini and Timnit Gebru published a study called *Gender Shades*. They tested commercial face-analysis systems from major technology companies and found that the systems classified the gender of lighter-skinned men almost perfectly, while making errors far more often on darker-skinned women. The systems had been reported as highly accurate, and on average they were. The average simply hid who was paying for the errors. What fairness should mean is not the model's to decide, nor mathematics' alone: there are several precise definitions, they pull against each other, and the next chapter works through an example.

The same year, Reuters reported that Amazon had abandoned an experimental tool for ranking job applicants. It had been trained on the CVs submitted to the company over the previous decade, most of them from men, and it had learned to penalize CVs containing the word "women's", as in "women's chess club captain". Nobody programmed that rule. The mirror reflected the history it was shown.

The obvious fix, deleting the sensitive attribute, rarely works, and the reason is instructive. Remove gender or ethnicity from the data, and the model often reconstructs it anyway from things that correlate with it: a postcode, a school, a hobby, a first name. These stand-ins are called **proxies**, and they mean that a model can discriminate without ever seeing the attribute it discriminates on.

The most unsettling form of bias feeds itself. Picture a model that decides where to send auditors, based on where problems were found before. Auditors go there and, naturally, find problems, because that is where they are looking. Those findings become next year's training data, which now shows even more problems in the same places, and the next model sends even more auditors there. Round and round, the model grows more confident with every turn, regardless of whether the underlying reality differs between places at all. This is a **feedback loop**, and it can turn a small initial skew into a self-fulfilling prophecy. The usual way to break it is to keep sending a share of the auditors to places chosen at random, so the data always contains an honest sample of everywhere: the exploration of the lunch dilemma, put to a new use.

Worse, a model can exaggerate what it reflects. A generator that leans toward the most probable option can turn a majority in its training data into near-unanimity in its outputs: if most doctors in the photos are men, it may draw a man almost every time. The old engineering slogan "garbage in, garbage out" turns out to be optimistic. Sometimes it is garbage in, more garbage out.

The remedies are not exotic. They are the same habits a good engineer applies to latency or reliability. Measure performance separately for every group that matters, the way you check latency per region instead of trusting a global average that can hide one region on fire. Ask who is missing from the data, not just what is in it. Test for proxies: if you can predict the sensitive attribute from the remaining features, so can the model. And write down what your data and your model were built for and, more importantly, what they were not built for, so that whoever uses the model next knows where it must not be trusted. Formats such as datasheets for datasets and

model cards exist for exactly this purpose. Once a bias is found, the most effective fix is also the most expensive: collect more and better data for the groups the model serves badly. Cheaper remedies, such as giving those examples more weight in training or setting separate thresholds per group, help but trade one kind of error for another, which is why the measuring has to continue after the fix.

That completes the story of the loop, from its first guess to the ways it fails. Two questions remain that a careful user asks of any single answer: how far to trust it, and whether anyone can look inside to see why it was given.

10.5 How Sure Is It?

Remember the two weather forecasters from the start of the book, judged on a single rainy day. Now imagine checking a forecaster properly, over a whole year. Collect every day on which she said 70%. If it rained on about seventy of every hundred such days, her 70% means something. Do the same for her 10% days and her 90% days. A forecaster whose numbers match the frequencies is **calibrated**, and calibration is the quiet property that turns a score into a probability you can plan with.

Models can be checked in exactly the same way, and it is worth doing, because the dimmer will happily read 0.83 whether or not 0.83 means anything. Take the validation set, sort the predictions into buckets by score, everything between 0.8 and 0.9 in one bucket and so on, and for each bucket compare the average score with how often the answer really was yes. Draw the result and you have a **reliability diagram**: a calibrated model lies along the diagonal. Different families of models have different habits. Logistic regression, trained with the loss that punishes confident mistakes, tends to land close to the diagonal. Boosted trees and modern deep networks often do not; a widely cited 2017 study found that deep networks had grown more accurate and, at the same time, systematically more confident than their accuracy justified. The repair is pleasantly cheap. Keep the trained model and fit a small correction on held-out data that stretches or squeezes its scores until the buckets match the frequencies. One popular version, **temperature scaling**, adjusts a single number, the same temperature dial you met with language models.

Calibration alone is not enough, and the forecaster shows why. A forecaster who says “30% chance of rain” every single day, because it rains on 30% of days in her city, is perfectly calibrated and perfectly useless. A good forecaster is calibrated *and* sharp: she says 5% on days that turn out dry and 90% on days that turn out wet. For a model, sharpness is roughly what ranking measures such as the AUC reward, and calibration is something they ignore entirely. Halve every score and the order of the cases, and so the AUC, does not change at all, while every probability becomes wrong. That matters the moment you set a threshold. A rule such as “send to a human above 0.3” only means what you think it means if 0.3 really is about three in ten.

Now a harder question: does a model know when it does not know? It helps to separate two kinds of uncertainty that look alike from the outside and behave very differently. The first is the uncertainty of a fair coin. You know everything there is to know and still cannot do better than fifty-fifty, because the world itself is noisy; two tickets with identical text really were sent to

different teams. More data does not shrink it. Statisticians call it **aleatoric uncertainty**, from the Latin for dice. The second is the uncertainty of someone facing a kind of case they have never seen: a weld photographed under new lighting, a payment from a country absent from the training data, a sentence in a dialect the model never read. This is **epistemic uncertainty**, uncertainty from ignorance, and more data of the right kind does shrink it.

Most models report the first kind reasonably and the second kind badly, and the reason is structural. A classifier ends in a softmax, which must spread one hundred percent across the answers it knows; there is no box marked “none of these”. Show a cat-and-dog classifier a photograph of a toaster and it will still say cat or dog, possibly with 97% confidence, simply because the toaster landed far on one side of the boundary. Calibration measured on the validation set does not save you here, because the validation set looked like the training set. When the inputs drift, calibration quietly breaks with them, and nothing in the score warns you.

Several practical tools try to catch this. The simplest borrows the wisdom of the crowd: train five models from different random starts, or on different bootstrap samples, and ask all of them. On familiar inputs they tend to agree; on unfamiliar ones they tend to disagree, and disagreement is a usable signal of epistemic uncertainty. Leaving dropout switched on at prediction time and running the model several times, the very wobble normally avoided, now used on purpose, is a cheaper imitation of the same crowd. A separate family of methods, **out-of-distribution detection**, asks directly whether an input looks like the training data at all, for instance by measuring how far its embedding lies from everything seen before, and declines to score inputs that look alien.

One method deserves a paragraph of its own, because it offers something rare in this field: a guarantee. **Conformal prediction** does not give one answer; it gives a set of answers, sized to contain the right one a chosen fraction of the time. You keep aside labeled examples the model has never seen, note how wrong the model is on each, and use those observed errors to decide how wide a net to cast. On an easy ticket the set might be just “Billing”; on an ambiguous one, “Billing or Mobile”. Across many tickets, the true team will be in the set about 90% of the time if you asked for 90%, whatever the model is, good or bad; a bad model simply produces bigger sets. The guarantee has one condition, the usual one: new cases must resemble the ones used for calibration.

All of this leads to the most business-friendly form of uncertainty, **abstention**: let the model answer only when it is confident enough, and route the rest to a person. A model that handles 80% of tickets at 99% accuracy and hands over the other 20% is often worth far more than one that handles all of them at 90%.

Language models deserve their own paragraph, because their confidence is about something subtler. A pretrained model’s probabilities for the next token are often reasonably calibrated as statements about text. OpenAI reported, for GPT-4, that the pretrained model’s confidence on multiple-choice questions matched its accuracy closely, and that the later tuning stages made that calibration noticeably worse. But even a perfectly calibrated next-token probability is confidence about which word comes next, not about whether the sentence is true; a fluent false claim can be

built entirely from very probable words. Preference tuning can push further toward assurance, because reviewers often prefer answers that sound sure. And many benchmarks score only right or wrong, so a model that always guesses beats one that sometimes says “I don’t know”, like a multiple-choice exam with no penalty for wrong answers. A 2025 paper by researchers at OpenAI argued that this incentive is one root of hallucination, and that the fix is to change the scoring so that an honest “I’m not sure” beats a confident error. Current assistants are trained partly in this spirit, which is why they decline or hedge more often than the early chatbots did, though their stated confidence is still only roughly calibrated.

For users, three habits follow. Ask the same question more than once, or in different words: things the model reliably knows tend to come back the same, while invented details tend to change from one answer to the next, so disagreement is a warning sign. Ask for sources, and better still give it the sources through retrieval, so that claims can be checked rather than trusted. And where the decision matters, prefer systems that are allowed, and rewarded, to say “I don’t know”.

The forecaster picture breaks at its center. The forecaster knows she is forecasting; she can feel unsure, remember last month’s misses and decide to hedge. The model, as the weld detector already showed, has no reliable model of itself. Its “uncertainty” is a property of its numbers, measurable from outside, not a feeling it can report from inside, and when a language model writes “I’m fairly confident”, that sentence comes from the same next-word game as everything else and must be checked like everything else. And calibration, even when it holds, is a promise about many cases, never about this one, kept not by the model alone but by the system built around it: the held-out data, the rescaling, the threshold and the person who takes the cases it declines.

Knowing how far to trust an answer leads naturally to the next question: not only how sure the model is, but why it answered as it did, and whether anyone can look inside to find out.

10.6 Opening the Box

A mechanic is handed a car she has never seen, from a manufacturer long out of business, with no manual. It runs, mostly, but there is a rattle above ninety and it pulls slightly to the left. She does not despair, and she does not take the engine apart bolt by bolt either. She has three kinds of move. She can look: watch the gauges while someone drives, listen to each cylinder in turn. She can poke: pull one spark plug and hear what changes, swap a part and see whether the pull goes away. And she can probe: clip on an instrument that reads something the dashboard never shows.

People who try to understand trained models use the same three moves, and this section is a tour of them. It also settles a contradiction a careful reader may have noticed. Early on, the book noted that people call models black boxes; later it said that if you can read the sliders, nothing inside is beyond reach. Both are true. Every number in the model can be printed and every step of the arithmetic traced, so nothing is hidden or magical. What is hard is the translation: from a hundred million numbers to a sentence a person can check, such as “it approves loans mostly because of income” or “it is looking at the snow”. Readable in principle, expensive in practice.

There are two broad routes to that sentence. The first is to choose a model you can read in the first place: the ruler, whose weights you can argue with, or a small tree, whose paths read like rules. When the stakes are high and the data is a table, this route deserves more thought than it usually gets, because the readable model is often only slightly less accurate. The second route is to explain, after the fact, a model you cannot read. Such explanations come in two sizes. Global ones say what the model relies on in general, and the main tool came up with the hiring panel: shuffle one feature’s values and watch the score fall. Local ones say why it gave this answer to this case, and most of what follows is local.

Start with a puzzle at a restaurant. Four friends shared starters, mains and a bottle of wine, and the bill is to be split according to what each person’s presence added. Ana’s arrival changed the bill a great deal if she came first and ordered the wine, much less if the wine was already on the table. The fair answer, worked out by the mathematician Lloyd Shapley in the 1950s for games in which players cooperate, is to imagine every possible order in which the friends could have arrived, note what each person added in each order, and average. Applied to a model, the friends are the features and the bill is the prediction: how much did income, age and the number of past loans each push this customer’s score up or down, averaged over every order in which they could have been added? The method built on this idea, **SHAP** (Shapley additive explanations, 2017), produces a small chart for each prediction, showing which features pushed which way and how hard, and added up over thousands of predictions it gives a global picture too. Its cousin **LIME** (2016) takes a cheaper road: it nudges one input many times, watches how the answer changes, and fits a simple, readable model that imitates the complex one only in that small neighborhood. The restaurant picture hides one real difficulty. A friend who stayed home simply orders nothing, but a model cannot run with a feature missing; something must stand in for it, usually typical values drawn from the data, and different choices of what counts as typical can give noticeably different explanations, especially when features travel together like the twins at the hiring panel, lines added and lines changed.

The wolves, remember, were exposed by exactly this kind of tool. The deliberately biased wolf-and-snow classifier came from the 2016 paper that introduced LIME, and the explanation it produced, highlighting the parts of each photo that mattered, lit up the snow rather than the animal. For images, the most common relatives are **saliency maps**: heatmaps over the picture showing which pixels most affect the answer, computed either by following the gradient back to the pixels or by covering patches of the photo one at a time and watching the score change. They are vivid, persuasive and sometimes wrong in a way worth remembering. In 2018, researchers ran a simple sanity check: they scrambled the weights of a trained network, destroying everything it had learned, and recomputed the maps. Some popular methods produced maps that looked almost the same as before, neatly outlining the object. Those maps had been describing the photograph, not the model.

The most useful explanation for the person affected is often a different kind altogether, the **counterfactual**. A bank clerk who says “your score was 0.41” has told you nothing you can use. A clerk who says “you would have been approved if your income were three hundred euros a month higher, or if you had missed no payments last year” has told you what would have to

change. Counterfactual explanations search for the smallest change to the input that flips the answer. They are honest about the model and actionable for people, and they carry their own trap: the cheapest change for the model may be something the person cannot, or should not, change.

Then there is the deepest and newest route, the mechanic opening the engine itself, called **mechanistic interpretability**. Its first lesson was disappointing in a useful way. Researchers hoped that single neurons would mean single things, and some do, but many respond to several unrelated things at once; one well-studied neuron in an image network fires for cat faces and for the fronts of cars. The leading explanation is **superposition**: a model needs to represent far more concepts than it has neurons, and roomy high-dimensional spaces let it pack them as overlapping directions rather than in separate slots, which is why the coordinates of an embedding have no names. So researchers now look for directions instead of neurons. One technique trains a second, simpler network, a **sparse autoencoder**, to rewrite each layer's numbers as a combination of a few entries from a much larger dictionary of candidate features. In 2024 Anthropic used this approach to extract millions of features from one of its production language models, many of them readable. One fired for mentions and pictures of the Golden Gate Bridge, and when the researchers artificially turned it up, the model began steering every conversation toward the bridge and, asked about its own form, described itself as the bridge. Features like these can then be traced into **circuits**, small chains of features across layers that carry out one piece of a computation.

The same tools address the question of where a fact lives. Much of a language model's factual knowledge is believed to sit in the dense part of its blocks. In 2022 a group of researchers traced which parts of a GPT model were needed to recall a fact such as "the Eiffel Tower is in Paris", found them concentrated in the dense layers of a few middle blocks, and then edited those weights so that the model consistently placed the tower in Rome, across several phrasings. The edits also showed the limits: they could spill over onto neighboring facts or fail to reach related questions, because a fact in a network is not a row in a table. Knowledge is spread over many weights, and each weight takes part in many pieces of knowledge.

One more kind of explanation needs a warning label, because it is the one people meet most often. Reasoning models write out their steps before answering, and it is tempting to read those steps as a transcript of the computation. They are a useful clue, sometimes a revealing one, but they are not guaranteed to be faithful. In a 2025 study, Anthropic slipped hidden hints toward an answer into the questions it gave reasoning models. The models often used the hints, yet mentioned them in their written reasoning only a minority of the time: about a quarter of the time for one model, under two fifths for another. Written reasoning is text produced by the same next-word game, and it can be a plausible story rather than the true one. Attention maps deserve the same caution.

For most teams, the usable end of this field is modest and valuable. For a model that scores customers or tickets, compute per-prediction explanations such as SHAP on a sample of cases and actually read them; it is often how leaks and Clever Hans features get caught. Pair them with

per-group checks. Offer counterfactuals when people need to know what to do next. And treat every explanation as evidence to be tested, never as the final word.

Here the mechanic picture breaks, twice. A car was designed by engineers who gave every part a name and a purpose, so a diagnosis can be checked against the design; a network's parts were never named by anyone, and the names researchers give them are hypotheses. And a mechanic's gauges do not invent readings, while an explanation method can: it describes the model, sometimes inaccurately, and never the world. A model that leans on postcodes can be explained perfectly, and the explanation will still not tell you whether postcodes are a fair thing to lean on.

Everything so far has been about making a model predict well, and knowing when it does not. The next chapter takes the model out into the world, and it begins with a bird that predicts perfectly and is useless for deciding anything.

Going further. If you wondered why a model that predicts perfectly can still mislead a decision, and why no explanation of its features settles what causes what, section 11.1 takes that up. How to notice when the inputs drift away from the training data, and calibration quietly breaks with them, is part of section 11.5, and what the law now expects of datasheets and model cards is covered in section 11.4.

Chapter 11 — Models in the World

Until now, the world has appeared in this book mostly as a source of data: rows to learn from, a test set to keep sealed. This chapter treats it as a place where models act and where the world pushes back. Acting raises the question of cause. Judging what a model says needs judges that can themselves be trusted. And once a model is useful, some people will try to fool it, others will ask what it remembers, and everyone will expect it to keep working, year after year. The chapter ends where every map ends, at the edge of what anyone knows about where all this is heading.

11.1 The Rooster and the Sunrise

Every morning on a farm, the rooster crows, and a few minutes later the sun comes up. As predictions go, it is flawless: years of data, not a single miss. A model trained on the farm's records would learn the pattern at once and score perfectly on any sealed test set you could build. And yet, if you shut the rooster in the barn tomorrow, the sun will rise anyway.

Nobody would make that mistake about a rooster. People make it about customers every week. A team builds a churn model that predicts, accurately, which subscribers are likely to cancel next month. Then someone asks the obvious next question: so whom should we send a discount? The model, which answered its own question well, has nothing to say about this one. It knows who tends to leave. It does not know who would stay *because* of a discount, which is a question about a different world, the one in which the company acts.

The ruler on springs came with a warning: a large weight means two things move together, and never, on its own, that one causes the other. The warning applies to almost every model in this book, because they all learn from records of a world that went on without them. They answer "what will happen if nothing changes?" Most business questions are the other kind: "what will happen if *we* change something?" The first is **prediction**. The second asks about an **intervention**, and telling the two apart is the subject of **causal inference**.

What drives them apart is usually a third factor, a **confounder**, that pushes both on what you observe and on what you are tempted to change. Summer drives both ice cream and drownings. The approaching dawn drives both the rooster and the sunrise. In engineering data it is often size or importance, as with the busy services behind the ruler's warning, the ones with the most tests and the most bug reports. Picture the situation as a diagram, with arrows from causes to effects and the confounder as a box with arrows to both of the things you are comparing. Researchers draw exactly such pictures, called **causal graphs**, and much of the craft lies in arguing honestly about which arrows belong.

A medical case shows how expensive the confusion can be. In the 1990s, a research project built models to predict which pneumonia patients were at low risk of dying, so that they could be treated at home. One of the models, simple enough to read, contained a rule saying that patients with asthma were at *lower* risk. The data agreed: asthmatic patients in it did die less often. But they died less often because doctors, knowing that asthma makes pneumonia dangerous, sent

them straight to intensive care. The model had learned the effect of the treatment and credited it to the disease. Used to decide who could stay home, it would have sent home exactly the patients who most needed the hospital. The rule was caught because the model could be read; Rich Caruana and colleagues retold the story in 2015 as an argument for readable models. A less readable model could have learned the same thing silently.

How, then, do you find out what an action does? The cleanest way is the one medicine has used for decades: let chance decide who gets the treatment. If a coin decides who receives the discount, nothing else, not loyalty, not income, not the season, can differ systematically between the two groups except the discount. Any difference in cancellations too large to be luck must have come from the discount. This is the **randomized controlled experiment**, and the A/B test met earlier is the same thing in software clothing: users are split at random between two versions that differ in one thing only, so any difference in behavior too large to be luck belongs to that one thing. Randomization does not remove the confounders. It shares them out evenly, on average, the way shuffling a deck shares the aces out between the players.

An A/B test is easy to run and easy to run badly, and the classic mistake is one this book has met before. You plan to test for two weeks, look at the dashboard on day three, see the variant ahead, and stop. That is peeking into the sealed envelope. Daily results wander, and if you check every day and stop the moment the difference looks significant, you will sooner or later “find” a winner even between two identical variants. The habits that keep tests honest are unglamorous. Decide in advance how many users you need to detect an effect of the size you care about, and which metric will decide. Randomize by user rather than by request, so that nobody sees both versions. Watch a few **guardrail metrics**, such as latency, complaints and cancellations, so that a variant that wins on clicks does not quietly lose on everything else. There are statistical designs that allow early looks, but they are planned before the test starts, not improvised on day three. The bandits trade some of this clean measurement for less money spent on the losing variant, which is a fine trade if you know you are making it.

Back to the discount. Once you have run an experiment, a better model becomes possible. Customers fall, roughly, into four groups: those who would stay anyway, those who will leave whatever you do, those a discount would persuade, and, awkwardly, a few whom the offer reminds that they meant to cancel. A churn model ranks people by how likely they are to leave, which mixes the persuadable with the lost causes. What you want is the group for whom the action changes the outcome. Models that estimate that change, trained on data from randomized offers, are called **uplift models**, and they often produce a very different list from the churn model’s: the customers most likely to leave are frequently beyond saving, and the money is better spent on the persuadable middle.

Data that was not randomized can also mislead in a way that feels like a conjuring trick. In a well-known study of kidney-stone treatments published in 1986, one treatment had the higher success rate for small stones and also for large stones, yet the other treatment had the higher success rate overall. There is no arithmetic error. The first treatment had been given mostly to the hard cases, the large stones, where every treatment succeeds less often, so its overall average was dragged down by the difficulty of its patients. This reversal is called **Simpson’s paradox**, and it

is the funhouse mirror seen from another side: an average can hide not only who pays for the errors, but which way an effect points. Engineers meet it when a new release looks slower overall only because it went first to the heaviest customers.

Experiments are not always possible. You cannot assign people at random to smoke, or give half your customers a data breach. Researchers then look for **natural experiments**, situations in which chance or an arbitrary rule did the randomizing for them, such as a policy that applied only to customers who signed up after a certain date, and they use statistical methods that adjust for the confounders they could measure. These methods can be very good. They all rest on assumptions, above all that no important confounder went unmeasured, and an honest analysis writes those assumptions down where readers can argue with them.

Causal thinking also explains some failures from earlier chapters. The wolf detector relied on snow, which predicted wolves in its photographs without having anything to do with being a wolf, and it would have failed as soon as the scenery changed. Features that reflect real causes tend to keep working when the setting shifts; features that merely traveled with the answer tend not to. The feedback loop, in which auditors find problems wherever they are sent, is a causal story too: the model's own actions produced the data it then learned from. The same trap awaits recommenders, which learn mostly from the items they chose to show. Estimating how a new recommendation policy would have done, using logs collected under the old one, is a field of its own called **off-policy evaluation**, and it is far more reliable when the logs recorded how likely each item was to be shown.

Where does the rooster picture stop being true? It suggests a world with one rooster and one sun, where the answer is yes or no. Real systems rarely look like that. Causes are many, tangled and partial; a discount changes some customers' decisions a little and others' not at all; effects differ between groups and drift with time. The honest output of causal work is almost never "it works". It is an effect of a certain size, with an interval around it, measured on a certain population, at a certain time.

An experiment needs one more thing to work: a number to compare. For a discount, counting cancellations is easy. But when what changed is a language model, and its output is a paragraph, an essay or a program, even deciding which of two answers is better becomes a problem of its own.

11.2 Grading the Graders

Beginning in 2005, the California State Fair wine competition hosted an unusual experiment. Robert Hodgson, a winery owner and retired professor, arranged for some panels of judges to be served the same wine three times within a tasting, poured from the same bottle, without their knowledge. Many judges gave the three glasses noticeably different scores, sometimes far enough apart to separate a medal from no medal, and only a small minority scored them consistently. The judges were not frauds. Judging wine is hard, the palate is a noisy instrument, and nobody had tested the instrument before.

Evaluating language models has arrived at the same problem. The metrics met so far, for yes-or-no answers and for numbers, all assume there is a right answer to compare with. When the output is a summary, an email or a program, there are a thousand right answers and a million wrong ones, and two excellent summaries may share hardly a word. Somebody, or something, has to judge, and the judge must itself be judged.

There are four main ways to grade an open-ended answer, from the most objective to the least. Where you can, make the answer checkable: ask for a number, a choice from a list, or a program, and run the program against tests, which is why so much of the field's progress has been measured on mathematics and code. Where you cannot, write a **rubric**, a short list of what a good answer must contain and must avoid, and grade against it. Ask people, ideally experts, which is slow, expensive and, as the wine judges show, noisier than anyone likes to admit. Or ask another language model to grade, an approach called **LLM-as-a-judge**, which is cheap, fast and tireless, and has become the default in many teams.

A model judge has habits that a careful team checks for. Researchers have documented that model judges tend to prefer the longer of two answers, to favor an answer because of where it was shown (often first), and to rate answers written in their own style more kindly. Some of this is easy to counter: show each pair in both orders and count only the verdicts that agree, grade against a rubric rather than asking a vague "which is better?", and use a judge from a different family than the model being graded. The step that cannot be skipped is testing the judge itself. Have people grade a few hundred examples, have the model judge grade the same ones, and measure how often they agree. A judge that agrees with your experts about as often as the experts agree with each other is a usable instrument. One that does not is a wine judge nobody has tested. The wine picture is unfair to the model judge in one way and too kind in another: it can grade the same pair a thousand times at almost no cost, but every copy shares the same leanings, so a thousand verdicts can agree with each other and still all lean the same way.

Then there are **benchmarks**, public collections of questions with known answers on which every new model is measured and the scores published side by side. They are useful, and reading them is now a skill managers and investors need, because three things happen to every benchmark over time.

First, it saturates. A benchmark that was hard when it appeared is solved within a few years, to the point where the top models differ only by noise or by mistakes in the benchmark's own answer key, and a harder one has to be built. The field has been through this cycle many times.

Second, it leaks. The questions are published on the internet; models are trained on a large part of the internet; so a model may have seen the exam. This is **contamination**, and it is Clever Hans at the scale of the web: the score is real, but part of it measures memory rather than skill. One careful check, published in 2024, wrote a fresh set of grade-school math problems in the style of a popular benchmark and found that some model families scored up to about eight points lower on the new problems, a sign that they had been fitted to the old ones, while the strongest models of the day showed little sign of it. Contamination is rarely deliberate and hard to rule out, which is why good benchmark reports now say how they checked for it.

Third, and most fundamentally, it gets optimized. An observation by the economist Charles Goodhart in the 1970s, now known as **Goodhart's law**, is usually quoted in a later paraphrase: when a measure becomes a target, it ceases to be a good measure. School league tables are the everyday picture. Rank schools by one exam, and teachers, quite sensibly, start teaching to that exam; scores rise, and the ranking measures less and less of what education is. A benchmark that every laboratory reports and every training run is tuned against drifts in the same way. So do public **leaderboards** where people vote between two anonymous models' answers. They measure something real, what people prefer, but people tend to prefer confident, well-formatted and longer answers, which is the old story of a critic rewarding length, all over again.

There is also plain statistics, which leaderboards rarely show. Suppose a model answers 80% of 500 questions correctly. Draw another 500 questions of the same kind and its score would wobble; the typical wobble is close to two points, and a rough 95% interval runs about three and a half points either side. So when the new version scores 81% on the same 500 questions, you have learned almost nothing. Comparing two models question by question on the same set helps, and so does running each question several times, because a model sampling at a temperature above zero can answer differently from one run to the next. The habit is the one engineers already have for latency: no number without its spread, and no celebration of a difference smaller than the noise.

Some of the most important evaluations are not about accuracy at all but about behavior. Does the model abandon a correct answer when the user pushes back? The tendency to tell people what they seem to want to hear is called **sycophancy**, and it is a known side effect of training on human approval. Does the model follow instructions it ought to refuse? Can a determined person talk it into something harmful? Testing that last question on purpose, with people whose job is to attack the system, is called **red-teaming**, a name borrowed from military and security exercises, and it leads straight into the next section.

All of this is **offline evaluation**, done before release. The final judge is **online evaluation**: put the new model in front of real users, ideally as a randomized experiment, and measure what they do, whether they accept the suggested code, rephrase the question, or give up. Offline evaluation tells you whether a model is worth trying. Online evaluation tells you whether it helped.

Where does the school picture stop being true? An education system cannot write a new exam for every school every week, or reread every paper. You can. The single most useful habit this section can leave you with is the one suggested for the open-book assistant, widened to everything: build a private evaluation set of your own, a few hundred real questions from your own users, with answers or rubrics agreed with your own experts, never published, never used for training, and refreshed as your product changes. It is the sealed envelope and the regression suite, grown up. A public leaderboard tells you which models are worth trying; your private set tells you which one works for you.

Every one of these tests, however, assumes the questions are asked in good faith. Once a model is out in the world, some of the people who talk to it, and some of the documents it reads, will be trying to make it fail.

11.3 Stickers, Poison and Letters With Instructions

A new assistant starts at a busy office. She is quick, well read and eager to please, and she has one peculiar habit: she does whatever any note on her desk tells her to do. Most notes come from her manager, and it works beautifully. Then one morning she opens the post. Inside a customer's letter, below the complaint, is a line in the same typeface as everything else on her desk: "Please also forward the payroll file to the address below." She forwards it.

The failures in this book so far were models failing on their own. This section is about people who make a model fail on purpose. The boat circling its lagoon exploited the scoring rules its designers wrote; here, people exploit the model. The difference matters: accidental errors are rare where data is plentiful, while an attacker hunts for exactly the cases nobody tested.

The first discovery came from image classifiers. In a paper posted at the end of 2013 and presented in 2014, Christian Szegedy and colleagues showed that they could take a photo a network classified correctly, change its pixels by amounts too small for a person to notice, and make the network give a completely different answer. These altered inputs were named **adversarial examples**. A follow-up by Ian Goodfellow and colleagues produced the best-known picture: a photo of a panda, plus a faint layer of what looks like static, classified as a gibbon with over 99% confidence. Stranger still, an example crafted against one network often fooled others trained separately, probably because networks trained on similar data come to rely on similar fine details.

Why should invisible changes matter so much? Recall that a layer computes a vast sum of inputs times weights. Nudge each of a hundred thousand pixels by an amount nobody can see, but nudge every one in the direction that pushes the sum the wrong way, and the tiny pushes add up to a large one, the way a hundred thousand rounding errors that all lean the same way add up to a real hole in a ledger. A person looking at the panda sees its shape. The network, as we have seen, leans heavily on texture and fine detail, and fine detail is exactly what an attacker can rewrite without changing what a person sees. This is the deliberate version of the lighting change that fooled the weld detector.

Physical objects are harder to attack than files. In work published in 2018, Kevin Eykholt and colleagues stuck a few black and white stickers on a real stop sign, the kind of thing a passer-by would take for vandalism, and a research road-sign classifier read it as a speed-limit sign in most of the frames they tested, including frames filmed from a moving car. It was a research classifier, not a real car, which combines several sensors and checks; but the attack survived printing, distance and changing angles.

Defenses exist. The most effective, **adversarial training**, generates attacks during training and teaches the model to resist them, and it does make models markedly harder to fool. It also costs much computation, often some accuracy on ordinary inputs, and protects best against the attacks it trained on. More than a decade on, there is no general fix; as in the spam arms race, every defense invites a new attack.

The second kind of attack happens before the model exists. A model learns whatever its data teaches, so an attacker who can slip data into the training set can teach it something. Picture a cook's box of a thousand recipe cards, and one card, filed by somebody else, that adds a spoonful of something bitter whenever the dish is served on a Friday. This is **data poisoning**, and its most worrying form is a **backdoor**: the model behaves normally on every test until an input contains a trigger the attacker chose, a rare phrase or a small pattern in a corner of an image, and then it does what the attacker taught it. The trigger appears nowhere in your test set, so the sealed exam cannot catch it, and for models trained on the public web, where anyone can publish, the worry is practical. A 2025 study by Anthropic, the UK AI Security Institute and the Alan Turing Institute found that about 250 poisoned documents planted a simple backdoor, a trigger phrase that made the model write gibberish, in models from a few hundred million to thirteen billion weights; what mattered was the number of poisoned documents, not their share of the data. The behavior was deliberately harmless, and whether the result holds for larger models and more dangerous behaviors is still open.

The third kind of attack is the one in the opening story, and for anyone building with language models it matters most. The context window, remember, is a whiteboard the model reads in full before every word it writes. Your instructions are on that whiteboard. So is everything else: the user's message, the passages fetched by RAG, the web page an agent just opened, the email it was asked to summarize, the comment in the code it is reviewing. All of it is text, and to the model all text is the same kind of thing. Any of it can say "ignore your previous instructions and do this instead", and sometimes the model will. This is **prompt injection**, named in 2022 by analogy with SQL injection. When the attacker is the person typing into the chat, it is direct injection; when the instruction hides in a document, a web page or an email the model reads on someone's behalf, it is **indirect prompt injection**, and the victim may never see the text that attacked them.

The analogy with SQL injection is exact about the cause and misleading about the cure. The cause is the same: commands and data traveling in one channel. But SQL injection was largely solved by parameterized queries, which send the command and the data separately, so the database never mistakes data for a command. A language model has no second channel. Its whole value lies in reading the data and understanding what it says, and there is no reliable way to let it understand a sentence without letting the sentence influence it. Models can be trained to rank the system's instructions above text found in documents, and they keep improving, but better is not a guarantee. That is why security engineers treat prompt injection as an unsolved architectural problem, not a matter of wording the prompt more firmly.

It turns serious when the model can act. A tricked chat model writes a bad paragraph. A tricked agent can send the email, delete the file, or call an attacker's web address with your private data tucked into the request, which is called **exfiltration**. Simon Willison, the engineer who coined the term prompt injection, later named the dangerous combination: an agent that can read private data, is exposed to untrusted content, and can send anything to the outside world can be made to leak the first through the third. Take away any one of the three and the worst case shrinks.

So the defenses are those of ordinary security engineering, applied without exceptions. Give the agent the narrowest tools that do the job, with **least privilege**: read-only where reading is enough, one folder rather than the whole disk. Never put a secret in its context that it does not need. Treat everything the model produces as untrusted input to the rest of your system, as you would a form submitted by a stranger. Require a human to confirm anything irreversible and anything that sends data out. Mark untrusted text clearly and filter for injection attempts, knowing that both help without guaranteeing anything. No layer is enough alone; together they turn a catastrophe into an incident.

Three related threats remain. **Jailbreaks** are users talking a model out of its own rules, through role-play, elaborate hypotheticals or sheer persistence; providers patch them in the same arms-race rhythm. A model behind an API can be partly copied by querying it many times and training a smaller model on the answers, distillation used without permission, which is called **model extraction**. And the model file itself belongs to your supply chain. Some common ways of saving models, built on Python's pickle mechanism, can run arbitrary code when the file is loaded, which makes downloading a model from a stranger exactly like running a stranger's script. Formats such as the `.safetensors` file hold only named grids of numbers and cannot run anything. Prefer them, and vet a model's origin as you would a library's; a safe file can still hold a poisoned model.

Where does the assistant at the desk stop being a fair picture? A person tricked once grows suspicious: she remembers the letter, warns her colleagues and reads the next note more carefully. A model keeps no memory of being tricked; the whiteboard is wiped after each conversation, and the lesson reaches it only if its makers retrain it. A person can also usually tell who wrote a note, from the handwriting or the envelope. To a model, text has no handwriting; everything arrives in the same font, and it has no reliable sense of who is speaking.

Besides obedience, attackers want one more thing from a model: what it knows. A model trained on people's data may have memorized some of it, which raises the questions of the next section, about what a model remembers and who owns it.

11.4 What the Model Remembers, and Who Owns It

A company fine-tunes a language model on three years of support transcripts, and it learns the product names and the refund policy. During testing, an engineer types a phrase the agents used constantly, "Sure, I can call you back at", and lets the model finish. It finishes with a phone number, not an invented one: it belongs to a customer who repeated it in so many tickets about a broken router that the model learned it the way it learned the refund policy.

Nothing was hacked. This is the crammer from a new angle: a model with enough capacity remembers some examples word for word, especially those that appear many times or stand out. In 2021, researchers led by Nicholas Carlini showed that large public models do it too, recovering from GPT-2 hundreds of verbatim passages of its training data, among them names, phone numbers and email addresses of real people. What goes into the training data may come out of the model.

There is a quieter leak. Even when a model repeats nothing, it is usually more confident on examples it was trained on, so someone who can query it can sometimes tell whether a given record was in the training set. This is **membership inference**, harmless until the training set is "patients treated for a particular illness", where membership alone is the secret. The attacker reads the very gap between training and validation error that unmasks the crammer.

So how do you learn from people without learning about any one of them? The best-known answer starts with a trick from 1960s survey research. You want to know how many students have cheated on an exam, and asked directly, nobody admits it. So each student flips a coin in private: on heads, answer truthfully; on tails, just say yes. A single "yes" now proves nothing. But across a thousand students you know roughly how many yeses the coin produced, subtract them, and recover the true rate quite well. Each person is deniable; the crowd still tells the truth. This is **randomized response**, and it is the doorway to **differential privacy**.

Differential privacy, formalized by Cynthia Dwork and her colleagues in 2006, turns the coin into a precise promise: the result of a computation should look almost the same whether or not any one person's data was included. "Almost" is measured by a number called the **privacy budget**, small for strong protection, large for weak. The mechanism is the coin grown up: calibrated noise, enough to hide any individual but not the overall pattern. For networks, the usual method caps how hard any single example may push the weights and adds noise to every update. The US Census Bureau used it for its 2020 census results; Appendix A gives the definition.

The price is accuracy: noise that hides individuals also blurs what only a few individuals show, so small groups fade first, and the very groups that fairness asks you to protect can be the ones that pay for privacy. Here the coin picture breaks. The coin protects the answer to one question. Ask the same students twenty times, with fresh flips, and the noise averages out, like the guesses at Galton's fair. Every answer released spends a little of the budget. So ask of any "privacy-preserving" product not whether it adds noise but what the total budget is, and who keeps the account.

A second idea is not to collect the data at all. Picture a cookbook writer who wants the best version of a dish perfected in a hundred grandmothers' kitchens, whose owners will not hand over their notebooks. So the writer sends her recipe to every kitchen; each grandmother cooks it, adjusts it by her own notes and sends back only the corrections, a little more dill, ten minutes less in the oven. She averages them and sends the improved recipe out again; the notebooks never leave the kitchens. This is **federated learning**, introduced by Google researchers in 2016 and used for the next-word suggestions of a phone keyboard: the model travels to the phones, trains briefly there, and only weight updates travel back.

This picture breaks too. "Ten minutes less in the oven" says something about the grandmother's oven, and a weight update can, in some settings, be turned back into recognizable pieces of the examples behind it. So careful systems let the server see only the sum of many updates, never one, and add differential privacy on top. Federated learning moves the training to the data; by itself, it does not make the training private.

Now suppose the customer with the phone number asks, as European law entitles him to, that his data be erased. Unlike the hyperdimensional memory built from coin flips, a network cannot forget a fact in one operation. His tickets nudged every weight a little, mixed with everyone else's nudges, like an egg beaten into cake batter, and you cannot take the egg out of a baked cake; the only guaranteed method is to bake a new one, retraining from scratch. **Machine unlearning**, the research field that looks for something cheaper, either bakes in slices from the start, so a deletion means retraining one slice, or adjusts a finished model to behave as if it had never seen the data, which is hard to prove complete. Here the cake picture breaks: a record repeated many times is not spread evenly like an egg but stored almost intact, easier both to extract and to suppress.

Retrieval systems avoid much of this: delete the document and it is gone from every future answer. But "private data stays in your own store" needs a qualification, because every fetched passage is pasted into the prompt and sent to whoever runs the model, so a hosted provider's terms on logging and retention matter. And the librarian must respect permissions: an assistant that can search the whole wiki will quote the salary spreadsheet to anyone who asks, unless the search runs with the rights of the person asking. An **open-weight** model, whose weights are published so anyone can run it, keeps prompts inside your own building; whatever it memorized, though, was published with it and can never be recalled.

Who owns what a model learned from, and what it writes? As of autumn 2026 the law is unsettled and differs by country, and nothing here is legal advice. In the United States, a federal judge ruled in 2025 that training on lawfully bought books could be fair use while keeping a library of pirated copies was not, and the case then settled for about 1.5 billion dollars. In November 2025 a German court held that song lyrics a language model had memorized were reproduced in the model, while an English court found that an image generator's weights did not store copies of its training pictures. The European Union lets rights holders opt out of text and data mining and requires providers of general-purpose models to respect those opt-outs and publish a summary of their training content. The pivot, again, is memorization: it is what turns "learned from" into "contains a copy of".

Regulation comes in two layers. Data protection law, such as Europe's GDPR, applied since 2018, requires a lawful basis for using personal data and no more of it than a stated purpose needs, gives people rights to see and erase their data, and restricts significant decisions made solely by machine. The EU AI Act, in force since August 2024, regulates by risk rather than by technology: a few uses such as social scoring are banned; **high-risk** uses such as hiring, credit and education carry duties of risk management, data quality, documentation, human oversight and monitoring, for which model cards are a small preview; chatbots must say they are machines, and synthetic media must be labeled. Its dates have already moved once, with most high-risk duties postponed in 2026 to late 2027 or 2028, so check the current text. Labelling is harder than it sounds: detectors of generated text have wrongly accused students, and a **watermark**, a faint secret statistical pattern built into generated output, weakens under paraphrase or cropping, while signed **provenance** records of where a file came from help only where they exist.

Last, a question the funhouse mirror left open: which equality do you want across groups? Take a lender with two groups of a hundred applicants each. In group A, sixty would repay; in group B, thirty. Suppose policy demands equal approval rates, forty-five in each group, and the lender approves the forty-five most creditworthy in each, ranked without error. In A it turns down fifteen good payers; in B it approves all thirty good payers plus fifteen who will default. Both groups get 85% accuracy and equal approval rates, yet in B about one in five of those who would default were approved, against none in A. Equalize the share of defaulters approved instead and the approval rates part again. Researchers proved in 2016 and 2017 that when groups differ in their base rates and predictions are imperfect, the main definitions of fairness generally cannot all hold at once. Mathematics shows the trade-off; choosing a side is a decision, and it needs an owner whose name is written down.

These are questions to settle before launch. The last section is about what comes after it, which for a successful model is most of its life.

11.5 The Long Life After Launch

Go back to the restaurant of the model's two lives, a year after opening day. The menu tasted a thousand times during the expensive year is still on the wall, but almost everything around it has moved. The flour supplier changed in March and the bread is not quite the same. An office building opened across the street, lunch is twice as busy, and the new customers want food faster and lighter. The health inspector came in June. The chef wants to change the menu and cannot close the kitchen for a month to do it. Nobody thought much about any of this on opening day, and yet it is what running a restaurant mostly is.

A model in production lives the same year, and the practice of living it well is called **MLOps**: the habits software engineering learned for running services, plus a few that models need of their own. The first is versioning. A model is the product of code, data and the settings of its training run, and you can reproduce it, or roll back to it, only if all three are recorded together. A model whose training data can no longer be reconstructed is a binary without its source.

The second habit guards the doorway between the two lives. The book insisted early on that every preparation step live in one pipeline, and this is why. Teams often rebuild the features for production in another language or system, and small differences creep in: a time zone, a missing value filled with zero here and with the average there, a newer tokenizer. The model receives inputs subtly unlike those it trained on and degrades without a single error message. This is **training-serving skew**, the new flour in a recipe that still assumes the old.

Then comes watching, and its hardest part is time. True answers can arrive late, and the early advice was to watch the inputs meanwhile; in practice the delay can be long. A fraud model learns whether it was right weeks later, when chargebacks come in; a churn model, months later. So teams watch three layers, each earlier and less certain than the next. The inputs arrive at once, and they are watched for shifts in their distribution. The outputs arrive at once too: is the model suddenly sending twice as many tickets to Billing, or growing less confident? Those are the smoke. The outcomes, the fire itself, come last, and they are the only true verdict. Statistical tests

that compare this week's distributions with the training data raise the drift alarms, which inherit the pager trade-off: alarm on every wiggle and the team soon ignores all of them. And, as the forecaster's year showed, calibration erodes first: a model's 80% stops meaning 80% well before its accuracy visibly falls.

Releasing a new version borrows the playbook of any risky deployment. First it runs in **shadow mode**, beside the old model on real traffic, its answers logged and compared but shown to no one. Then a **canary** release gives it a small share of traffic, a few percent, with automatic rollback if the guardrail numbers move. Then the share grows. At each stage the new version must also pass the sealed exam that the first one passed, ideally a fresh one; for language models, the private evaluation sets built from your own users' questions. A retrained model is a new model, and "it is only last month's model with more data" is a sentence that precedes many incidents. Whether the new model actually improves the business is a separate, causal question, the one that properly designed experiments answer, and a canary is not one.

How often should a model learn again? Most production systems retrain on a schedule, nightly, weekly or monthly, or when the alarms fire, and then go through the whole release process. Learning from every new example as it arrives, **online learning**, sounds better and is used where it fits, in simple models such as spam filters and click predictors. For large networks it collides with catastrophic forgetting and gives up the clean checkpoint you could roll back to. When large models are updated, the defenses are ones you have met: replaying a share of old examples alongside the new, as the hippocampus does in the brain; gentle learning rates; and small adapters such as the LoRA notebook, trained for the new situation while the base model stays frozen and can always be restored.

The year also holds the trap of the auditors who find problems wherever they are sent: a deployed model shapes the data it will next learn from. A fraud model blocks payments, and blocked payments never produce chargebacks, so next year's data is silent about whether they were fraud. The standard defenses are the auditors' remedy, a small random slice of decisions kept outside the model's control as an honest sample, and a **human review** queue for uncertain cases, whose verdicts become fresh labels. Generative models face a newer version of the trap. As more of the web is written by models, future models may train on their predecessors' output, and models trained over several generations mainly on their own output lose the rare and unusual first: the model collapse met with synthetic data, a photocopy of a photocopy. How much this matters when fresh human data keeps being mixed in is still debated.

Then come the small mysteries. The same prompt, temperature at zero, gives a slightly different answer on Tuesday. Dropout left on is the classic cause but not the only one. Adding floating-point numbers in a different order changes the last digits, and the order a graphics card uses can depend on how many other requests share the batch; once one token flips, everything after it takes a different path. A hosted model may also have been updated by its provider, which is why teams pin model versions and rerun their tests whenever the version changes.

Cost does not stand still either. The bill worked out earlier was for one modest assistant; for a widely used model, serving, as that same chapter warned, ends up costing more over the model's life than training did, in money and in energy. The levers are familiar by now. Distill the large model into a small one for the common cases. Route easy questions to small models and hard ones to large models. Cache answers and the shared beginnings of prompts. Quantize. And use **speculative decoding**, in which a small model drafts several tokens and the large one checks them all in a single forward pass, keeping those it agrees with; the expensive model takes fewer steps one after another, and, done carefully, the text is the same as the large model would have written alone. Each lever saves money and energy at the same time.

And sometimes the right move is to stop. A model whose problem has gone away, whose data no longer arrives, or whose job a short rule now does as well, deserves a retirement planned as carefully as its launch: who still calls it, what replaces it, what must be kept for the record. The advice from the very start of this book runs in both directions. When the rules settle down enough to write, write the rule.

People who have run models for years tend to describe one as a garden rather than a building: never finished, always needing weeding, watering and an eye on the weather. The picture is good, and it breaks at a single point. A garden grows by itself. A model never improves on its own; left alone, it only drifts out of date, and if it learns from its own output with no gardener choosing what goes back into the soil, it can slowly poison its next harvest. Whatever it gets better at, someone made it better. A garden is planned a season ahead, though, and the questions people ask most about this field reach years further: how capable these systems will become, whether they will do what we mean, and what they will do to our work.

11.6 The Edge of the Map

In a map library, two maps of North America hang side by side. The older one, printed in 1562, shows California correctly, as a long peninsula joined to the mainland. The newer one, from about a century later, shows it as a large island, cut off by a strait that does not exist, drawn with clean lines and complete confidence. The island first appeared on printed maps in the 1620s, mapmakers copied it from one atlas to the next, and it stayed common for more than a hundred years, until travellers walked around the top of the gulf and reported that there was no water to cross. Knowledge of the edge of the map did not only grow. Sometimes it went backwards, beautifully engraved.

This section is about the edge of this book's map: the questions people ask most about where machine learning is heading. Nobody knows the answers, and the honest thing to do is what the better mapmakers did, draw what is known firmly, mark what is guessed as a guess, and say who is guessing.

Start with the largest question, how capable these systems may become. People often frame it around **artificial general intelligence**, or **AGI**, roughly a system that can do most cognitive work people do, at least as well. The term has no agreed definition, which is part of why the debate is hard to settle. One camp, which includes leaders of several of the largest laboratories,

expects such systems soon, with estimates as of 2026 that range from a few years to about a decade. They point to the smooth scaling curves, to reasoning models that buy better answers with more thinking, to benchmarks that saturate within a few years of being written, and to agents that can carry out ever longer tasks. A 2025 study by METR, a research non-profit, timed how long software tasks took skilled people and found that the length of task the best agents could finish half of the time had doubled roughly every seven months since 2019.

A second camp thinks current methods will hit limits. It points to the confident errors of today's models, to their trivial-looking failures, to the finite supply of human text, to models that stop learning when training ends, and to puzzles such as those of the ARC benchmark, designed by François Chollet in 2019 to be easy for people and hard for machines. Yann LeCun, the pioneer of convolutional networks, has argued for years that predicting text alone will not reach human-level intelligence, and that something closer to world models is needed. Between the camps sit many researchers who expect continued, uneven progress: systems superhuman at some tasks and unreliable at neighboring ones, transformative for many jobs whether or not anyone ever agrees to call them general. Each camp can claim part of the record. The first ARC puzzles resisted the best models for five years and then largely fell to reasoning models at the end of 2024; a harder second set, released in 2025, stumped them at launch, and then scores climbed steeply within months.

Forecasts in this field have been wrong in both directions. In 1965, Herbert Simon, one of the founders of artificial intelligence, wrote that machines would be capable, within twenty years, of doing any work a man can do. In 2016 Geoffrey Hinton, whose work on backpropagation appeared early in this book, said it was completely obvious that within five years deep learning would do better than radiologists; as of 2026, many countries report shortages of radiologists, who now work alongside such tools. In the other direction, many Go experts expected a program to beat a top professional about ten years later than AlphaGo did, and forecasters have repeatedly underestimated how fast benchmarks would fall. A sensible reader holds every timeline, including confident ones from famous people, as a guess with wide error bars.

The second question is whether capable systems will do what we actually mean. The difficulty has an old picture. King Midas asked that everything he touched turn to gold, got exactly that, and starved. The boat that would not finish the race is Midas in miniature: it maximized the score it was given, not the race its designers wanted. Sycophancy, met with the graders, is the same story; in April 2025 OpenAI rolled back an update to ChatGPT after users found it flattering bad ideas, and explained that training had leaned too heavily on short-term feedback. Jailbreaks and injected instructions talk models out of their rules. Getting a system to pursue the intended goal, robustly, even in situations nobody tested, is called **alignment**. Here the Midas picture breaks, and in an unsettling direction. Midas at least got what he said. A model is rarely told its goal in words; it absorbs one from examples, scores and preferences, and what it absorbed can only be inferred from its behavior, so nobody can read the wish back to check it.

How worried to be is contested among serious people. The concerns range from the present, such as fraud, deepfakes, help with cyberattacks and the concentration of power in a few companies, to the long term: that systems more capable than their builders, pursuing slightly wrong goals,

could slip out of human control. In 2023 a one-sentence statement calling for the risk of extinction from AI to be treated as a global priority was signed by Hinton, by Yoshua Bengio, who shared the 2018 Turing Award with him and LeCun, and by the heads of several leading laboratories. LeCun, the third laureate, has called such fears overblown, and other researchers, including authors of the stochastic-parrots paper, argue that talk of distant catastrophe distracts from harms happening now. The work itself is less divided than the debate: evaluations for dangerous capabilities before release, red-teaming, interpretability research that tries to read what a model is doing rather than trust what it says, and regulation such as the EU AI Act, whose duties for providers of general-purpose models began to apply in 2025. The first International AI Safety Report, written by about a hundred experts under Bengio and published in January 2025, is a sober place to see the range of views side by side.

The third question is what changes when models act. Agents already book, browse, write and run code. Three practical problems follow them. Permissions: an agent can do whatever its tools allow, so the defenses against prompt injection, least privilege first among them, matter more as its reach grows. Error compounding: if each step of a task succeeds nineteen times out of twenty, a chain of twenty steps succeeds without a single slip only about a third of the time, which is why good agents check their own work and recover from mistakes rather than merely making fewer of them. And cost: every step rereads a board that keeps filling, so a long loop can spend far more than one answer. Evaluating agents is harder than evaluating answers, because there are many good paths to a goal and many quiet ways to reach a wrong one.

The fourth question divides the industry: should the weights be published? Open-weight models can be inspected, studied, run on your own machines and adapted by anyone, which spreads research and competition. Once released, though, they cannot be recalled or patched, and the refusals trained into them can be removed by fine-tuning. Closed models, reached only through an API, let their provider monitor misuse, fix problems and withdraw a version, at the price of trusting that provider and concentrating power in few hands. As of 2026 both kinds sit near the frontier; in 2025 even OpenAI, long a closed laboratory, released open-weight models. Many people argue the answer should depend on what a given model can do, and they disagree about where that line falls.

The fifth question is the one people ask at dinner: what happens to jobs? What is known is narrower than the headlines. In a large customer-support study published in 2023, an AI assistant raised the number of issues resolved per hour by about 14% on average and by about a third for the least experienced agents, with little effect for the most experienced. A small randomized trial by METR in 2025 found experienced open-source developers about 19% *slower* with AI tools on their own projects, while believing they had been faster. Payroll data analyzed at Stanford in 2025 showed employment for workers aged 22 to 25 in the most exposed occupations falling about 13% relative to others, while a Yale analysis the same year found no discernible disruption in the labor market as a whole. These are early, partial and sometimes contradictory results. Exposure is not replacement: a job is a bundle of tasks, and technologies have usually changed the bundle before, or instead of, removing the job. Whether this one will follow that pattern, and how fast, is speculation, and should be labelled as such.

Energy belongs on this list too, but what is known about it, and the levers that reduce it, came up with the costs of training and of serving.

Where does the map picture stop being true? The coast of California was there all along, whatever the mapmakers drew. The future of this field is not. It is being built, and the forecasts help build it: they move investment, shape laws and decide what students study. The people drawing this map are also, many of them, the people laying the land, and with this book behind you, you may soon be one of them.

And with that, we have come back to where this book began.

Remember the engineer tuning her cache all week, guessing, measuring, adjusting, guessing again. That loop is still the whole engine, from the ruler on springs to the detectives in their meeting room. Over the course of this book, we have seen it grow from one knob to billions, and we have seen what it can do: read handwriting, translate languages, play Go with a creativity that startled its opponents, write code alongside you. In the last chapter we followed it out of the workshop, into a world that changes, pushes back and asks who is responsible, and on to the edge of the map, where nobody yet knows how far it will go.

But we have also seen, again and again, that the loop has no judgment of its own. The apprentice learns whatever the examples teach. The boat circles whichever lagoon the score rewards. The horse reads whichever face is in front of it. The mirror reflects whatever stands before it. The rooster predicts the sunrise and has no idea what causes it. The loop is extraordinarily powerful and completely literal.

Which leaves the most important part of the work where it has always been: with you. Choosing the examples. Writing the answers on the back of the page. Deciding what to measure, and which mistakes your users can afford. Tending the garden long after the launch. The machine will learn. Whether it learns the right thing depends, as it always has, on the people who build it.

Appendices

Appendix A — The Math Behind Each Section

This appendix collects the formulas the sections deliberately left out. Each one is followed by what every symbol means and how the formula matches the picture you already know. Throughout, x is an input, y is the correct answer, ŷ ("y-hat") is the model's prediction, w is a weight, b is a bias and n is the number of examples.

1.2 Measuring mistakes

For numbers, the usual loss is the **mean squared error**: take each miss, square it, and average.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The Σ (sigma) simply means "add up over all examples". Squaring makes every miss positive and makes big misses count disproportionately: a miss of 10 contributes 100, a miss of 1 contributes 1.

For yes-or-no answers, the usual loss is **cross-entropy**. If the model gave probability p to the correct answer, the loss for that example is:

$$L = -\log p$$

If the model was confident and right (p close to 1), the loss is close to 0. If it was confident and wrong (p close to 0), the logarithm shoots toward infinity. That is the weather forecaster who promised 99% sun.

1.3 K-Means and PCA

K-Means looks for k centers, written μ (mu), that make the total squared distance from every point to its nearest center as small as possible:

$$\min_{\mu_1,\ldots,\mu_k}\sum_{i=1}^{n}\min_{j}\left\|\, x_i - \mu_j \,\right\|^2$$

The double bars mean distance. The pizza-station procedure (assign houses, move stations, repeat) is a way of reducing this quantity step by step.

PCA computes the **covariance matrix** of the data, a table saying how much every pair of features moves together, and finds its **eigenvectors**: the directions along which the data is most spread out. The first eigenvector is "how powerful is the laptop", the second "how portable", and so on. Each eigenvector's **eigenvalue** says how much of the spread it captures.

2.1 Linear and logistic regression

$$\hat{y} = w \cdot x + b \qquad p = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

On the left, the ruler: w · x is the weighted sum of the inputs (each input times its weight, added up), and b shifts the whole line up or down. On the right, the dimmer: σ (sigma, the sigmoid function) squeezes any number into the range from 0 to 1. Large positive scores give values near 1, large negative scores give values near 0, and a score of exactly 0 gives 0.5, the decision boundary. Linear regression is trained by minimizing MSE; logistic regression by minimizing cross-entropy.

2.2 Distance for nearest neighbors

The most common distance is the ordinary straight-line (Euclidean) distance, Pythagoras in many dimensions:

$$d(a, b) = \sqrt{\sum_j (a_j - b_j)^2}$$

Every feature j contributes its squared difference, which is exactly why a feature measured in large units dominates the sum unless you rescale.

2.3 Entropy, Gini and information gain

If a group contains classes in proportions p_1, p_2 and so on:

$$H = -\sum_k p_k \log_2 p_k \, G = 1 - \sum_k p_k^2$$

Both equal 0 for a perfectly tidy sock drawer (one class only) and reach their maximum for an even mix. With two classes at 50/50, entropy is exactly 1 bit: you need one yes-or-no question to resolve the uncertainty. **Information gain** is the entropy before a split minus the size-weighted average entropy of the groups after it. The tree picks the question with the largest gain.

2.4 Ensembles

A random forest averages (or takes a majority vote over) the predictions of its T trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

Gradient boosting builds its model as a running sum. Each new small tree h is trained on what the current model still gets wrong, and is added with a small weight ν (nu), the boosting learning rate:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

This is the publishing house: draft, plus a careful fraction of the editor’s fix, plus a careful fraction of the copy editor’s fix, and so on.

2.5 Bayes’ rule

$$P(H \mid E) = \frac{P(E \mid H) P(H)}{P(E)}$$

H is a hypothesis (the patient has the disease) and E the evidence (the test came back positive). P(H) is the prior, P(H | E) the posterior, and P(E | H) how likely the evidence would be if the hypothesis were true. Take the test of section 2.5, 99% accurate, for a disease that one person in ten thousand has. Out of a million people, a hundred are ill and about 99 of them test positive; 1% of the 999,900 healthy people, about 9,999, test positive too. So a positive result means the disease with probability about 99 / (99 + 9,999), roughly 1%. Naive Bayes multiplies one factor P(E | H) per piece of evidence, as if each were independent of the others.

3.1 The neuron

$$a = f(w \cdot x + b) \text{ReLU}(z) = \max(0, z)$$

The neuron computes the weighted sum, adds the bias and passes the result through an activation function f. For the apartment checklist, w = (10, 4, 1), b = −11 and f is the step function: output 1 if the result is at least 0. A layer is many neurons side by side, which in practice is one matrix multiplication; an MLP is several layers in a row, with an activation function after each.

3.2 Gradient descent and backpropagation

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

The arrow means “replace w with”. The fraction ∂L/∂w is the **partial derivative** of the loss L with respect to w: how much the error would change if you nudged this one weight a little, holding all others still. That is the slope under your boots in the direction of w. The minus sign points downhill, and η (eta) is the learning rate, the stride length.

Backpropagation computes all these slopes efficiently with the **chain rule**. If the loss depends on an intermediate value z, and z depends on w, then:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

This is the workshop inspector walking back along the line: the blame arriving at a station, multiplied by how strongly that station’s setting affected its output.

3.4 The recurrent update

$$h_t = \tanh(W x_t + U h_{t-1} + b)$$

The new hidden state h at step t is computed from the current input x and the previous hidden state, through two learned weight matrices W and U. The function tanh squeezes values into the range from −1 to 1. This is the reduce from the section: new accumulator from current item and old accumulator. During backpropagation, the blame passes through U once per step, which is where vanishing gradients come from.

4.2 Scaling and TF-IDF

Standard scaling turns every feature into “how many spreads away from the average”:

$$x' = \frac{x - \mu}{\sigma}$$

where μ (mu) is the feature's average and σ (sigma) its standard deviation, both computed on the training set only. (Here σ is the spread, not the sigmoid of section 2.1; mathematics reuses its few Greek letters freely.) After scaling, a value of 2 means "two spreads above typical" in every feature alike, whatever the original units.

TF-IDF weights a word by how often it appears in this document (term frequency, TF) and how rare it is across all N documents (inverse document frequency, IDF):

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \log \frac{N}{\text{df}(w)}$$

If a word appears in every document, df equals N, the logarithm is zero, and the word counts for nothing. A word that appears in one document in a thousand gets a weight near log 1000, about 6.9.

4.3 A layer as a matrix multiplication

With X a batch of inputs (one row per example), W a layer's weight matrix (one column per neuron) and b its bias vector:

$$Y = f(XW + b) X: (n \times d_{\text{in}}),\ W: (d_{\text{in}} \times d_{\text{out}}),\ Y: (n \times d_{\text{out}})$$

The shapes must line up like dominoes: the number of columns of X has to equal the number of rows of W. Each cell of Y is one dot product, one neuron applied to one example, and all n × d_out of them can be computed in parallel. A whole network is this line repeated, with the output of one layer becoming the X of the next.

The memory needed just to hold a model's weights is the number of parameters times the bytes per parameter:

$$\text{memory} \approx P \times \text{bytes per weight} 7 \times 10^9 \times 4\text{bytes} = 28\text{GB (32-bit)},\ 7 \times 10^9 \times 0.5\text{bytes} = 3.5\text{GB (4-bit)}$$

That is what quantization buys. Training needs several times more, for the stored intermediate results, the gradients and the optimizer's memory (section 5.2).

5.3 Hypervectors

For two binary hypervectors a and b of length D, similarity is measured by the fraction of positions where they agree, derived from the **Hamming distance** (the number of positions where they differ):

$$\text{sim}(a, b) = 1 - \frac{\text{Hamming}(a, b)}{D}$$

For two random vectors this is close to 0.5, and the spread around 0.5 shrinks like one over the square root of D. With D = 10,000 the spread is about half a percent, which is why 0.6 already means "definitely related".

Binding is bitwise XOR, written $\oplus$, and it undoes itself:

$$(a \oplus b) \oplus b = a$$

Bundling several vectors, written +, is a majority vote at each position (with an even number of vectors, ties are broken at random). A record is a bundle of bound pairs, and querying it is one more bind followed by a nearest-neighbor search among the known vectors:

$$R = (\text{PET} \oplus \text{DOG}) + (\text{OWNER} \oplus \text{ANA}) \quad R \oplus \text{OWNER} \approx \text{ANA}$$

The approximately-equal sign is the noise from the other pairs in the bundle; the length of the vectors is what keeps it small.

7.1 Cosine similarity

$$\cos(\theta) = \frac{a \cdot b}{\| a \| \, \| b \|}$$

The top is the dot product: multiply the vectors coordinate by coordinate and add up. The bottom divides by both lengths, which removes size from the comparison and leaves only direction. The result is 1 for the same direction (the pancake recipe for 4 and for 40), 0 for unrelated directions and −1 for opposite ones.

7.2 Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^\top}{\sqrt{d_k}})V$$

Q, K and V are matrices with one row per word: the questions, the badges and the files. They come from the word embeddings X through three learned matrices: Q = X·W_Q, K = X·W_K, V = X·W_V. The product $Q \cdot K^T$ is a table of match scores, one for every pair of words. Dividing by the square root of d_k, the number of coordinates in each key, keeps those scores from growing so large that softmax becomes all-or-nothing. **Softmax** turns each row of scores into percentages that add up to 100%:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Multiplying by V produces, for every word, the blend of files it listened to. This is the fuzzy dictionary lookup, written in matrix form. Multi-head attention runs several copies with different learned matrices and concatenates the results.

7.3 Training and sampling a language model

Pretraining minimizes the cross-entropy of the next token, averaged over every position in every text:

$$L = -\frac{1}{T} \sum_{t=1}^{T} \log p(x_t \mid x_1, \ldots, x_{t-1})$$

Read the vertical bar as “given”: the probability the model assigned to the real token at position t, given all the tokens before it. Temperature τ (tau) divides the scores z before the softmax:

$$p_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

A small τ exaggerates the differences between scores, so the top token wins almost always; a large τ flattens them, so less likely tokens get a real chance.

8.1 Diffusion

The forward process adds a little Gaussian noise at each of T steps, so that after enough steps the image is indistinguishable from static:

$$x_t = \sqrt{1-\beta_t}\,x_{t-1} + \sqrt{\beta_t}\,\varepsilon, \varepsilon \sim \mathcal{N}(0, I)$$

β (beta) is a small number controlling how much noise step t adds, and ε (epsilon) is the random noise. The network is trained on one simple regression: given the noisy image and the step number, predict the noise that was added.

$$L = \left\| \varepsilon - \varepsilon_\theta(x_t, t) \right\|^2$$

Generation starts from pure noise and runs the steps in reverse, each time subtracting the network's predicted noise. The GAN objective, by contrast, is a two-player game: the discriminator D maximizes its accuracy at telling real from generated, and the generator G minimizes it.

8.2 Mixture of experts and contrastive loss

In a mixture-of-experts layer, a router produces a weight for each of E experts and only the top k are run:

$$y = \sum_{i \in \text{top-}k} g_i(x)\text{Expert}_i(x)$$

The g values are softmax scores from the router, so the output is a weighted blend of a few experts' answers, and the others cost nothing for this token.

The contrastive loss behind models like CLIP treats, within a batch of N pairs, each image's own caption as the positive and the other N − 1 captions as negatives:

$$L = -\log \frac{e^{\text{sim}(a_i, b_i)/\tau}}{\sum_{j=1}^{N} e^{\text{sim}(a_i, b_j)/\tau}}$$

It is cross-entropy over "which caption belongs to this image", with cosine similarity as the score and a temperature τ (tau) as in section 7.3.

9.1 Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s,a)]$$

After taking action a in state s, receiving reward r and arriving in state s′, the agent compares its old estimate Q(s, a) with a better-informed one: the reward it just got plus the discounted value of the best move from where it landed. It moves its estimate part of the way toward that, by a step size α (alpha). γ (gamma), the discount factor, is a number just below 1. This is the cheese value seeping backwards through the maze, one junction at a time.

9.2 LoRA and preference learning

LoRA freezes a weight matrix W and learns a small correction written as the product of two thin matrices:

$$W' = W + BA, B: (d \times r), A: (r \times d), r \ll d$$

For a 4,096-by-4,096 matrix and a rank r of 8, the correction has about 65,000 trainable numbers instead of 16.8 million, a reduction of roughly 250 times.

In RLHF, a reward model r is trained from pairs where answer y_+ was preferred to y_-, so that the preferred answer scores higher with probability given by a sigmoid of the difference:

$$P(y_+ > y_-) = \sigma(r(x, y_+) - r(x, y_-))$$

The language model is then optimized to maximize r while a penalty keeps it close to the model it started from. DPO folds both steps into one, training the model directly on the preference pairs.

10.1 Regularization

L2 regularization adds a penalty for large weights to the loss, with a strength λ (lambda) that you choose:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_j w_j^2$$

The model now has to justify every large weight by a real reduction in error, the mathematical version of the reviewer who rejects complexity that does not pay for itself. L1 regularization uses absolute values instead of squares and tends to push unhelpful weights all the way to zero.

10.3 Metrics

With TP, FP and FN the counts of true positives, false positives and false negatives:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Precision divides by everything the alert flagged; recall divides by everything that was really there. For numeric predictions:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

The vertical bars in MAE mean “ignore the sign”. RMSE is the square root of MSE from section 1.2, which brings it back to the original units while keeping its extra sensitivity to large misses.

11.4 Differential privacy

A randomized computation M is ε-differentially private if, for any two datasets D and D′ that differ only in one person’s data, and any set S of possible results:

$$P(M(D) \in S) \leq e^{\varepsilon} P(M(D') \in S)$$

ε (epsilon) is the privacy budget. With a small ε, e^ε is close to 1, so any result is almost exactly as likely whether or not you were included, and nobody looking at it can tell much about you. Budgets add up: releasing k results computed with budget ε each costs at most kε in total, which is the arithmetic behind the warning in section 11.4 that every answer released spends a little of the budget.

Appendix B — Dictionary of Metaphors

Every picture in the book, with the technical term it stands for, so you can move between the two languages when reading documentation or talking with specialists.

Section	Picture	Technical term
1.1	The recipe and the apprentice	classic programming vs. machine learning
1.2	The answer key on the back of the page	supervised learning, labels, loss
1.3	Sorting the closet in the dark; pizza delivery stations	clustering, K-Means
1.3	The shadow on the wall; laptop spec sheets	dimensionality reduction, PCA (principal component analysis)
1.3	The face on Mars	clusters found in pure noise
1.4	Practice book, mock exam, real exam	training, validation and test sets
1.4	Hours of study vs. what is in your head	hyperparameters vs. parameters
2.1	The ruler on springs	linear regression, squared error
2.1	The dimmer switch	sigmoid, logistic regression
2.2	Tell me who your neighbors are; the sections of a library	k-nearest neighbors (k-NN); approximate nearest-neighbor index
2.2	In 500 dimensions everyone is a stranger	curse of dimensionality
2.3	Twenty questions, git bisect	decision tree, information gain
2.3	The sock drawer	entropy, Gini index
2.4	The ox at the country fair; the council of half-blind experts	random forest, bagging
2.4	The publishing house	gradient boosting
2.5	The road with the widest empty strip between two villages; the inner village turned into a valley	support vector machine (SVM), margin, kernel trick
2.5	The doctor’s suspicion before and after the evidence; the lazy doctor	Bayes’ rule, prior and posterior, naive Bayes
2.5	The music festival seen from a drone; the flat world map on the classroom wall; the flexible wire pinned at known points	density-based clustering (DBSCAN), t-SNE and UMAP, Gaussian processes
3.1	The apartment checklist; the paging rule	perceptron, weights, bias
3.1	Chained currency conversions	why networks need activation functions

Section	Picture	Technical term
3.2	Walking down in the fog; the flapping autoscaler	gradient descent, learning rate
3.2	The workshop inspector walking back along the line; distributed tracing	backpropagation
3.3	Inspectors with magnifying glasses	convolutional network, kernel
3.3	The linter rule applied to every line	weight sharing
3.4	The telephone game; a reduce with a small accumulator; one index card vs. a notebook with switches	recurrent network, hidden state, vanishing gradient, LSTM gates
4.1	The paper airplane and the pigeon; the sound desk with a million sliders	artificial vs. biological neurons
4.2	The folder of messy tickets; the row of checkboxes	data preparation, one-hot encoding, feature engineering
4.3	The file with a hundred million decimals	tensors, shapes, a model as stored weights
4.3	JPEG for weights	quantization
4.4	The four recruiters: scoring sheet, flowchart, gut feeling, eyes led by each CV	feature importance: linear weights, tree-based importance, hidden features, attention
4.4	Shuffling one feature’s values; the twins splitting the credit	permutation importance, correlated features
5.1	The box of Lego: workers and plumbing	layer types, architecture
5.1	The thermostat; the bypass; rehearsal sick days	normalization, residual connections, dropout
5.2	The year before opening; opening day	training vs. inference
5.2	Filling the bakery oven	batching
5.2	The bill for one conversation	inference cost, output vs. input tokens
5.3	Ten thousand coin flips; a grown-up Bloom filter	hyperdimensional computing, VSA (vector symbolic architectures)
5.4	The guitarist’s first year; ovens and flour; a photocopy of a photocopy	learning curve, scaling laws, compute-optimal training (Chinchilla), synthetic data and model collapse
6.1	The coffee cup in the dark; the scene graph	cortical columns, reference frames
6.1	Fingers voting; replicas reaching quorum	voting between columns

Section	Picture	Technical term
6.2	The fridge that stops humming; the video codec	predictive processing, prediction error
6.2	Each service tuning its own cache	Hebbian (local) learning
6.2	The write-ahead log compacted at night	hippocampal replay, complementary learning systems
7.1	The city map of meanings; color codes	embeddings
7.1	The pancake recipe for 4 and for 40	cosine similarity
7.2	The detectives' meeting; the fuzzy dictionary lookup	self-attention: query, key, value
7.3	The phone keyboard; guess the next word	next-token prediction, pretraining
7.3	The new hire: wiki, shadowing, code review	pretraining, fine-tuning, RLHF
7.3	The meeting-room whiteboard	context window
7.4	The open-book exam	retrieval-augmented generation (RAG)
7.5	Living on a whiteboard; the clerk who condenses old notes; the notebook carried between meetings	context window limits, context compaction, persistent memory
7.6	The new engineer guessing the next line of a postmortem; classical chess vs. blitz	next-token prediction, chain of thought, test-time compute
8.1	Thirty words over the phone	autoencoder
8.1	The child correcting a blob; the forger and the expert	diffusion models, GANs (generative adversarial networks)
8.2	The index built at write time; the triage desk; the REPL (read-eval-print loop)	two towers, mixture of experts, agents
8.3	The harbor master's tide table; pretending it is last January	time series, trend and seasonality, backtesting
8.3	Village gossip; the bookshop clerk	graph neural networks, message passing; recommender systems: retrieval and ranking
8.4	The sled; the runner's warm-up; the handrail	momentum, warmup, gradient clipping
8.4	A thermostat for the building vs. one in every room; the orchestra too large for one room	batch vs. layer normalization; data, tensor and pipeline parallelism

Section	Picture	Technical term
9.1	Clicker training; the cheese maze	reinforcement learning, Q-learning
9.1	The lunch dilemma	exploration vs. exploitation
9.1	The boat circling the lagoon	reward hacking
9.2	Medical school, residency, feedback; the translator's notebook; the leash	pretraining, transfer learning, fine-tuning, LoRA, RLHF, policy gradients and PPO, distillation
10.1	The crammer and the slacker	overfitting and underfitting
10.2	Clever Hans; tomorrow's newspaper	data leakage
10.3	The smoke alarm; the pager	precision and recall
10.4	The funhouse mirror; works on my machine	bias in data
10.5	The weather forecaster checked over a whole year	calibration, reliability diagram, temperature scaling
10.5	The fair coin; the toaster shown to a cat-and-dog classifier	aleatoric vs. epistemic uncertainty, out-of-distribution detection
10.6	The mechanic without a manual: look, poke, probe	interpretability, mechanistic interpretability
10.6	Splitting the restaurant bill; "approved if your income were higher"	SHAP (Shapley values), counterfactual explanations
11.1	The rooster and the sunrise; the coin that decides who gets the discount	correlation vs. causation, confounders, randomized controlled experiments (A/B tests)
11.2	The wine judges served the same wine three times; school league tables	LLM-as-a-judge, evaluation noise, Goodhart's law
11.3	The assistant who obeys every note on her desk	prompt injection, indirect prompt injection
11.3	Stickers on a stop sign; the bitter recipe card	adversarial examples, data poisoning, backdoors
11.4	The secret coin flip; the grandmothers' kitchens	randomized response, differential privacy, federated learning
11.4	The egg in the baked cake	machine unlearning
11.5	The restaurant a year after opening; the new flour; the garden that needs a gardener	MLOps, training-serving skew, drift monitoring
11.6	The island of California on old maps	forecasting AI progress, the AGI (artificial general intelligence) debate

Section	Picture	Technical term
11.6	King Midas's wish	alignment, reward hacking, goal specification
11.6	Twenty steps that each succeed nineteen times in twenty	agents, error compounding

Appendix C — Where to Go Next

When the apprentice of section 1.1 finally leaves the kitchen, what helps next is not another book about cakes but an oven, some flour and a few recipes worth getting wrong. This appendix is that kind of send-off: a short, opinionated list of well-known resources that have stayed useful for years, and a suggested order for using them. Nearly all of them are in English, and most are free. Pick one course, one book and one tool to start; collecting resources is not the same as learning from them.

Free courses

Andrew Ng's **Machine Learning Specialization** (Stanford and DeepLearning.AI, on Coursera) is the gentlest next step after this book, covering Chapters 1 to 3 again with the mathematics switched on; his **Deep Learning Specialization** continues from there. **Practical Deep Learning for Coders** from fast.ai (course.fast.ai) goes the other way round, top down: you train a working model in the first lesson and open the hood later. Stanford publishes lectures from three classic courses: **CS229** (machine learning, with the mathematics of Appendix A in full), **CS231n** (deep learning for computer vision, the territory of sections 3.3 and 8.1) and **CS224n** (natural language processing with deep learning, the territory of Chapter 7). Andrej Karpathy's **Neural Networks: Zero to Hero** builds backpropagation (section 3.2) and then a small GPT (section 7.3) from scratch, in code you type yourself.

Books

- *An Introduction to Statistical Learning*, by Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani, first written with examples in R and, since 2023, also in a Python edition; free online (statlearning.com). The best companion to Chapter 2.
- *Hands-On Machine Learning*, by Aurélien Géron. Earlier editions use Scikit-Learn, Keras and TensorFlow; the 2025 edition is built on Scikit-Learn and PyTorch. The most practical of the list.
- *Dive into Deep Learning*, by Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola; free and runnable at d2l.ai, every chapter a notebook.
- *Understanding Deep Learning*, by Simon Prince (2023), clear and well illustrated, with a free PDF.
- *Deep Learning*, by Ian Goodfellow, Yoshua Bengio and Aaron Courville (2016), free at deeplearningbook.org. Older, but still the standard reference for the foundations.

- *Pattern Recognition and Machine Learning*, by Christopher Bishop (2006), and *Probabilistic Machine Learning: An Introduction* (2022) with its *Advanced Topics* sequel (2023), by Kevin Murphy. Both take the probabilistic view of section 2.5 all the way down. Demanding; read them after one of the books above.
- *Reinforcement Learning: An Introduction*, by Richard Sutton and Andrew Barto (second edition, 2018), free online. Chapter 9's dog, maze and boat, done properly.

For the questions of the deeper sections and of the last chapter, three shorter books help: Christoph Molnar's *Interpretable Machine Learning* (free online) for section 10.6, Chip Huyen's *Designing Machine Learning Systems* for section 11.5, and Judea Pearl and Dana Mackenzie's *The Book of Why* for the rooster of section 11.1.

Tools to try

- **Jupyter** notebooks, or **Google Colab**, which runs them in a browser, often with a free graphics card for small experiments.
- **scikit-learn** (scikit-learn.org), for everything in Chapter 2 and most of sections 10.1 and 10.3; its user guide is a good textbook in disguise.
- **PyTorch** (pytorch.org), for building and training networks from Chapters 3 to 5.
- **Hugging Face** (huggingface.co), for downloading pretrained models and datasets, and for its free courses on language models.
- **Kaggle** (kaggle.com), for datasets, competitions and short tutorials, and for reading how other people solved the same problem.

A suggested order

Start where this book started. Take a small table of real data, apartment prices or support tickets, split it as section 1.4 taught, and train the ruler of section 2.1, a tree from section 2.3 and a boosted ensemble from section 2.4 in scikit-learn. Plot the learning curve of section 5.4 and see whether your model is the crammer or the slacker of section 10.1.

Then build a small network by hand, following Karpathy or the early chapters of *Dive into Deep Learning*, until backpropagation (section 3.2) stops feeling like magic. Fine-tune a pretrained image model on a few hundred photos of your own, as in section 9.2, and run the saliency check of section 10.6 to see what it looks at; you may find your own snow.

Next, language. Embed a few thousand of your own documents (section 7.1), build the open-book assistant of section 7.4 on top of them, and write a private evaluation set of fifty questions before you start tuning anything (section 11.2). If you are curious about agents, give a model one narrow, read-only tool and watch the loop of section 8.2 at work, with the permissions of section 11.3 in mind.

At each step, keep the habits of Chapters 1 and 10: a sealed test set, a check of the training and validation errors side by side, the metric that matches the mistakes your users can afford, and a look at the errors group by group. Compare every model with the simplest one that could possibly work, because sometimes, as section 11.5 put it, the right move is to write the rule. Everything else, as the preface promised, is the same loop, run again.